COMITE D'ETUDE ET D'EXPLOITATION
DES CALCULATEURS ELECTRONIQUES (C.E.C.E.), a.s.b.l.

31, rue Belliard, Bruxelles

Document n°1

Manuel de programmation

pour la machine mathématique IRSIA-FNRS

(VI + 157 pp.)

BRUXELLES

1957

org.
com. tamb. 0006.
transf. rub. A sur tamb.
dér. A fin rub.
$\alpha = Z_3$.
org transf. ~~S~~ 1 mot ss. piste aux.
org. tamb. 0005.
transf. F 1 mot.
~~$\alpha + Z_1 = Z_4$.~~
$0001 = Z_4$.
Rv inh f.t. 0040.
$0000 = Z_4$.
$+ * \hat{V}$
$= Z_1$.
$+ 0R_1 . (Z_1 + 0000)$.
$- 0R_1 . F$.
$+ 0R_2 (Z_1 + 000)$.
$- 0R_2 \; F$.
$= E$.
$-$ mod.
$+ E$.
$=$ ch
Rv 0090.
$- * \hat{Z}_4$
$=$ ch.
Rv 30.
$- * \hat{Z}_3$
$=$ ch.
Rv 95
$Z_3 = 0001$.
com. tamb 0006.
~~$\Rightarrow$~~ Rv 30

90	0006	
54		
92		
61	0000	3
95	0004	~~1~~
90	0005.	
05	1000.	
52	~~000~~	
61	0001	4
48	2000	
61	0000	4
05	2000.	
06	0006	
62	0000	1.
01	0000	16.
02	0002	06.
01	0000	17
02	0002	07
10	0001.	
28		
01	0001.	
~~10~~	~~0004~~	
45	3000	
17	0000	4
45	1000	
17	0000	3
45	4000	
61	0001	1.
90	0006.	
40	1000	
05	3000.	
47		
05	4000.	

alarme vide.

Avant-propos

Le présent manuel a été rédigé par V. Belevitch, en collaboration avec F. Dagnelie et J. Meinguet et vérifié par Melle M. Léonard. Il fournit tous les renseignements nécessaires aux utilisateurs pour pouvoir composer des programmes et les faire exécuter par la machine dans les conditions normales d'exploitation automatique. Un second volume, en préparation, sera consacré à l'exploitation pas-à-pas, et à l'introduction manuelle de données ou d'instructions dans la machine, notamment en vue de la localisation des fautes.

Le premier chapitre décrit les procédés suffisants pour préparer la programmation de la plupart des problèmes usuels. Les problèmes exigeant le calcul à précision multiple, ou ceux dont les données numériques sont si abondantes qu'elles demandent le recours aux rubans magnétiques pendant le calcul font l'objet des chap. II et III. L'inscription initiale des données et l'impression des résultats se font normalement par l'intermédiaire des rubans et sont aussi discutées au chap. III. Le chapitre IV, traitant de la programmation des fonctions élémentaires les plus usuelles, constitue une illustration des méthodes décrites dans les chapitres précédents. Le dernier chapitre décrit l'organisation de l'exploitation pour les problèmes standard programmés une fois pour toutes sur un ruban de routines.

La liste des ordres usuels est résumée dans l'appendice III pour permettre une consultation commode. Les équations et figures sont numérotées en recommençant à chaque chapitre.

Table des matières

Chap.I. Calculs à virgule flottante

1. La notion de programmation à simple adresse 1
2. Virgule flottante. Notation des nombres 3
3. Ordres d'opération 3
   3.1. Préfixes d'opération
   3.2. Addition et soustraction
   3.3. Addition et soustraction sans normalisation
   3.4. Multiplication
   3.5. Multiplication à facteur constant
   3.6. Addition de zéro, avec ou sans normalisation
   3.7. Multiplication par l'unité
   3.8. Précautions à prendre pour l'entrée de nombres non normalisés
   3.9. Temps d'opération
4. Ordres d'inscription 9
5. Altérations 10
   5.1. Généralités
   5.2. Préfixes d'altération
   5.3. Remarques
   5.4. Altération noex
   5.5. Altération exno
6. Mémoire des ordres. Renvois 13
7. Bifurcations dans le programme = renvois cond 14
8. Opérations sur les adresses 18
   8.1. Registres d'indices
   8.2. Notations
   8.3. Modifications d'adresses 19 p.19
   8.4. Envoi d'un nombre en registre d'indice
   8.5. Utilisation de l'adresse comme nombre
   8.6. Le registre M

9. Programmes cycliques 24

9.1. Comptage des cycles

9.2. Reprise conditionnelle avec nombre maximum de reprises

10. Contrôles 26

11. Alarmes 27

11.1. Alarme du groupe calculateur

11.2. Alarme de programme

12. Vidanges 29

13. Instructions et notations pour la programmation 30

Chap.II. Virgule fixe, haute précision et calcul sur les ordres

1. Calculs à virgule fixe 31

2. Calculs sur les ordres 33

3. Multiplication sans arrondi 35

4. Calculs à haute précision 37

4.1. Addition et soustraction

4.2. Multiplication

Chap.III. Utilisation des rubans. Périphériques d'entrée et de sortie

1. Généralités 41

2. Notations relatives aux registres d'indices 42

3. Ordres d'organisation des rubans et des tambours 42

4. Repères sur les rubans 44

5. Transferts 45

5.1. Déroulements

5.2. Organisation du transfert

5.3. Ordres de transfert

5.4. Transfert dans les registres E et F

6. Registres associés aux rubans 48

7. Calculs sur les rubans 49

7.1. Ordres d'opération

7.2. Ordres d'inscription

8. Impression 50

8.1. Impression des rubans de programmes ou de données

8.2. Impression des résultats

8.3. Impression sans passer par le ruban de résultats

9. Pupitres 54

9.1. Généralités

9.2. Synchronisation

9.3. Autorisations d'inscription

9.4. Commandes manuelles

9.5. Pupitres d'impression

10. Préparation des rubans 62

10.1. Généralités

10.2. Choix du ruban et préparation de l'inscription

10.3. Inscription des mots-ordres

10.4. Inscription des mots-nombres

10.5. Impression d'un ruban pour contrôle

10.6. Correction des erreurs

11. Transfert et exécution d'un programme 65

11.1. Généralités

11.2. Démarrage

11.3. Programme de démarrage

11.4. Programme de transfert et de contrôle

12. Programme d'impression 71

12.1. Généralités

12.2. Préparation du ruban définitif

12.3. Contrôle du ruban définitif

13. Le registre V 76

Chap. IV. Programmes des fonctions élémentaires

1. Généralités 77

2. Ruban des fonctions élémentaires 77

3. Instructions d'emploi 82

4. Remarques sur la structure des séquences 83

Chap. V. Compilation

1. Définition 85

2. Attribution de rubans 85

3. Structure du ruban de routine et modification à prévoir 86
4. Routines initiale et finale 88
5. Exemple de compilation 93

Appendice I. Information notée sur le ruban des fonctions élémentaires 95

Appendice II. Analyse des séquences des fonctions élémentaires

1. Programme de transfert et de contrôle 111
2. Séquence de l'inverse 112
2.1. Structure de la séquence
2.2. Détail des ordres
3. Fonction $y = x^{\frac{1}{2}}$ 115
3.1. Réduction à l'intervalle (0,1 - 1)
3.2. Structure de la séquence
3.3. Détail des ordres
4. Séquence $y = \sin x$ 121
4.1. Structure de la séquence
4.2. Détail des ordres
5. Séquence $y = \mathrm{Arc\ tg}\ x$ 126
5.1. Structure de la séquence
5.2. Détail des ordres
6. Séquences $y = 10^x$ et $z = 10^x - 1$ 133
6.1. Réduction à l'intervalle de base
6.2. Remarques sur la structure
6.3. Détail des ordres
7. Séquences $\log x$ et $\log(1+y)$ 143
7.1. Remarques générales
7.2. Détail des ordres

Appendice III. Liste récapitulative des ordres 152

pour doc ode !
~ calcul de la p. 13 !
pour ...

- que signifie les flèches ?

- calcul d'adr =
{ Commentaires
{ Note "

→ organ précédent ?
est

p. 57 Liste ...

Rubans [magnétiques]

résultats intermédiaires

! ne parle pas

- des I/O

- de la mémoire des instructions
voir p. 13

[de l'horloge interne]

Voc. ~ architecture p1

1. mémoire de nombre :
→ données nu
et résultat

2. Unité arithmétique (groupe calcul.
{ opère sur données → ...
{ → ...

3. Contrôle
Commande
organise le trafic selon
les instructions

4 (instruct.) (ordre) ... → program

---

p1

Aucune mémoire des nombres

Lecture

inscription ([= écriture]

---

p1

1 adresse de la mémoire des nombres
(tambour.

= 4 chiffres

$\underbrace{\times\times}_{\text{numéro de pist.}}$ $\underbrace{\times\times}_{\text{Cond. ✗}}$

00→99 00→95 par pas de 5

Δ 100 pistes Δ 20 secteurs

Chapitre I : Calculs à virgule flottante

1. La notion de programmation à simple adresse

Toute machine mathématique comporte essentiellement une mémoire pour la conservation des données numériques et des résultats (mémoire de nombres), une unité arithmétique (ou groupe calculateur) qui opère sur des données extraites de la mémoire et renvoie les résultats en mémoire, et des circuits de commande qui organisent le trafic selon une suite d'instructions opératoires (ou ordres) constituant le programme.

Les ordres du programme ne réfèrent pas aux nombres eux-mêmes, mais à leurs adresses dans la mémoire de nombres, c.à.d. aux numéros des cases de mémoire dont les données doivent être extraites ou vers lesquelles les résultats doivent être envoyés.

La mémoire de travail pour les nombres est un tambour magnétique de 100 pistes numérotées de 00 à 99, chaque piste étant divisée en vingt secteurs numérotés de 00 à 95 par pas de 5. Une adresse se compose de la coordonnée longitudinale (numéro de piste) suivie de la coordonnée angulaire (numéro de secteur) et est donc un nombre de quatre chiffres dont le dernier est 0 ou 5. Une adresse complète est donc un nombre qui varie de 0000 à 9995 par pas de 5, et les adresses forment une boucle modulo 10 000. Le même pas de 5 qui mène d'un secteur au suivant sur la même piste, mène aussi du dernier secteur d'une piste au premier secteur de la piste suivante, et de la dernière à la première case du tambour. Le temps d'accès de la mémoire, qu'il s'agisse d'une lecture ou d'une inscription, varie de 0 à 14 ms selon la position angulaire du tambour au moment de l'interprétation de l'ordre de sélection, et est donc de 7 ms (temps d'une demi-révolution du tambour) en moyenne. Le temps de lecture ou d'inscription proprement dit est de 0,7 ms.

Le groupe calculateur comporte une entrée unique et une sortie unique, et conserve dans sa mémoire interne le résultat du calcul qu'il vient d'effectuer. Dans ce qui suit, ce résultat sera toujours noté par ω.

La machine est à simple adresse, ce qui signifie que tout ordre se compose d'un préfixe opératoire suivi d'une seule adresse. P.ex., dans l'ordre $+\alpha$, le symbole opératoire + est représenté par un préfixe, selon un code numérique

(toujours cinq chiffres décimaux) qui sera précisé plus loin, et est suivi de l'adresse α (toujours quatre chiffres décimaux). L'effet de cet ordre est d'extraire la donnée numérique qui est en mémoire à l'adresse α, de l'amener à l'entrée du groupe calculateur, de l'additionner au résultat précédent ω, et de conserver le nouveau résultat $\omega' = \omega + \alpha$ dans le groupe calculateur. L'effet de l'ordre $+\alpha$ sera donc noté symboliquement par $\omega + \alpha = \omega$. Un tel ordre, comportant la sélection d'une donnée, et sa combinaison avec un résultat précédent contenu dans le groupe calculateur selon une opération notée dans le préfixe, s'appelle ordre d'opération.

L'envoi d'un résultat ω du groupe calculateur vers la mémoire de nombres à une destination définie par son adresse γ sera noté $= \gamma$, ou $\rightarrow \gamma$, selon que ω est effacé ou non dans le groupe calculateur. De nouveau, le symbole = ou $\rightarrow$ est représenté comme préfixe selon un code numérique. Un tel ordre, ayant pour effet d'inscrire un résultat dans la mémoire, s'appelle ordre d'inscription. La machine est conçue de façon que l'envoi d'une information, vers un registre ou une mémoire qui en contenait déjà une autre précédemment, détruise automatiquement l'information ancienne.

Après exécution de l'ordre = (inscription avec effacement), on a $\omega = 0$ dans le groupe calculateur. On pourra donc entrer une donnée fraîche par l'ordre $+\alpha$ (qui effectue $0 + \alpha$), et une succession d'ordres du type

$$+a \mid \times b \mid + c \mid = d \mid + e \mid \times f \mid - g \mid = h$$

donne

$$ab + c = d \quad ; \quad ef - g = h$$

Des résultats intermédiaires qui interviennent fréquemment dans le calcul peuvent être mis en mémoire dans deux registres électroniques appelés E et F; ceci permet de gagner du temps car le temps d'accès à ces registres est nul. Les ordres d'opération et d'inscription relatifs à ces registres sont notés sous la forme $+E$, $=F$, etc , et sont représentés par des préfixes spéciaux du code des ordres.

2. Virgule flottante. Notation des nombres

La machine est en principe conçue pour le calcul à virgule flottante où tout nombre est noté sous la forme

$$\pm \underbrace{abcd...}_{15 \text{ chiffres}} \quad \pm pq \qquad (1)$$

signifiant

$$\pm\, 0,\ abcd...\ 10^{\pm pq} \qquad (2)$$

La mantisse se compose de 15 chiffres décimaux, la virgule se trouvant en tête. On peut toujours s'arranger pour que le premier chiffre a soit différent de zéro (en décalant la mantisse vers la gauche et en corrigeant l'exposant); lorsque le nombre est sous cette forme, on dit qu'il est normalisé. L'exposant se compose de deux chiffres et est limité à 49, de façon qu'il n'y ait jamais de report au troisième rang lors d'une addition de deux exposants, ce qui se produirait pour 50 + 50. De cette manière, le résultat sera toujours correct. Une alarme (dépassement d'exposant) est déclenchée lorsqu'on donne un ordre d'opération introduisant dans le groupe calculateur une donnée fraîche dont l'exposant dépasse 49 en module (qui aurait pu être introduite par mégarde dans la mémoire de nombres, car l'opérateur peut inscrire des exposants jusque 99), ou lorsque le résultat d'un calcul en cours engendre un tel exposant. Dans ce cas, la machine achève l'opération en cours, puis s'arrête.

Il est également possible de travailler à virgule fixe, ce qui est nécessaire notamment pour les calculs à double précision où deux cases de mémoire sont réunies pour noter un nombre unique à 30 décimales. La programmation est cependant beaucoup plus compliquée dans ce cas, qui fera l'objet d'un chapitre séparé. Normalement, et jusqu'à nouvel ordre, la machine fonctionne à virgule flottante.

3. Ordres d'opération

3.1. Préfixes d'opération

Tout ordre se compose d'un préfixe (5 chiffres) suivi d'une adresse (4 chiffres). Pour les ordres d'opération, la liste des préfixes est donnée

au tableau ci-dessous en regard du symbole opératoire correspondant (voir paragraphes suivants pour l'explication des symboles inhabituels). La dernière colonne indique l'opération effectuée lors de l'exécution de l'ordre, avec les notations expliquées au § 1, et en notant toujours par α la donnée fraîche introduite dans le groupe calculateur, qui se trouvait en mémoire sur le tambour à l'adresse α mentionnée dans l'ordre.

préfixe	adresse	symbole	signification
04016	α	$+\alpha$	$\omega + \alpha = \omega$
04026	α	$-\alpha$	$\omega - \alpha = \omega$
04066	α	$+^*\alpha$	$\omega + \alpha = \omega$ sans norm.
04076	α	$-^*\alpha$	$\omega - \alpha = \omega$ sans norm.
04036	α	$\times\alpha$	$\omega \times \alpha = \omega$
04035	α	$\times(-\alpha)$	$\omega \times (-\alpha) = \omega$
04086	α	$\circledast\alpha$	$\beta \times \alpha = \omega$
04085	α	$\circledast(-\alpha)$	$\beta \times (-\alpha) = \omega$

En remplaçant la tête 040 d'un de ces préfixes par 021, et en laissant l'adresse vide (0000), on extrait la donnée fraîche non de la mémoire de nombres, mais de la mémoire électronique E. De même, si la tête du préfixe est remplacée par 020, on fait intervenir le registre F. La condition d'adresse vide n'est en fait pas strictement nécessaire : on peut laisser une adresse quelconque car celle-ci n'est pas pertinente pour l'exécution de l'ordre. En résumé :

020**	préfixe d'opération cherchant la donnée en F
021**	préfixe d'opération cherchant la donnée en E

3.2. Addition et soustraction

Le dernier résultat en mémoire dans le groupe calculateur, ω, étant normalisé (pas de zéro en tête de la mantisse) ou non normalisé, l'addition et la soustraction avec normalisation permettent de lui additionner ou d'en

soustraire un nombre quelconque (normalisé ou non) et de former le nouveau résultat sous forme normalisée; la machine exécute alors automatiquement les processus suivants :

(1) Si les deux nombres ont des exposants différents, la machine aligne le nombre affecté du plus petit exposant sur celui affecté du plus grand exposant (même si l'un des deux termes a des zéros en tête); la queue du premier est donc perdue.

(2) La machine effectue l'addition ou la soustraction, compte tenu des signes. Si l'opération est une addition proprement dite, il peut y avoir un report de 1 en tête de la mantisse; pour pouvoir noter ce 1, le groupe calculateur décale la mantisse d'un rang vers la droite (en laissant tomber le dernier chiffre) et augmente l'exposant d'une unité.

(3) Si l'opération est une soustraction proprement dite, ou si le nombre à ajouter ou soustraire n'est pas normalisé, le résultat peut comporter des zéros en tête de la mantisse. Le groupe calculateur le normalise en le décalant vers la gauche d'autant de rangs qu'il y avait de zéros et en injectant des zéros comme chiffres de queue. L'exposant est diminué d'autant d'unités qu'il y avait de zéros.

Deux cas particuliers sont à considérer :

(1) Après exécution d'un ordre d'inscription avec effacement (=), le groupe calculateur contient zéro sous la forme $+ ,00\ldots 0.10^{+00}$. Une opération d'addition ou de soustraction ultérieure, entrant une nouvelle donnée, est effectuée sans alignement préalable sur 10^{00}, pour ne pas perdre les chiffres de queue si l'exposant du nombre est négatif. Le résultat est cependant normalisé, de manière à supprimer éventuellement les zéros de tête de la mantisse.

(2) Si le résultat d'une addition ou d'une soustraction est zéro, celui-ci est noté avec le signe du résultat du calcul précédent et est accompagné de l'exposant du nombre additionné ou soustrait. Par exemple, si le groupe calculateur est initialement vide et si a est un nombre positif, $+ a \mid - a$ donne $+ 0$, tandis que $- a \mid + a$ donne $- 0$; dans les deux cas, l'exposant est celui de a. De plus, une addition ou une soustraction ultérieure est effectuée sans alignement préalable, comme dans le cas précédent. En particulier, si le groupe calculateur est initialement vide et si l'on ajoute ou soustrait $\pm 0.10^{n}$, le résultat est $+ 0.10^{n}$.

3.3. Addition et soustraction sans normalisation

Pour certaines applications, les chiffres du résultat doivent occuper une place correspondant à leur rang, ce qui revient à imposer une valeur déterminée à l'exposant. Il peut alors y avoir des zéros en tête et on doit bloquer la normalisation. On utilise les opérations d'addition et de soustraction sans normalisation (notées par un astérisque dans les programmes). Le résultat est alors affecté de l'exposant sur lequel on a fait l'alignement initial, éventuellement augmenté d'une unité s'il y a eu un report en tête.

Les règles données au § 3.2 à propos des inscriptions avec effacement et des résultats nuls sont encore valables pour les opérations sans normalisation.

3.4. Multiplication

La multiplication est effectuée dans le groupe calculateur à peu près comme elle le serait à la main. Le multiplicateur est alors le premier facteur introduit, c'est-à-dire ω dans $\omega \times \alpha$. Le groupe calculateur est toujours organisé de façon à garder indéfiniment en mémoire tout multiplicateur jusqu'à ce que le programme amène une multiplication ultérieure : à ce moment, le nouveau multiplicateur prend la place du précédent en effaçant celui-ci.

Si p et q sont les exposants des facteurs, l'exposant du résultat est $p + q - 1$ s'il n'y a pas eu de report de tête, ou $p + q$ s'il y a eu un report de tête. On en déduit que si les deux facteurs sont normalisés, le produit le sera aussi. Si un au moins des deux facteurs n'est pas normalisé, le produit est encore correct et l'exposant du résultat est toujours $p + q - 1$.

3.5. Multiplication à facteur constant

Le symbole ⊗ représente la multiplication à facteur constant. Dans ce cas, l'ancien multiplicateur est conservé pour plusieurs opérations.

L'exemple type de son emploi est le cas où on a à multiplier une série de nombres a_i par un facteur constant c de façon à obtenir une série de résultats $b_i = ca_i$. Sans l'opération $\otimes$ on devrait donner la succession d'ordres

$$+ c \mid \times a_1 \mid = b_1 \mid + c \mid \times a_2 \mid = b_2 \mid + c \mid \times a_3 \mid = b_3 \mid \ldots$$

tandis qu'en fait on écrira

$$+ c \mid \times a_1 \mid = b_1 \mid \otimes \quad a_2 \mid = b_2 \mid \otimes a_3 \mid = b_3 \mid \ldots$$

et le facteur constant (arbitrairement désigné par β au § 3.1.) est toujours celui sélectionné dans l'ordre précédant immédiatement le dernier ordre de multiplication ordinaire. Il est bien entendu que la multiplication spéciale $\otimes$ n'a de sens que suivant immédiatement un ordre d'inscription : elle n'utilis pas ω (résultat de l'opération précédente) qui est donc perdu (chassé par le nouveau résultat), et il n'aurait servi à rien de le calculer sans l'inscrire.

3.6. Addition de zéro avec ou sans normalisation

Une addition de zéro telle que

$$\begin{array}{lll} & ,\ abcd\ldots & 10^p \\ +^* & ,\ 0000\ldots & 10^q \end{array}$$

est sans effet si $q \leqslant p$. Au contraire, si $q > p$, les $q-p$ chiffres de queue de $,\ abcd\ldots$ sont coupés par alignement sur le zéro, et le résultat est

$$,\ \underbrace{00\ldots}_{q-p}\ abcd\ldots \quad 10^q \qquad (3)$$

Si l'addition est avec normalisation, (3) se transforme en

$$,\ abcd\ldots\ \underbrace{000\ldots}_{q-p} \quad 10^p \qquad (4)$$

En conclusion, pour couper les n derniers chiffres d'un nombre, il faut lui ajouter zéro affecté de l'exposant $q = n + p$. Cela suppose donc qu'on connaisse l'exposant p du nombre. Ceci n'est plus requis si l'on désire couper les chiffre de queue d'un nombre à partir d'un certain rang absolu, en particulier si l'on veut calculer la partie entière d'un nombre. Si l'exposant est p, la partie

entière se compose des p premiers chiffres, et il faut couper les 15-p derniers, donc prendre q = 15. On a donc

$$\text{partie entière de } x = x + 0.10^{15} \qquad (5)$$

3.7. Multiplication par l'unité

Pour décaler une mantisse de n rangs vers la droite, donc transformer $,abc\ldots\ 10^{p}$ en

$$,\underbrace{000\ldots}_{n}\ abc\ldots\ 10^{p+n} \qquad (6)$$

ce qui ne change pas la valeur de l'expression, mais en coupe les n derniers chiffres, il suffit de multiplier le nombre original par

$$,\underbrace{00\ldots0}_{n}\ 10000\ldots\ 10^{n+1} \qquad (7)$$

c.à.d. en fait par l'unité. La différence entre ce porcédé et celui du § 3.6 est que dans (3) intervient l'exposant p du nombre à décaler, exposant qu'il faut connaître si l'on veut un décalage donné. Au contraire, pour obtenir des décalages relatifs (p.ex. dans le cas du calcul de la partie entière), la méthode du § 3.6 est toute indiquée.

3.8. Précautions à prendre avec les nombres non normalisés

Dans les décalages et amputations étudiés aux paragraphes précédents, interviennent des nombres valant 0 ou 1 notés avec des exposants bien définis. Certaines précautions sont indispensables pour ces opérations :

(1) L'addition de zéro n'est efficace que si le nombre auquel elle s'applique a été introduit précédemment dans le groupe calculateur. Par exemple, si le groupe calculateur est initialement vide, la suite d'opérations $+\ 0.10^{q}$; $+\ \underline{a}$ est toujours sans effet car le résultat de la première addition étant nul, on bloque l'alignement initial de l'addition suivante.

(2) Pour la multiplication par l'unité, si le nombre auquel s'applique l'opération est un résultat de calcul, il suffit d'effectuer $\times\ (+,0\ldots010\ldots0.10^{n+1})$. Si le nombre est en mémoire, on a intérêt à effectuer $+^{*}\ (+,0\ldots010\ldots0.10^{n+1})$; $\times\ \underline{a}$, (§3.9 : gain de temps), mais l'addition de l'unité doit se faire sans normalisation pour conserver la forme du nombre.

FUAA!

3.9. Temps d'opération

Le temps d'opération pour une addition ou une soustraction avec ou sans normalisation est de 4,1 ms. Le temps de la multiplication est de 13,3 ms, mais on gagne 0,75 ms par zéro dans la mantisse du multiplicateur (sauf pour le dernier chiffre, même s'il est zéro). Si a contient plus de zéros que b, il y a donc avantage à faire $+ a \mid \times b$ plutôt que $+ b \mid \times a$; ceci est particulièrement important pour la multiplication par 1 au §3.7.

Le temps de sélection moyen sur tambour de 7 ms (§1) chevauche en tout ou en partie avec le temps d'opération de l'ordre précédent.

4. Ordres d'inscription

— MC ou vers registres

Ils sont composés d'un préfixe, suivi d'une adresse que nous désignerons par γ. Les préfixes sont :

préfixe	adresse	symbole	signification
09096	γ	$\rightarrow \gamma$	$\omega \rightarrow \gamma$ (ω reste)
09046	γ	$= \gamma$	$\omega = \gamma$ (ω effacé)

En remplaçant la tête 090 du préfixe par 071, on envoie le résultat non en mémoire de nombres, mais vers la mémoire électronique E. De même, l'inscription se fait dans le registre F si la tête du préfixe est 070. L'adresse est non pertinente dans ces cas et sera normalement notée 0000. En résumé :

070**	préfixe d'inscription envoyant le résultat en F
071**	préfixe d'inscription envoyant le résultat en E

Lorsque l'inscription de résultats se fait sur une même piste du tambour, c'est-à-dire que les deux premiers chiffres de l'adresse sont inchangés par rapport au dernier ordre d'inscription exécuté, le temps d'inscription moyen est de 8 ms, mais ce temps peut chevaucher en tout ou en partie avec un temps

d'opération. Lorsqu'on change de piste, il s'ajoute un délai supplémentaire de 25 ms; il est donc très important de noter les résultats piste par piste autant que possible.

5. Altérations

5.1. Généralités

On appelle altération une opération portant sur un seul nombre, et modifiant, ou permutant entre elles, ses différentes parties (signes, mantisse, exposant). Un ordre d'altération se compose d'un préfixe suivi d'une adresse vide. L'ordre d'altération est un ordre d'organisation du groupe calculateur. L'altération porte toujours sur la donnée fraîche introduite dans le groupe calculateur par l'ordre suivant immédiatement, et est effectuée à l'entrée de ce nombre dans le groupe calculateur. L'opération arithmétique notée dans le deuxième ordre est effectuée ensuite sur le nombre altéré. Ainsi la suite d'ordres

$$+ a \mid + \text{sgn} \mid \times b \mid = c$$

signifie

$$c = a \text{ sgn } b$$

tandis que

$$+ a \mid + \text{sgn} \mid + b \mid = c$$

signifie

$$c = a + \text{sgn } b$$

L'exécution d'un ordre d'altération ne prend pas de temps supplémentaire par rapport à une entrée non altérée. Cela provient de ce que les altérations sont simplement préparées par l'ordre d'altération, et s'effectuent à l'entrée du groupe calculateur au début de l'exécution de l'ordre suivant.

5.2. Préfixes d'altération

La liste en est donnée ci-dessous; dans la dernière colonne on a représenté l'effet de l'altération sur un nombre du type

$$s \text{ abcd...} \; ; \sigma \text{ pq} \qquad (8)$$

où s est le signe de la mantisse et σ celui de l'exposant.

préfixe	symbole	effet
00012	+ man	+ abcd... ; + 00
00062	- man	- abcd... ; + 00
00022	+ sgn	+ s 1000... ; + 01
00072	- sgn	- s 1000... ; + 01
00032	+ mod	+ abcd... ; σ pq
00082	- mod	- abcd... ; σ pq
00042	+ exp	+ 1000... ; σ pq
00092	- exp	+ 1000... ; - σ pq
00013	ncex	+ 1000... ; s ab
00023	exno	σ pq00... ; + 02

5.3. Remarques

L'altération mantisse (man) remplace l'exposant par zéro et prend le module de la mantisse en lui attribuant le signe spécifié; si elle porte sur un nombre normalisé, le résultat est donc ramené dans l'intervalle 0,1 - 1.

L'altération sgn est la signature définie par

$$\operatorname{sgn} x = \begin{cases} +1 & \text{pour } x > 0 \\ -1 & \text{pour } x < 0 \end{cases}$$

et avec la convention $\operatorname{sgn}(\pm 0) = \pm 1$.

L'altération mod x est le module de x au sens usuel.

L'altération exp remplace la mantisse d'un nombre par 0,1 (et non par l'unité) et conserve sa caractéristique exponentielle. Elle donne donc l'ordre de grandeur du nombre.

5.4. Altération ncex

Elle transforme en exposant les deux premiers chiffres d'une mantisse, le signe étant également amené vers l'exposant. Pour des nombres entiers, cette opération donne la fonction exponentielle en base 10 moyennant certaines précautions de virgule. L'exposant est en effet noté à virgule fixe, et seul un

nombre inférieur à 50 peut être transformé en exposant. Si un tel nombre comporte deux chiffres significatifs, p.ex. s'il s'agit de 32, il est en fait noté $,3200... 10^{02}$ et l'altération noex le transformera en $,100... 10^{32} = 10^{31}$, et on a

$$\text{noex}(x) = 10^{x-1} \qquad (9)$$

Si le nombre ne comporte qu'un chiffre, p.ex. s'il s'agit de 7, il est en fait noté $,7000... 10^{01}$ et l'altération le transformerait en $,1. 10^{70}$. Pour avoir un résultat semblable à (9) il faut au préalable transformer $,7.10^{01}$ en $,0700... 10^{02}$, ce qui est possible au moyen de l'addition de 0.10^{02} (cfr §3.6) sans normalisation. Comme une telle addition est sans effet dans le cas de notre premier exemple, on obtient 10^{x-1} dans tous les cas par la suite d'ordres

$$+ x \,|\, + 0.10^{02} \,| = E \,|\, \text{noex} \,| + E \qquad (10)$$

où E est le registre électronique employé comme mémoire de réserve pour le résultat intermédiaire.

Le résultat est correct pour tout entier x compris entre -99 et +99 mais donne l'alarme de dépassement d'exposant pour $|x| > 49$. Pour x non entier, la suite d'ordres (10) donne encore 10^{x-1} mais avec remplacement de x par sa partie entière, si $|x| < 100$. Pour $|x| > 100$ le résultat est franchement incorrect et aucune alarme ne se produit. La comparaison est donc à faire au préalable.

5.5. Altération exno

Elle transforme l'exposant (avec son signe) en nombre. Si ~~le nombre x sur lequel elle opère est normalisé,~~ on a $x = N. 10^p$ ~~avec $0,1 < |N| < 1$, et~~ on a $\text{exno}(x) = p$.

Il est à remarquer que le résultat de l'altération n'est pas normalisé (bien que correct), si le premier chiffre de l'exposant de x était 0; c'est sous cette forme qu'il est combiné avec le dernier résultat en mémoire dans le groupe calculateur, par l'ordre d'opération qui suit l'ordre d'altération; si cet ordre d'opération est une addition ou une soustraction ordinaire, le résultat final sera normalisé; si c'est un ordre d'addition ou de soustraction sans normalisation ou une multiplication, le résultat peut ne pas être normalisé, de sorte que des précautions sont nécessaires.

Lorsque l'exposant du nombre x est 00, son signe dépend de la manière dont cet exposant a été obtenu et doit être considéré par le calculateur comme indéterminé. Le résultat de l'opération qui accompagne l'altération est cependant toujours correct, avec la limitation que si ce résultat est lui-même nul, son signe n'est pas connu. Prenant les logarithmes décimaux, il vient $\log|x| = p + \log|N|$ avec $-1 < \log|N| < 0$, et p-1 est la partie entière de log x On a donc toujours correctement

$$\text{exno}(x) = 1 + \text{partie entière de } \log_{10}|x| \qquad (11)$$

6. Mémoire des ordres . Renvois.

Les ordres composant le programme sont eux-mêmes en mémoire dans la machine, et sont rangés dans l'ordre naturel où ils seront exploités. La mémoire d'ordres est un tambour magnétique identique à la mémoire de nombres et est divisée en cases auxquelles on attribue des adresses selon le même schéma que pour les nombres. Comme un ordre se compose de 9 chiffres (5 pour le préfixe et 4 pour l'adresse), tandis que la longueur d'un nombre équivaut à 18 chiffres (15 pour la mantisse, 2 pour l'exposant, 1 pour les signes), on peut noter une paire d'ordres dans une case de mémoire et caractériser la position de cette paire par une adresse unique.

En fait le tambour de gauche et de droite peuvent indifféremment servir à noter des nombres ou des paires d'ordres (nous dirons mots lorsque la nature de l'information n'est pas spécifiée); la spécialisation usuelle (tambour "0" pour les ordres et tambour "1" pour les nombres) est notifiée une fois pour toutes à la machine par l'envoi au début du programme d'un ordre d'organisation composé du préfixe 90000 suivi de l'adresse 0006.

Lorsqu'on désire faire un saut dans le programme et ne pas l'exploiter dans l'ordre naturel, on donne un ordre de renvoi comportant un préfixe différent de ceux étudiés jusqu'ici. L'adresse qui suit ce préfixe est alors celle de la nouvelle paire d'ordres à laquelle on veut renvoyer, et par l'exécution du renvoi le programme saute au premier des ordres de la paire vers laquelle on a renvoyé. Il est impossible de renvoyer vers le second ordre d'une paire.

A la fin d'un programme de calcul, il faut un ordre de fin pour arrêter la machine.

On a donc les trois nouveaux ordres :

préfixe	adresse	symbole	signification
90000	0006		Organis. normale des cylindres
40000	λ	Rv λ	Renvoi à l'adresse λ
42000	0000	fin	Ordre de fin

7. Bifurcations dans le programme

La plupart des problèmes mathématiques contiennent une ou plusieurs bifurcations dans leur programme. Une bifurcation (ou aiguillage) survient lorsque la continuation de la marche à suivre à un moment donné dépend des résultats obtenus antérieurement. Un aiguillage vers plus de deux directions peut se traiter comme une succession de bifurcations, et il suffit donc de considérer un aiguillage à deux branches. La poursuite du programme vers l'une ou l'autre des deux branches peut se symboliser par un signe $\pm$, fonction de résultats antérieurs. On peut toujours s'arranger pour que ce signe soit celui d'un nombre, résultat d'un calcul plus ou moins compliqué.

La fig.1. représente schématiquement un programme avec aiguillage après A. Dans la séquence A du programme, on obtient par le calcul le nombre x dont le choix décidera de la continuation vers B ou C, et on envoie le signe de x dans une mémoire spéciale, dite registre de choix. On note, d'autre part, à la fin de la séquence A, un ordre de renvoi conditionnel vers le début de la séquence C. Le terme "conditionnel" signifie que ce renvoi ne sera exécuté que si le registre de choix contient le signe (-) au moment où l'on arrive à l'ordre de renvoi. Au contraire, si le registre contient le signe (+), le renvoi sera inhibé et le programme suit l'ordre naturel.

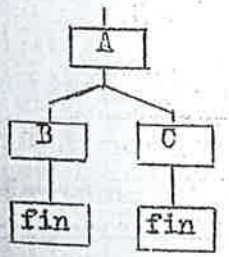

fig. 1

La programmation se fait donc de la façon suivante :

séquence A
Envoi de x dans choix
Renvoi conditionnel vers adresse λ
séquence B
fin

séquence C
fin

La séquence C est à noter dans une partie de la mémoire, commençant à l'adresse λ d'ailleurs arbitraire.

L'envoi du signe d'un résultat dans le registre de choix (avec ou sans effacement du résultat dans le groupe calculateur) est un ordre d'inscription (§4) modifié en remplaçant la tête 090 du préfixe par 074. Comme normalement on ne désire pas garder le résultat, le préfixe usuel sera 07446. D'autre part, le renvoi conditionnel se note par le préfixe 41000. A la séparation entre les séquences OA et AB on notera donc les ordres suivants :

préfixe	adresse	symbole	signification
07446	0000	= ch	Envoi en registre de choix
41000	λ	Rv - λ	Renvoi conditionnel

Bien entendu les deux ordres peuvent être dissociés. Comme pour les registres, une information envoyée dans ch détruit celle qui s'y trouvait précédemment, et y subsiste tant qu'elle n'est pas détruite à son tour par une nouvelle information.

L'exemple de la fig.2. représente schématiquement la structure logique d'un programme comportant des bifurcations vers quatre séquences, avec un retour vers une séquence commune (H) et un by-pass (G, I, K).

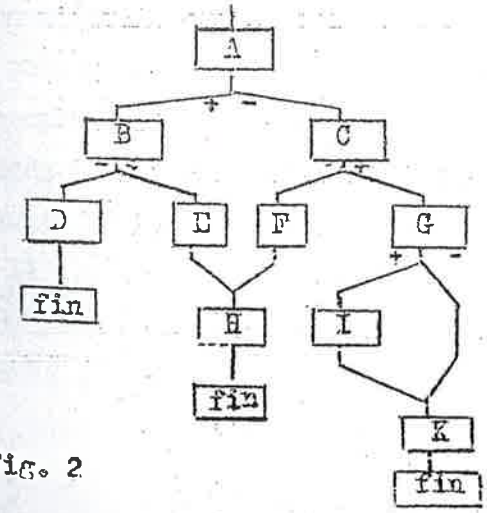

fig. 2

On commence par indiquer par des signes ± , le signe qui devra prendre la variable de choix x à chaque bifurcation.

Le choix est partiellement arbitraire, mais :

(1) si l'on peut dire, a priori, qu'une éventualité sera plus fréquente que l'autre, il y a avantage à ne pas avoir

de renvoi dans ce cas : on réalise ainsi une économie de temps importante (2) dans le cas d'un by-pass (de G à K), le signe - correspond nécessairement à la branche by-passant la séquence.

A la réunion de deux branches vers une branche commune, il n'y a pas de choix à effectuer. La branche commune est placée à la suite d'une des deux branches, tandis que l'autre y renvoie par un <u>renvoi inconditionnel</u> (voir § 6).

On définit une première séquence en partant de A et en prenant à chaque bifurcation la direction +. Supprimant les branches déjà exploitées, la structure dégénère en un certain nombre de programmes disjoints, et on recommence une division en séquences selon le même principe. On arrive ainsi, dans le cas de notre exemple, à une liste de quatre séquences :

sq I	sq II	sq III	sq IV
A	(0)	(0)	(0)
Rv- C	C	D	F
B	Rv- F	fin	Rv H
Rv- D	G		
E	Rv- K		
(0)	I		
H	(0)		
fin	K		
	fin		

Les séquences constituant le programme peuvent être notées à la suite les unes des autres dans un ordre arbitraire; il faut seulement veiller à ce que le premier ordre de toute séquence, et de toute branche commune à plusieurs séquences (telle que H) soit de rang impair, puisque les adresses sont attribuées à des paires d'ordres et qu'on ne peut renvoyer qu'au premier ordre d'une paire. Pour remplir cette condition, il y a éventuellement lieu d'intercaler entre les séquences des ordres vides 00000 0000 ; ceux-ci ont été désignés par (0) et introduits à tous les endroits critiques dans le programme de la fig.2.

Il arrive souvent que le choix, dans un renvoi conditionnel, soit fait en comparant deux nombres a et b. On est amené à prévoir les quatre éventualités suivantes :

Rv⁻ si $a \geqslant b$	Rv⁻ si $a > b$	Rv⁻ si $a \leqslant b$ pour $b \neq 0$	Rv⁻ si $a < b$
- a + b = ch Rv⁻	+ b - a = ch Rv⁻	- b + a = ch Rv⁻	+ a - b = ch Rv⁻

Remarques

(1) Les choix $a \geqslant 0$, $a > 0$, $a < 0$ peuvent se faire par la même méthode en faisant b = 0. Pour le cas $a > b$, il n'est pas nécessaire d'effectuer +b si b est identiquement nul.

(2) Le choix $a \leqslant 0$ ne peut pas se faire par la méthode indiquée ci-dessus, car le premier ordre -0 donnerait le résultat +0, puisque $\omega = +0$ initialement. Dans ce cas, on peut effectuer la suite d'opérations :

$$-a \mid x\,(-0) \mid = ch \mid Rv^-$$

Si on doit effectuer le choix $a \leqslant b$ et que b peut s'annuler (sans être toujours nul), on peut employer la suite d'opérations :

$$+b \mid -a \mid x\,(-0) \mid = ch \mid Rv^-$$

(3) La suite d'ordres

$$= E \mid - mod \mid + E \mid = ch \mid Rv^- \qquad (12)$$

exécute un renvoi si $\omega \neq 0$ et l'inhibe si $\omega = 0$. En effet, après vidange par =E, le groupe calculateur contient +0, et l'opérateur produit $+0 - |\omega|$ qui est négatif pour $\omega \neq 0$, mais qui vaut +0 d'après le §3.3 pour $\omega = 0$. Cet artifice est utile lors d'un contrôle par comparaison.

Une opération conditionnelle peut parfois se faire sans utiliser le registre de choix, mais en utilisant les altérations mod ou sgn. Ainsi, par l'artifice $a + b + |a - b|$ on obtient 2a si $a > b$, mais 2b si $a < b$. On trouvera d'autres exemples dans l'app.II à propos de la réduction de l'argument au premier quadrant dans le calcul du sinus. Les procédés sont toujours préférables au

renvoi, car ils prennent moins de temps. Un renvoi conditionnel n'est en effet exécuté que lorsque tous les ordres précédents ont été exécutés (car il est possible que l'un d'eux modifie le contenu du registre de choix), ce qui provoque un délai difficilé à préciser, puisqu'il dépend de la nature des opérations restant à effectuer; ce délai est évidemment important si ces opérations sont des multiplications. Au contraire, un renvoi inconditionnel ne prend qu'un temps de sélection (7 ms en moyenne).

8. Opérations sur les adresses

8.1. Registres d'indices

L'adresse contenue dans un ordre peut être l'adresse d'un nombre (ordre d'opération ou ordre d'inscription) ou l'adresse d'une paire d'ordres (ordre de renvoi). Il est parfois nécessaire de préciser cette adresse au cours d'un calcul, sans intervention de l'opérateur. Par exemple, lorsqu'un paramètre à introduire dans les calculs dépend d'un résultat antérieur, on dispose les divers paramètres en mémoire et la machine calcule l'adresse de celui qui convient. Ceci est utile notamment pour la lecture dans une table mise en mémoire, l'argument de la fonction tabulée servant à calculer l'adresse à laquelle la table doit être consultée. Plus généralement, une adresse précisée au cours de calcul permet de simuler une notation à indice telle que a_i, la valeur de i servant à modifier l'adresse.

Une modification de l'adresse contenue dans un ordre se fait toujours par addition du contenu d'un registre d'indices. Les registres d'indices sont des mémoires électroniques d'une capacité de quatre chiffres décimaux. L'addition de leur contenu à l'adresse contenue dans un ordre, se fait toujours dans une unité arithmétique auxiliaire et il n'y a pas de report de tête, de sorte que l'on passe bien de l'adresse 9995 à l'adresse 0000 par addition de 0005 (cf. §1). En particulier, en additionnant le contenu d'un registre d'indices à une adresse initialement vide, c'est le contenu du registre d'indices qui sert d'adresse. Après modification d'une adresse, le contenu initial du registre d'indices reste toujours dans le registre, et peut donc être utilisé à plusieurs reprises. Il n'est effacé que par l'envoi d'une nouvelle information.

La machine possède quatre registres d'indices, désignés par G, H, I, K. Leurs contenus sont désignés par les mêmes symboles, mais pour la brièveté des définitions, il sera commode de noter ces registres et leurs contenus par Z_i ($i = 1, 2, 3, 4$), les indices correspondant à l'ordre alphabétique naturel, donc

$$Z_1 = G \quad ; \quad Z_2 = H \quad ; \quad Z_3 = I \quad ; \quad Z_4 = K$$

8.2. Notations

Nous avons convenu au § 1, de désigner un nombre par son adresse dans la mémoire. Cette adresse peut être symbolisée par une lettre (α, γ, ...) ou être explicite (nombre de quatre chiffres : abcd). La lettre ω a été réservée pour désigner le résultat qui se trouve dans le groupe calculateur.

Nous verrons que, dans certains cas, une adresse peut être utilisée elle-même comme un nombre et introduite dans le groupe calculateur : ce nombre est alors nécessairement de la forme

$$+ \ , \text{abcd } 0000 \ldots \ .10^{+00} \qquad (13)$$

Nous convenons de représenter une adresse utilisée comme nombre par $\hat{\alpha}$ ou $\widehat{abcd}$.

Inversement, un résultat ω peut être utilisé comme adresse ou servir à modifier une adresse (par l'intermédiaire des registres d'indices) : on ne conserve alors que les quatre premiers chiffres de la mantisse. Si on a :

$$\omega = s \ , \text{abcdefg} \ldots \ .10^{\sigma pq} \quad (\text{cf. } \S 1)$$

on ne retient que abcd et on convient de représenter ce nombre par $\hat{\omega}$.

En résumé, l'accent circonflexe dénote l'emploi anormal d'un symbole. Remarquons, une fois pour toutes, que le calcul des adresses par l'intermédiaire du groupe calculateur nécessite certaines précautions : toutes les additions doivent se faire sans réalignement pour ne pas faire disparaître les zéros éventuels de tête et il faut s'inquiéter de l'effet d'un éventuel report de tête.

8.3. Modifications d'adresses

Dans tous les préfixes étudiés jusqu'ici, le troisième chiffre était un zéro. En remplaçant ce chiffre par un indice $i = 1, 2, 3, 4$, on produit la

succession des effets suivants :

(1) on ajoute Z_i (contenu du registre d'indice Z_i) à l'adresse mentionnée dans l'ordre

(2) on exécute l'ordre avec la nouvelle adresse ainsi produite.

Un voici quelques exemples où l'on a utilisé le registre $\Pi = Z_2$, donc l'indice 2 en troisième position du préfixe.

préfixe	adresse	symbole	
04216	α	$+ (\alpha + \Pi)$	$\omega + (\alpha + \Pi) = \omega$ la donnée fraîche est extraite de l'adresse $(\alpha + \Pi)$ au lieu de l'adresse α mentionnée dans l'ordre
09246	γ	$= (\gamma + \Pi)$	$\omega = (\gamma + \Pi)$ le résultat est envoyé à l'adresse $(\gamma + \Pi)$ et non à l'adresse γ mentionnée dans l'ordre
40200	λ	$\rightarrow (\lambda + \Pi)$	renvoi à l'adresse $(\lambda + \Pi)$ et non à l'adresse λ mentionnée dans l'ordre

Les ordres à adresse modifiée prennent le même temps que les ordres à adresse normale correspondants.

8.4. Envoi d'un nombre en registre d'indice

Pour remplir un registre d'indice, on peut : (1) amener la donnée du groupe calculateur, c.à.d. amputer ω comme expliqué au § 8.2, (2) utiliser l'adresse α de l'ordre lui-même, (3) combiner les deux traitements. Lorsque l'opération a trait au registre Z_i, le résultat dans ce registre après opération est

$$(1) \quad Z_i + \hat{\omega}$$

$$(2) \quad Z_i + \alpha$$

$$(3) \quad \alpha + \hat{\omega}$$

mais les cas (1) et (3) existent en deux variantes, selon que le résultat est

conservé ou non dans le groupe calculateur. Une quatrième opération possible est celle qui envoie α de l'ordre dans Z_i, effaçant le contenu ancien; c'est la seule qui permette de vider Z_i, si l'on y spécifie α = 0000. Dans l'équation (1), α n'est pas utilisé et l'adresse est donc non pertinente et sera normalement prise 0000.

Les codes de ces opérations sont :

préfixe	adresse	signification
06i46	0000	$Z_i + \hat{\omega} = Z_i$; ω effacé dans le groupe calculateur
06i96	0000	$Z_i + \hat{\omega} \rightarrow Z_i$; ω conservé dans le groupe calculateur
06i00	α	$Z_i + \alpha = Z_i$
06(5+i)46	α	$\alpha + \hat{\omega} = Z_i$; ω effacé dans le groupe calculateur
06(5+i)96	α	$\alpha + \hat{\omega} \rightarrow Z_i$; ω conservé dans le groupe calculateur
06(5+i)00	α	$\alpha = Z_i$

Dans ce tableau, la notation (5+i) pour le troisième chiffre signifie que celui-ci est 6, 7, 8, 9 respectivement pour i = 1, 2, 3, 4.

Les ordres ne faisant pas intervenir ω ne prennent aucun temps : ils ne font qu'occuper en moyenne le temps normal de sélection d'un ordre soit 3,5 ns (puisqu'il faut en moyenne 7 ns pour une paire d'ordres). Les ordres faisant intervenir ω attendent la fin des calculs en cours et retardent donc légèrement l'interprétation des ordres suivants.

8.5 **8.5. Utilisation de l'adresse comme nombre.** → inl pour les bouclé! : on garde les dernier chiffres!

En remplaçant par 03i la tête d'un préfixe d'opération (§3.1) on combine, selon l'opération indiquée, le résultat ω se trouvant dans le groupe calculateur avec la donnée fraîche $(\widehat{\alpha + Z_i})$, c.à.d. avec un nombre de la forme (13) résultant de l'adresse α mentionnée dans l'ordre, auquel on ajoute le contenu du registre Z_i. En outre, si i = 0, on garde l'adresse α sans rien y ajouter, produisant donc le nombre $\hat{\alpha}$.

p.4

adresse 4 bits, 2e à h.-h

p. 13

Exemples :

Préfixe	Adresse	Symbole	Signification
03066	α	$+^* \hat{\alpha}$	$\omega +^* \hat{\alpha} = \omega$
03266	α	$+^* \widehat{(\alpha + H)}$	$\omega +^* \widehat{(\alpha + H)} = \omega$

L'usage de ces codes permet d'économiser du temps et de la mémoire dans le cas d'opérations sur des nombres de quatre chiffres, à virgule fixe, de la forme indiquée, l'introduction de ces nombres pouvant se faire sans utiliser la mémoire des nombres. Ceci est spécialement utile pour les comptages dans les programmes cycliques (§9.1). Le même artifice rend inutile l'introduction dans la mémoire magnétique de toute constante numérique inférieure à l'unité et ayant moins de quatre chiffres après la virgule.

8.6. Le registre M

La machine contient, à chaque instant, dans un registre M l'adresse de la paire d'ordres à exploiter après l'ordre en cours d'exécution, de façon à ne pas perdre le fil du programme. Dans une séquence normale, le registre M contient l'adresse d'où provient l'ordre en cours, augmentée de 0005 de façon à préparer la machine à la sélection de la paire d'ordres suivante. En cas de renvoi, l'adresse λ spécifiée dans le renvoi est envoyée en M, effaçant donc son contenu ancien, et le fil du programme principal est perdu. Si après exécution d'une séquence, on désire revenir à l'endroit quitté du programme principal, et que cet endroit est inconnu (ou difficile à prévoir) au moment de la programmation de la séquence principale, on peut mettre le contenu de M en mémoire, p.ex. dans un registre d'indice, avant l'exécution du renvoi. Dans ce but, on a assimilé M aux autres registres d'indice et on a prévu des opérations analogues à celles décrites jusqu'ici.

(a) modification d'adresse. Le processus est analogue à celui du § 8.3. En remplaçant le troisième chiffre d'un ordre par 5, on provoque l'exécution de l'ordre avec la nouvelle adresse $\alpha + M$, où α désigne l'adresse mentionnée dans l'ordre. En remplaçant le troisième chiffre par 5+i, on provoque l'exécution avec l'adresse $\alpha + Z_i$; l'adresse mentionnée dans l'ordre est alors non pertinente et sera

normalement vide. Ceci vaut également pour les ordres du § 8.5.

(b) envoi d'un nombre en registre d'indices

Le cas (2) du § 8.4 est complété par le code

préfixe	adresse	signification
01(5+i)00	α	$\alpha + M = Z_i$

(c) exemple : renvoi avec mémorisation de l'adresse quittée

Si, au cours d'un programme, on a besoin de calculer une fonction élémentaire (un logarithme, p.ex.), dont le sous-programme est en mémoire, il faudra effectuer un renvoi vers cette séquence et revenir ensuite au programme principal. Soit α l'adresse du dernier mot (paire d'ordres) de la tranche du programme principal précédant le renvoi au sous-programme. D'après ce qui a été dit au début du § 8.6, le registre K contient α + 0005 au moment de l'achèvement de la tranche en question. Si donc le renvoi est exprimé par la paire d'ordres

$$\begin{cases} 0000 + M = K \\ Rv\,\beta \end{cases} \qquad (14)$$

où β est l'adresse du premier mot du sous-programme auquel on renvoie, la paire (14) constitue le mot d'adresse α + 0005, et au moment de son exploitation, le contenu de M est α + 0010. C'est ce contenu qui est mis en réserve dans K par le premier ordre de la paire (14). Mais α + 0010 est aussi l'adresse du premier mot de la tranche suivante du programme principal, à laquelle il faudra revenir après le sous-programme. Il suffit donc pour le retour de terminer le sous-programme par un renvoi à l'adresse contenue dans K, c.à.d. par l'ordre Rv(0000 + K).

En résumé, il suffit, dans la séquence principale, de faire précéder le renvoi par l'ordre 0000 + M = K, de code 01900 0000, et de terminer la sous-séquence par l'ordre Rv(0000 + K), de code 40400 0000. Il est cependant essentiel, pour la succession correcte des adresses des paires d'ordres, que les deux ordres (14) combinés par une accolade dans la séquence principale forment une

paire; il est toujours possible d'y arriver en faisant éventuellement précéder cette paire par un ordre vide.

2. Programmes cycliques

De nombreux calculs numériques nécessitent de répéter plusieurs fois la même opération. Cette opération ne sera inscrite qu'une fois en mémoire et sera reprise autant de fois qu'il le faut, avant de poursuivre la suite du programme. On obtient ainsi un programme cyclique. On peut distinguer deux cas, suivant que le nombre de reprises est déterminé ou non au moment de la programmation.

2.1. Comptage des cycles

Lorsque le nombre de reprises est fixé a priori, on organise un comptage des cycles. S'il faut effectuer n cycles en tout, nous dirons qu'il y a une séquence directe et (n-1) reprises. La fig.3 indique le schéma d'un programme cyclique où l'on a supposé, pour être général, que la bifurcation vers la sortie se fait au milieu du dernier cycle; dans ce but, le cycle est divisé en deux parties B et D; après bifurcation, on a admis en outre une séquence C préparant la sortie. La bifurcation à la sortie de B est régie par un renvoi conditionnel résultant du comptage des cycles. La désignation par + et - de l'une ou l'autre branche de la fourche est en principe arbitraire, mais les remarques suivantes sont à prendre en considération :

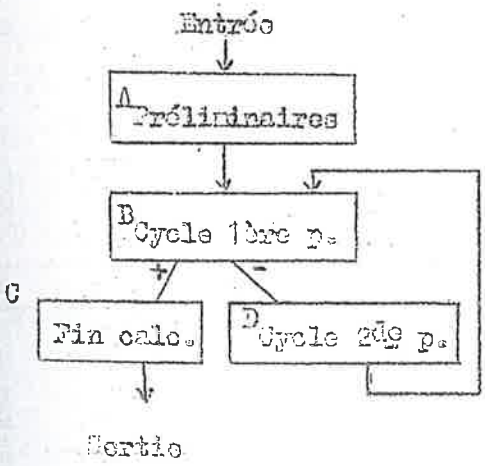

fig. 3

(a) Le choix adopté à la fig.3 permet, dans le cas où la séquence D n'existe pas, de renvoyer directement au début de la séquence B par le renvoi conditionnel. Avec le choix opposé, la séquence D, même vide, devrait contenir un ordre unique de renvoi fixe vers le début de la séquence B. Les conclusions sont opposées si la séquence C se réduisait à zéro, et c'est alors le choix de signe opposé à celui de la fig.3 qui serait le plus avantageux. En supposant que la

19 § 8.5

séquence D existe réellement, le choix de la fig.3 impose un renvoi conditionnel et un renvoi fixe à chaque cycle, tandis que le choix inverse n'impose qu'un renvoi fixe à chaque cycle.

(b) Considérons le choix des signes de la fig.3, et supposons que le programme comporte 3 reprises (4 cycles). Ce nombre doit être mis en mémoire à une adresse r, d'ailleurs arbitraire, au début du programme, c.à.d. pendant la séquence A. Le comptage de cycles se fera le plus simplement en utilisant les adresses comme nombres, et nous verrons qu'avec la convention adoptée pour les signes, il convient d'envoyer initialement dans r la valeur négative -3, et d'y ajouter une unité à chaque cycle. Le contenu de r passe en effet ainsi après les cycles successifs par les valeurs -2, -1, 0 et +1 et, après le quatrième cycle, la valeur positive ainsi obtenue, envoyée dans le registre de choix, inhibe le renvoi vers D et aiguille le programme vers C. Avec le choix de signes opposé à celui de la fig.3, le comptage se ferait à partir de +3 par soustraction de l'unité.

(c) L'envoi initial de -3 en r s'exprime par les ordres

$$\left.\begin{array}{l} -^{*}\widehat{0003} \\ \\ = r \end{array}\right\} -3 \text{ dans } r \qquad (15)$$

L'addition de l'unité et le conditionnement du renvoi demandent les ordres

$$\left.\begin{array}{l} +^{*}r \\ +^{*}\widehat{0001} \\ \rightarrow ch \end{array}\right\} (r+1) \text{ dans } ch$$

$$\begin{array}{ll} = r & r+1 \text{ dans } r \\ \hline Rv^{\neg} & \text{renvoi} \end{array} \qquad (16)$$

dont les trois premiers et le dernier font nécessairement partie de la séquence B. Quant au quatrième, il a pour but de remettre dans r le nouveau résultat du comptage et peut éventuellement se faire après le renvoi, donc dans la séquence C. Comme cet ordre coïncide avec le dernier de (15), ils peuvent être tous deux combinés en tête de la séquence B. La décision dans chaque cas sera dictée par le groupement des ordres en paires. Il est également loisible de placer dans D les deux premiers ordres de (16), et de démarrer à partir de A avec -2 comme contenu de r.

9.2. Reprise conditionnelle avec nombre maximum de reprises

La décision d'effectuer une reprise ou de poursuivre le programme, peut dépendre du résultat obtenu, p.ex. lors d'un calcul par itération, quand la correction apportée devient inférieure à une tolérance donnée. Dans ce cas, il suffit de comparer la correction à la tolérance et d'envoyer le résultat dans le registre de choix pour conditionner un renvoi. On peut imaginer diverses structures selon les polarités des bifurcations.

Il pourrait arriver qu'il se produise une erreur systématique dans le calcul et que l'on reprenne continuellement le même calcul; aussi, fixe-t-on un nombre maximum de reprises. Au moment où l'on voudrait dépasser ce nombre, on donne un ordre de fin avec signal d'alarme (cfr § 11.2).

10. Contrôles

Les erreurs de calcul peuvent être décelées assez simplement : par exemple en effectuant le calcul par des méthodes complètement différentes, ou, éventuellement, en contrôlant si les résultats vérifient une relation existant entre eux. Lorsqu'un contrôle de calcul ne se vérifie pas, on provoque une reprise de calcul. Comme ceci ne remédie pas à une erreur systématique, on doit prévoir un nombre maximum de reprises, suivi d'une alarme. C'est une forme de programme cyclique. On a, p.ex. :

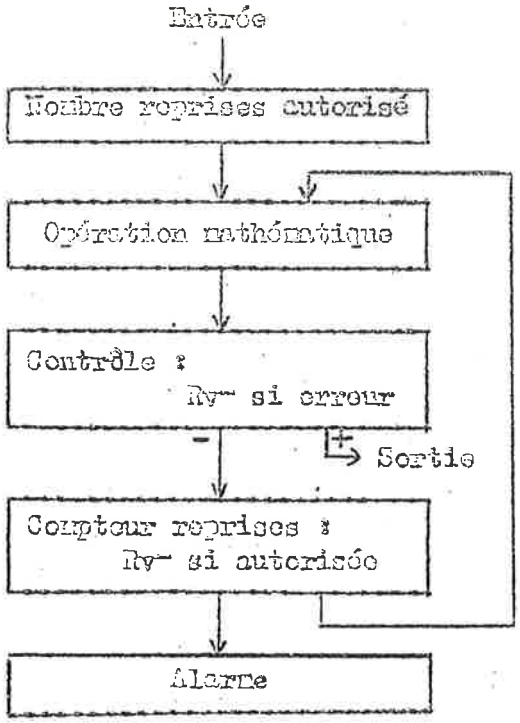

fig.4

Une autre solution consiste à ne pas fixer d'avance un nombre maximum de reprises, mais à faire intervenir l'opérateur manuellement à chaque erreur.

Lors d'une reprise, il convient de repartir avec des données aussi sûres que possible.

Il est plus difficile de détecter des erreurs de programme. Dans certains cas, un contrôle de calcul échouera et déclenchera l'alarme, mais il peut arriver qu'après une première reprise, le calcul se fasse correctement, à partir de données fausses. En général, un contrôle de calcul global, effectué pour l'ensemble du problème, sera le plus sûr. Dans les programmes compliqués, on peut organiser un comptage de séquences principales au fur et à mesure du calcul.

11. Alarmes

Les codes des différents ordres provoquant ou utilisant une alarme sont donnés dans le tableau ci-dessous et seront ensuite expliqués.

préfixe	adresse	symbole	
00014	0000	bloc.al.	blocage alarme groupe calc.
00024	0000	débloc.al.	débloc. " " "
00034	0000	eff.al.	effacement " " "
46i00	λ	rv.ex.al.g.c.	renvoi à l'adresse λ' (fonction de λ et de i) exécuté seulement s'il y a eu alarme du gr.calc.
47000	0000	fin al.	fin programme avec alarme

11.1. Alarme du groupe calculateur

Lors d'un dépassement de la capacité des registres, p.ex. dépassement d'exposant dans les calculs à virgule flottante (cfr. §2), ou report en tête d'une addition à virgule fixe, le calcul en cours est achevé, mais le résultat

peut-être incorrect puisqu'il n'est pas tenu compte du dépassement. Dans ce cas, la machine s'arrête et une lampe rouge AL.G.C. s'allume sur le pupitre de commande.

En appuyant sur le bouton EFF.AL., on détruit tous les effets de l'alarme et le programme reprend à l'ordre suivant, mais le contenu du groupe calculateur est toujours le résultat ayant provoqué l'alarme (donc éventuellement faux).

Dans certains cas, un dépassement peut être utilisé à dessein pour provoquer une bifurcation dans le programme. Il n'est alors pas souhaitable que la machine s'arrête; il est simplement nécessaire qu'un dépassement, ayant eu lieu, soit gardé en mémoire de façon à pouvoir conditionner un renvoi ultérieur. Une telle exploitation est possible si l'on a donné, au préalable, dans le programme un ordre de blocage d'alarme du gr.calc.; cet ordre empêche, jusqu'à nouvel ordre, les alarmes du gr.calc. d'agir sur le pupitre et d'arrêter la machine, mais l'alarme est gardée dans une mémoire capable de conditionner un renvoi. Un renvoi faisant appel à cette mémoire est caractérisé par le préfixe 46i00, où i (s'il est non nul) sert à transformer l'adresse λ en une nouvelle adresse selon le contenu des registres Z_i ou H (§§ 8.3 et 8.6.a). Le contenu de la mémoire servant à noter le fait qu'il y a eu alarme du groupe calculateur peut être effacé par l'ordre d'effacement d'alarme (préfixe 00034). Le retour aux conditions normales d'exploitation (où les alarmes de dépassement agissent de nouveau sur le pupitre) s'obtient jusqu'à nouvel ordre par l'ordre de déblocage de l'alarme (préfixe 00024). On ne peut donner cet ordre de déblocage que s'il y a eu un ordre d'effacement d'alarme auparavant.

11.2. Alarme de programme

Celle-ci est l'ordre de fin avec alarme mentionné au § 9.2, à prévoir dans toute séquence interdite d'un programme. Si l'aiguillage vers une telle séquence a lieu, cet ordre provoque l'allumage d'une lampe FIN AL. sur le pupitre, et arrête la machine.

Dans tout programme, la bifurcation vers l'ordre "fin avec alarme" est normalement précédée par un renvoi conditionnel de sorte que cet ordre constitue souvent une séquence à lui tout seul. Lorsqu'il y a plusieurs moments successifs dans un programme pouvant renvoyer vers un ordre d'alarme, on peut songer à inscrire celui-ci à une adresse quelconque p.ex. 2475, et renvoyer toujours à cette même adresse à chaque condition d'alarme. En cas d'alarme, il ne reste alors aucun repère de la condition ayant provoqué l'alarme, puisque la machine s'arrête toujours à l'ordre d'adresse 2475. Il est donc préférable de prévoir des ordres "fin avec alarme" séparés, à autant d'adresses distinctes qu'il y a de conditions d'alarme à envisager dans le programme.

12. Vidanges

Comme le dernier résultat d'une opération se trouve normalement dans le groupe calculateur et n'en est extrait que par un ordre d'inscription du type =, qui peut faire partie d'une séquence ultérieure, éventuellement by-passée dans certains cas, il arrive qu'une nouvelle sous-séquence doive commencer par un ordre de vidange du groupe calculateur. Le code de cet ordre est 07346 0000.

Pour les mêmes raisons que ci-dessus, on peut avoir besoin de vider un registre électronique, p.ex. si un tel registre doit contenir une variable de comptage de cycles, et qu'on désire y inscrire i = 0 au début d'un programme. Dans le cas des registres d'indices, la seule possibilité de vidange est celle mentionnée au § 8.4.

Pour vider les registres E ou F, il suffit d'y faire une inscription à un moment où le groupe calculateur est vide, de sorte que l'on inscrit 0 = $+0.10^{+00}$.

Il est à remarquer que l'inscription en mémoire magnétique ou électronique d'un zéro sous la forme $+0.10^{+00}$ n'est pas toujours mathématiquement adéquate, car l'opération a + 0 fait perdre les chiffres de queue de a par alignement sur 0, si l'exposant de a est négatif (le groupe calculateur étant initialement vide). Dans les problèmes où ce cas se présenterait et serait un inconvénient, il est essentiel de noter 0 sous la forme la plus petite possible, soit $+0.10^{-49}$. Un tel zéro doit être noté en mémoire magnétique et ne peut s'engendrer dans un registre par vidange. Par contre, l'opération + 0 | + a se fait sans alignement de a sur 0 car la reconnaissance du résultat 0 bloque l'alignement préalable.

13. Instructions et notations pour la programmation

On appelle séquence élémentaire une tranche d'opérations entre deux noeuds consécutifs dans la structure du programme lorsque celle-ci est représentée sous forme d'un schéma tel que celui de la fig.4 (I p.26). Les séquences élémentaires sont numérotées A, B..., en principe dans l'ordre qu'elles occuperont dans le programme. Dans la préparation des séquences, on numérote les paires d'ordres consécutivement en commençant par 1. On réfère au premier ou au second ordre de la paire 12 p.ex. en disant qu'il s'agit de l'ordre 12a ou 12b. Dans le programme, on introduit des séparations en trait plein entre séquences élémentaires. On fait éventuellement précéder d'un gros point (•) un ordre auquel on doit pouvoir renvoyer et qui doit être le premier d'une paire.

On introduit des séparations en trait interrompu après chaque ordre d'inscription : une tranche entre deux séparations consécutives en trait interrompu correspond alors à une équation mathématique, qu'on écrit en notation algébrique en regard de l'ordre d'inscription qui en sort le résultat. Lorsque, au cours d'un calcul, on utilise un résultat gardé en mémoire depuis une séquence précédente ou mis en registre E ou F au cours d'une séquence précédente, il est bon de rappeler en regard de l'ordre qui extrait cette donnée, le numéro de l'ordre de la séquence antérieure qui l'y a envoyé.

? Sec.!

Chap.II. Virgule fixe, haute précision et calcul sur les ordres

1. Calculs à virgule fixe

La machine travaille à virgule fixe jusqu'à nouvel ordre si l'on donne l'ordre 00021 0000, et l'on revient au calcul à virgule flottante (jusqu'à nouvel ordre également) par l'ordre 00011 0000.

En virgule fixe, l'exposant de toute donnée fraîche est remplacé par 00 à l'entrée du groupe calculateur, et tous les alignements et normalisations sont supprimés (même si les opérations sont spécifiées avec normalisation). Les résultats sont toujours affectés de l'exposant 00. Un nombre a donc 15 décimales précédées d'une virgule fixe en tête, et d'un signe de nombre. Un report de tête éventuel dans une addition est alors un dépassement et donne l'alarme du groupe calculateur. La multiplication de deux nombres inférieurs à l'unité ne produit jamais de report de tête, et la queue est amputée de façon à ramener le résultat à 15 chiffres après la virgule.

Lorsqu'il s'agit de faire des calculs à virgule fixe sur des nombres non inférieurs à l'unité, il faut préalablement affecter toutes les données d'un facteur 10^{-n} convenable. Les opérations linéaires (addition, soustraction ...) restent alors correctes, mais le résultat d'une multiplication est affecté d'un facteur 10^{-2n}, et est donc 10^{n} fois trop petit. Il est donc nécessaire de provoquer un déplacement de n chiffres vers la gauche, sur un des facteurs ou sur le résultat (ou plusieurs déplacements partiels donnant le même total). Dans ce but, il est prévu une altération dg (code 00033 0000) permettant, par une multiplication ultérieure, de décaler de n chiffres vers la gauche un résultat se trouvant dans le groupe calculateur. Le nombre n est déterminé par un multiplicateur de la forme

$$m = \ ,\underbrace{00\ldots0\,1}_{n}\,000\ldots \qquad (1)$$

devant suivre l'ordre d'altération. La succession des ordres

$$|dg|\times m| \qquad (2)$$

transforme donc ω en ω' décalé, comme suit :

$$\text{si } \omega = \ ,\underbrace{000\ldots0}_{n}abcd\ldots \quad ; \quad \omega' = \ ,abcd\ldots\underbrace{000\ldots}_{n} \qquad (3)$$

Si les ordres (2) portent sur un ω ayant moins de n zéros en tête, le décalage se fait quand même, mais le résultat ne donne plus correctement une multiplication par 10^n. Comme aucune alarme ne se produit dans ce cas, il est nécessaire de connaître a priori les ordres de grandeur des nombres à tout instant, ou de les contrôler par sous-programme. L'altération dg n'est pas utilisable en virgule flottante.

Un décalage de n chiffres vers la droite s'obtient en fonctionnement à virgule fixe, au moyen d'une multiplication ordinaire par le nombre (1), ci-dessus. Il est à remarquer que cette même multiplication effectuée en virgule flottante donnerait un décalage de (n-1) chiffres vers la droite car il y aurait en même temps correction de (-1) à l'exposant (cfr ch.I §3.4).

L'altération ± mod donne le module ou son opposé, comme en virgule flottante; l'altération ± sgn donne 0,1 affecté du signe du nombre auquel elle s'applique, ou de son opposé, et non 1 comme en virgule flottante. Les autres altérations du chapitre I §3.2 n'ont pas d'intérêt en virgule fixe.

Dans le cas de nombres inférieurs à l'unité, l'inverse ne peut être noté et la division est impossible. Lorsque les nombres sont implicitement affectés de facteurs 10^{-n}, la division est possible dans certains cas, mais exige une séquence spéciale, celle du chap.IV, basée sur des approximations successives, n'étant évidemment pas applicable. En fait, les calculs à virgule fixe ne sont utiles que pour les calculs linéaires, et comme éléments de calcul à haute précision qui sera discuté ci-dessous.

Le procédé du chap.I §3.6 pour calculer la partie entière d'un nombre est basé sur l'alignement et échoue dans les calculs à virgule fixe. Dans ce but et pour permettre d'autres manipulations analogues, l'altération tri est prévue. Son code est 00043 0000. L'altération n'a de sens que si la donnée fraîche α, introduite par l'ordre suivant, est uniquement composée de chiffres 0 et 1. Soit donc p.ex.

$$\omega = \ ,a\,b\,c\,d\,e\,f\ldots$$

$$\alpha = \ ,1\,0\,0\,1\,0\,1\ldots$$

La suite d'ordres

$$| \text{tri} | \times \alpha |$$

transforme ω en

$$\omega' = \ , a\ 0\ 0\ d\ 0\ f \ldots$$

c.à.d. que les chiffres de ω de même rang que les zéros de α sont remplacés par des zéros. Les signes de ω et α sont combinés comme dans une multiplication ordinaire. Cette altération ne peut pas s'employer à virgule flottante.

Préfixe	Adresse	Symbole	Signification
00021 00011	0000 0000	FI FLO	Calcul à virgule fixe Calcul à virgule flottante
00035	0000	dg	Altération décalage à gauche
00043	0000	tri	Altération tri

2. Calculs sur les ordres

Les ordres étant associés par paires dans un mot, un calcul sur les ordres (p.ex. pour composer automatiquement un programme au moyen de sous-programmes existants) exige de pouvoir dissocier les deux ordres d'une paire, de les traiter séparément dans le groupe calculateur et de les recombiner ensuite. Un calcul sur le mot complet est impossible par suite du traitement spécial des chiffres correspondant à l'exposant et aux signes d'un nombre ordinaire.

A l'entrée, il s'agit donc de prendre les 9 premiers chiffres (OR1) ou les 9 derniers chiffres (OR2) d'un mot, pour en faire un nombre positif ayant 9 chiffres significatifs après la virgule, suivis de 6 zéros et d'exposant +00.

A la sortie, on ne conserve que les 9 premiers chiffres du résultat (sans signe, ni exposant) pour former un ordre et on combine 2 ordres pour former une paire. Le premier ordre de la paire est obtenu par une sortie OR1°, tandis que le deuxième et la combinaison sont obtenus par une sortie OR2. On ne peut intercaler aucune sortie entre ces deux ordres. Une sortie OR2 qui n'est pas précédée d'une sortie OR1° donne une paire dont le premier ordre est un ordre vide et dont le deuxième est l'ordre calculé.

Le symbole ° signifie qu'il n'y a pas de sélection de mémoire pour cette opération. L'adresse est non pertinente et est prise nulle. Le calcul peut se faire à virgule fixe (FI) ou à virgule flottante (FLO); dans ce dernier cas, on travaillera sans normalisation, puisque les ordres peuvent commencer par un zéro ; ce sont les codes indiqués au tableau; il n'est pas nécessaire de noter le signe + , car on n'en utilise pas d'autres.

Du point de vue de l'apparition possible de reports, il est important de savoir que, dans la machine, la succession des chiffres comporant un ordre n'est pas celle de l'écriture adoptée dans ce manuel. La notation du manuel étant :

a b c d e	f g h k	(5)
préfixe	adresse	

cet ordre apparaît, pour le calcul, sous la forme :

+ , f g h k e d c b a 0 0 0 0 0 0 , + 0 0 (6)

C'est ce nombre qui est exploité, en FI ou en FLO, suivant les règles habituelles. A la sortie, on effectue la transformation inverse, en supprimant les signes, l'exposant et les six chiffres de queue.

A titre d'exemple, l'addition des mots complets aux adresses a et b , et l'inscription du résultat en g se fait comme suit, (supposant $\omega = 0$ au départ) :

OR1 + a

OR1 + b

OR1° =

OR2 + a

OR2 + b

OR2 = c

préfixe	adresse	symbole		
04168	α	OR1 + α	$\omega + \alpha^{0}_{1} = \omega$	
04179	α	OR1 - α	$\omega - \alpha^{0}_{1} = \omega$	toujours
04169	α	OR2 + α	$\omega + \alpha^{0}_{2} = \omega$	sans
04179	α	OR2 - α	$\omega - \alpha^{0}_{2} = \omega$	normalisation
00049	0	OR1° =	avec effacement	
00099	0	OR1° →	sans effacement	
09149	γ	OR2 = γ	avec effacement	
09199	γ	OR2 → γ	sans effacement	

En modifiant les trois premiers chiffres des préfixes comme aux chapitres I et III, on utilise d'autres mémoires que le tambour.

3. Multiplication sans arrondi

Le produit de deux nombres de 15 chiffres donne un nombre de 30 chiffres au plus, sans arrondi. Si les deux nombres de 15 chiffres sont exacts (c'est-à-dire si les chiffres suivants sont des zéros), tous les chiffres du produit sont exacts. La multiplication sans arrondi ($\times\times$) permet d'obtenir un résultat de 30 chiffres en deux tranches de 15. La tranche de queue doit être envoyée en mémoire la première, par un ordre = = . La tranche de tête peut être sortie normalement par l'un des ordres = ou ⇒ . Le groupe calculateur doit être organisé à virgule fixe pour effectuer ces ordres, et les exposants n'interviennent donc

pas. Le signe du produit est noté sur les deux tranches du résultat. La multiplication à facteur constant peut également se faire sans arrondi par l'ordre (× ×). On a donc les nouveaux codes :

préfixe	adresse	symbole	
04137	α	× × α	ω × α } sans arrondi
04187	α	(× ×) α	ω ⊗ α } sans arrondi
09147	γ	= = γ	sortie tranche de queue en γ

Lorsqu'on a effectué une multiplication sans arrondi (× ×), l'ordre d'inscription de la tranche de queue (= =) doit suivre immédiatement. On ne peut pas continuer un calcul sur le résultat à 30 chiffres sans utiliser le calcul à haute précision (§4). D'autre part, on doit nécessairement effectuer cet ordre (= =) sans quoi on risque de perturber des opérations ultérieures (et il n'aurait servi à rien d'effectuer l'opération × ×).

Il est à remarquer en particulier que, lorsqu'on travaille avec des nombres entiers à virgule fixe en simple précision, les nombres sont tous affectés d'un facteur 10^{-15} dans la machine; un produit est alors affecté de 10^{-30} et est 10^{-15} fois trop petit. Si les facteurs du produit sont des nombres entiers suffisamment petits pour que le produit ne dépasse pas 15 chiffres, ce produit constituera la tranche de queue T après une multiplication × ×, et on aura ω = 0 pour la tranche de tête. On travaille donc correctement sur des nombres entiers, en évitant l'altération dg, à condition de toujours employer la multiplication × × et de sortir effectivement tout produit par = = . Si l'on désire, en outre, contrôler qu'il n'y a jamais eu de dépassement des 15 chiffres dans les produits, il suffit de sortir et accumuler ω dans une case de travail après chaque multiplication, et de vérifier que cette case contient encore 0 à la fin du problème. Soit r cette case, une multiplication a × b = c demande alors la suite d'ordres

+ a | × × b | = = c | + r | = r

4. Calculs à haute précision

La machine peut traiter des nombres à virgule fixe de longueur indéfinie en les fractionnant en tranches de 15 chiffres, à condition d'attribuer une adresse à chaque tranche. A cet effet, on doit introduire les opérations d'addition et de soustraction en haute précision (en effet, un report peut passer d'une tranche à la suivante, et le signe d'une différence n'est connu qu'à la fin de l'opération). En combinant ces opérations avec la multiplication sans arrondi, il est possible d'effectuer une multiplication haute précision, en la décomposant par programme.

4.1. Addition et soustraction haute précision

Dans une opération normale du groupe calculateur sur une tranche unique, la soustraction se fait par prise de complément préalable sur le second terme; selon les signes et valeurs absolues des termes, le résultat est alors sous forme normale ou complémentaire et sa remise éventuelle sous forme normale demande un second tour d'addition, régi automatiquement par le groupe calculateur. Dans l'addition ou la soustraction à haute précision, le premier tour d'addition tranche par tranche diffère de l'addition en ce que les reports de tête doivent agir d'une tranche à l'autre, et l'opération correspondante sera symbolisée par ++ . Ce premier tour doit de même être suivi par un second pour remettre éventuellement toutes les tranches sous forme normale; ceci équivaut à l'addition ++0 tranche par tranche, mais la décision automatique de prise de complément ou non, résultant du report final et des signes au premier tour, rend ces additions légèrement différentes de celles du premier tour. Dans ce but, les opérations du premier tour doivent être suivies de l'ordre "fin 1" qui altère jusqu'à nouvel ordre la structure du groupe calculateur. Après le fin du second tour, on donne l'ordre "fin 2" qui restaure la structure normale.

Soient donc a_3 a_2 a_1 les adresses des tranches successives du nombre a écrit en virgule fixe dans l'ordre

$$0,\ a_3 \quad a_2 \quad a_1$$

(15 ch) (15 ch) (15 ch)

le signe étant répété chaque fois. Soient de même b_3 b_2 b_1 les adresses des tranches de b. Si l'on désire faire a + b = c, le résultat c venant aux adresses c_3 c_2 c_1, il faut donner la suite d'ordres :

+ a_1	+ c_1
++ b_1	++ 0
= c_1	= c_1
+ a_2	+ c_2
++ b_2	++ 0
= c_2	= c_2
+ a_3	+ c_3
++ b_3	++ 0
= c_3	= c_3
fin 1	fin 2

On a donc les nouveaux codes

préfixe	adresse	symbole	signification
04167	α	++ α	ω ++ $\alpha' = \omega$
04177	α	-- α	ω -- $\alpha' = \omega$
00051	0	fin 1	fin premier tour
00041	0	fin 2	fin second tour

4.2. Multiplication à haute précision

Celle-ci n'est qu'une combinaison d'additions et de multiplications par tranches. Soit p.ex. la multiplication à double précision de deux nombres inférieurs à l'unité :

$$(a_2 + a_1 10^{-15})\ (b_2 + b_1 10^{-15}) = a_2\ b_2 + (a_1 b_2 + a_2 b_1) 10^{-15} + a_1 b_1 10^{-30}$$

Si l'on désire le résultat à double précision, le dernier terme est négligeable, et les produits dans $a_1b_2 + a_2b_1$ peuvent se faire à simple précision, tandis que a_2b_2 doit être calculé sans arrondi. En outre, l'addition de $r_1 = a_1b_2$ avec $r_2 = a_2b_1$ et la queue de $r_3 = a_2b_2$ doit se faire à double précision de façon à ne pas perdre les reports éventuels, c.à.d. qu'il faut faire $r_1 + r_2$. On suppose toujours les signes répétés dans chaque tronche. On a donc la suite d'ordres

$$\left.\begin{array}{l} + a_1 \\ \times b_2 \\ = r_1 \\ \hline + a_2 \\ \times b_1 \\ = r_2 \end{array}\right\} \begin{array}{l} r_1 = a_1b_2 \text{ simple préc.} \\ \\ r_2 = a_2b_1 \text{ simple préc.} \end{array}$$

$$\left.\begin{array}{l} + r_1 \\ ++ r_2 \\ = r_1 \\ \hline + 0 \\ ++ 0 \\ = r_2 \\ \hline \text{fin 1} \\ \hline + r_1 \\ ++ 0 \\ = r_1 \\ \hline + r_2 \\ ++ 0 \\ = r_2 \\ \hline \text{fin 2} \end{array}\right\} (0, r_1) + (0, r_2) = (r_2, r_1) \text{ double préc.}$$

$$\left.\begin{array}{l} + a_2 \\ \times b_2 \\ = a_1 \\ = a_0 \end{array}\right\} a_2b_2 = (a_2, a_1) \text{ double préc.}$$

$$\left.\begin{array}{l}
+\ x_1 \\
++\ o_1 \\
=\ o_1 \\
\hline
+\ x_2 \\
++\ o_2 \\
=\ o_2 \\
\hline
\text{fin } 1 \\
\hline
+\ o_1 \\
++\ 0 \\
=\ o_1 \\
\hline
+\ o_2 \\
++\ 0 \\
=\ o_2 \\
\hline
\text{fin } 2
\end{array}\right\} \quad (x_2,\ x_1) + (o_2,\ o_1) = (o_2,\ o_1) \text{ doublo paso.}$$

Chap. III. Utilisation des rubans. Mécanismes d'entrée et de sortie

1. Généralités

Tout calcul se fait à partir d'un programme et d'une liste de données numériques qui sont normalement à introduire, avant le début du calcul proprement dit, respectivement dans le tambour d'ordres et le tambour de nombres. Comme ces programmes et données numériques peuvent être utilisés à différentes occasions, ils sont préparés (et conservés) sur des rubans magnétiques. La première inscription de l'information sur les rubans se fait à la main à l'aide du clavier du pupitre de commande.

Au cours d'un calcul, les résultats s'inscrivent normalement sur le tambour de nombres au fur et à mesure de leur production. Pour les imprimer, on les transfère préalablement sur un ruban de résultats. On peut ensuite alimenter l'imprimeur à partir de ce ruban, sur lequel on aura noté les instructions typographiques appropriées.

Il est également permis d'imprimer directement des résultats (de la sortie du groupe calculateur, sans passer par le ruban de résultats) au fur et à mesure de leur production (§ 8.3). Il est également permis de lire directement des données sur les rubans (§ 7.1) sans avoir préalablement transféré l'information vers le tambour de nombres. Dans ce but, les circuits de calcul de la machine ont accès à deux positions de lecture de ruban, appelées A et B. La sortie du groupe calculateur a d'autre part accès à une position d'inscription sur le ruban de résultats, appelée C. Par contre, il est impossible d'exploiter un programme directement à partir du ruban sur lequel il est noté, sans avoir transféré l'information vers le tambour d'ordres.

La machine possède six mécanismes de rubans, numérotés de 0 à 5, dont chacun peut être connecté à une des positions A, B, C. Cette connexion est commandée par un ordre d'organisation des rubans (§ 3) valant jusqu'à nouvel ordre.

Nous avons vu que les deux demi-tambours (numérotés "0" et "1") pouvaient indifféremment servir pour les nombres et les ordres (cfr. I, p. 15). En fait, les circuits de la machine possèdent deux entrées (vers les circuits de program-

ne, vers le groupe calculateur) et une sortie (du groupe calculateur). Ces trois accès peuvent être connectés indépendamment de diverses façons vers les demi-tambours, de sorte que l'on peut lire et inscrire sur n'importe quel demi-tambour. Ces connexions sont commandées par des ordres d'organisation des tambours (§3) valant jusqu'à nouvel ordre.

2. Notations relatives aux registres d'indices

La plupart des ordres dont il sera question ci-après sont modifiables en fonction du contenu de divers registres, ainsi que nous l'avons vu pour les opérations et les renvois dans I, §8. Ces modifications se font toujours en n'exploitant pas telle quelle l'adresse ξ (notation générale désignant aussi bien une adresse de lecture α, d'inscription γ ....) mentionnée dans l'ordre, mais en la changeant en ξ', fonction de ξ et d'un chiffre i inséré dans le préfixe de l'ordre, et indiquant la nature du changement selon le code

$$\xi' = \xi \qquad \text{pour } i = 0$$
$$\xi' = \xi + Z_i \qquad \text{pour } i < 5 \text{ (cfr. I, §8.3)}$$
$$\xi' = \xi + M \qquad \text{pour } i = 5 \text{ (cfr. I, §8.6.a)}$$
$$\xi' = M + Z_i \qquad \text{pour } i > 5 \text{ (cfr. I, §8.6.a)}$$

3. Ordres d'organisation des rubans et des tambours

préfixe	adresse	interprétation
99i00	ξ	Si l'adresse effective ξ' selon le §2 est xyz* ; les chiffres x, y, z correspondent aux numéros des rubans connectés respectivement aux positions A, B, C du §1. Il faut que x, y et z soient différents.
90i00	ξ	Si l'adresse effective ξ' est ***u, le chiffre u décide de la connexion des tambours selon le tableau ci-dessous.

n	Numéro du tambour connecté à :			emploi
	entrée circ.progr.	entrée calc.	sortie calc.	
6	0	1	1	connexion normale pour le calcul
5	0	0	1	débordement du tambour de nombres vers le tambour d'ordres pour les données (lecture)
1	0	1	0	id. pour les résultats (inscription)
0	0	0	0	id. pour les données et les résultat permet le travail sur un seul tambou Les deux derniers codes permettent, outre, de remplir le tambour d'ordre ("0") d'un programme préparé sur ruba
8	1	1	1	débordement du tambour d'ordres vers le tambour de nombres (lecture); permet le travail sur un seul tambour
3	1	1	0	ne sont guère utilisés
9	1	0	1	
4	1	0	0	

Remarques : 1) Lorsque la lecture des ordres et la lecture des nombres se font sur un même tambour, elles ne peuvent pas se faire simultanément si l'ordre de lecture des nombres est le second d'une paire, ce qui ralentit le calcul.

2) A la fin d'un programme, après le dernier ordre d'inscription sur ruban, il est prudent de modifier l'organisation des rubans (de façon quelconque), ce qui a pour effet d'interrompre le courant d'inscription et d'éviter de fausses manoeuvres lors de la manipulation ultérieure du ruban.

4. Repères sur les rubans

Dans les tambours, l'information est divisée en mots de 18 chiffres (nombres ou paires d'ordres) dont l'emplacement est caractérisé par une adresse de 4 chiffres. Sur les rubans, l'ensemble du mot et de son adresse forme une suite de 22 chiffres, l'adresse précédant le mot. Lorsqu'on transfère l'information d'un ruban vers un tambour, l'adresse précédant le mot dirige automatiquement ce mot vers l'emplacement qu'elle définit sur le tambour. Au contraire, lors de l'inscription d'un résultat sur un ruban, aucune adresse n'est nécessaire puisque les résultats s'inscrivent au fur et à mesure sur le ruban de résultats, qui chaque fois avance d'un mot. Dans le cas où l'on calcule un ruban de programme, qui sera ensuite déplacé et vidé dans le tambour, il est nécessaire de munir les mots d'adresses qui seront utilisées plus tard lors du transfert. D'autre part, lors de l'impression de résultats à partir d'un ruban, il est nécessaire de disposer d'instructions typographiques. Comme les adresses ne sont pas pertinentes dans un ruban de résultats numériques, l'emplacement correspondant est utilisé pour les instructions typographiques.

En dernier lieu, l'adresse de ruban est utile pour repérer la position d'un mot sur un ruban de données, c.à.d. pour décider d'une sélection de lecture sur les rubans connectés aux positions A et B. Comme un ruban peut contenir plus de 2.000 mots, les adresses se terminant par 0 et 5 ne suffisent pas. Les conventions suivantes permettent d'autres chiffres, sans détruire la correspondance entre les adresses de ruban et les emplacements que les mots sont destinés à occuper sur les tambours.

Les mots de ruban sont répartis en groupes, et les groupes sont divisés en tranches. On indique la fin d'un ruban (premier mot normalement vide, après

l'épaisseur, grâce à laquelle le ruban est devenu "sans fin") en additionnant 0002 à l'adresse du mot. On indique la fin d'un groupe en additionnant 0004 à l'adresse du dernier mot du groupe. On indique la fin d'une tranche en additionnant 0001 à l'adresse du dernier mot de la tranche. On indique fin de tranche et fin de groupe simultanément en additionnant 0003 à l'adresse du dernier mot de la dernière tranche du groupe. Deux mots d'un même groupe (et à fortiori d'une même tranche) ne peuvent avoir même adresse de ruban. Lorsqu'une adresse à laquelle on a ajouté un repère quelconque, est utilisée effectivement pour une sélection sur le tambour, le repère est non pertinent, mais dans tous les autres cas, c'est l'adresse modifiée par le repère qu'il faut considérer : ceci se présente notamment dans les ordres de déroulement de ruban à adresse donnée (§5.1) et les ordres de transfert jusqu'à adresse donnée (§5.2).

Outre la piste principale qui vient d'être décrite, les rubans comportent une piste auxiliaire dont le rôle sera décrit ultérieurement, ainsi qu'une base de temps n'intervenant pas dans cet exposé. Il est cependant important de savoir que cette base de temps accompagne l'information mot par mot, et est lue et inscrite avec celle-ci. En cas de surinscription sur un ruban non préalablement effacé, un décalage entre les deux bases de temps peut brouiller une lecture ultérieure. En cas de surinscription occasionnelle de quelques mots, ceci n'est pas dangereux car le décalage reste minime. En règle générale, on n'inscrira cependant que sur un ruban entièrement effacé au préalable. Ceci se fait en lui faisant faire un tour complet dans son mécanisme (en manoeuvrant à la main le bouton START-STOP du mécanisme) après avoir connecté ce ruban en position d'inscription et donné un ordre d'inscription quelconque pour établir le courant d'inscription.

5. Transferts

L'information des rubans connectés aux positions A et B peut être transférée vers le ruban de résultats connecté en position C, ou vers le tambour fonctionnant à ce moment comme tambour de nombres, ou vers l'imprimeur. Le transfert se fait pour un ou plusieurs mots et sans passer par le groupe calculateur. Pendant l'exécution d'un ordre de transfert, il est impossible de faire une inscription d'un résultat du groupe calculateur, ce qui empêche de poursuivre simultanément un calcul dans la plupart des cas. Le transfert est commandé par

un ordre de transfert choisissant la position de lecture A ou B et décidant de la destination du transfert (C, tambour, ou imprimeur). La lecture ou l'inscription sur ruban se fait toujours en commençant au mot suivant immédiatement la position actuelle des rubans, au moment où est donné l'ordre de transfert. Les modalités d'arrêt du transfert doivent être spécifiées au préalable (avant de donner l'ordre de transfert) par un ordre d'organisation du transfert. Si les rubans de lecture (position A et B) ne doivent pas être utilisés dans leur position actuelle, il faut les faire tourner sur eux-mêmes au préalable. Ceci est commandé par un ordre de déroulement jusqu'à un mot donné, qui sera effectué pendant que la machine calcule par ailleurs. Cette facilité n'est pas prévue pour le ruban de résultats, car il sera normalement rempli dans l'ordre naturel.

Le temps de lecture ou d'inscription sur ruban est d'environ 50 msec par mot, y compris le temps de démarrage et d'arrêt du ruban après chaque mot. La vitesse de frappe de l'imprimeur est de 10 caractères par seconde.

5.1. Déroulement

On peut dérouler un ruban jusque fin de ruban, fin de groupe, ou jusqu'à une adresse ξ (dans le groupe dans lequel on se trouve, sans quoi la spécification est ambiguë) et ce déroulement peut affecter le ruban en position A ou B. L'adresse ξ à spécifier est celle du mot précédant immédiatement le premier mot à transférer par la suite.

préfixe	adresse	déroulement jusque
92000	0000	fin ruban, A
91000	0000	fin groupe, A
93100	ξ	adresse ξ' selon ξ2, pos.A
97000	0000	fin ruban, B
96000	0000	fin groupe, B
98100	ξ	adresse ξ' selon ξ2, pos.B

5.2. Organisation du transfert

Celle-ci vaut jusqu'à nouvel ordre, et est nécessairement la même pour les deux positions de lecture A et B. On spécifie jusqu'où se fera le transfert (un mot, jusqu'à fin de tranche, jusqu'à adresse donnée de ruban, identifiée par comparaison avec le contenu d'un registre S préalablement rempli; dans ce dernier cas, il s'agit de nouveau du premier groupe où cette adresse se rencontre, sans quoi la spécification est ambiguë). La piste auxiliaire n'est éventuellement pertinente que dans le cas de transfert vers ruban ou imprimeur. Il est donc prévu des transferts sans piste auxiliaire : l'information auxiliaire du ruban dont on transfère n'est alors pas utilisée, mais la piste auxiliaire du ruban vers lequel on transfère (ou les organes équivalents de l'imprimeur) reçoivent alors une information constante, identique pour tous les mots jusqu'à nouvel ordre, à partir du registre électronique E (cfr. §6).

préfixe	adresse	interprétation
95i00	ξ	Si $\xi' = ****u$, u détermine la modalité du transfert selon le tableau ci-après
94i00	ξ	Envoi de ξ' dans le registre S, qui le conserve jusqu'à nouvel ordre

u	modalité de transfert
0	jusqu'à fin de tranche, avec piste auxiliaire
5	" " , sans piste auxiliaire
1	jusqu'à l'adresse S, avec piste auxiliaire
6	" " , sans piste auxiliaire
4	un seul mot, avec piste auxiliaire
9	" " , sans piste auxiliaire

5.3. Ordres de transfert

Ces ordres se composent d'un préfixe suivi d'une adresse vide. Le transfert vers un tambour est effectué sur celui où se font les inscriptions en vertu de l'organisation des tambours. Dans le cas de transfert vers tambour, l'adresse de ruban de chaque mot transféré décide automatiquement de l'emplacement sur le tambour, mais cette adresse de ruban ξ peut être modifiée en ξ' avant d'agir, si le préfixe de l'ordre de transfert contient $i \neq 0$. Il en est de même dans le cas de transfert vers imprimeur, la modification portant alors sur l'instruction typographique. Dans le cas de transfert de ruban vers ruban, l'adresse de ruban accompagnant le mot est également transférée, après modification éventuelle en fonction de i. Les règles de modification sont toujours celles du §2, mais limitées à $i \leq 5$.

préfixe	transfert du ruban	vers
50i00	A	imprimeur
53i00	A	ruban C
54i00	A	tambour
55i00	B	imprimeur
58i00	B	ruban C
59i00	B	tambour

5.4. Transfert dans les registres E et F

En plus des ordres de transfert indiqués aux paragraphes précédents, il est possible d'effectuer un transfert des rubans connectés aux positions A et B vers les registres E et F. L'organisation des transferts doit être : transfert d'un seul mot, sans piste auxiliaire. L'adresse accompagnant le mot sur le ruban est non pertinente. Ces ordres sont rarement utilisés.

préfixe	transfert du ruban	vers le registre
52100	A	E
52000	A	F
57100	B	E
57000	B	F

6. Registres associés aux rubans

Dans le cas des calculs sur rubans qui seront examinés au paragraphe suivant, l'entrée du groupe calculateur est raccordée à la position de lecture A ou B

Registre V: lorsqu'on a utilisé le contenu de V dans un ordre d'opération (p.ex. +* $\hat{V}$), V est vide et l'on ne peut s'en servir pour conditionner un renvoi du type Rv inh V = S ou Rv inh f.t.

Registre S:

Un ordre de déroulement du ruban jusqu'à une adresse § [93100 § ou 98100 §] modifie le contenu du registre S. Toute information mise préalablement dans ce registre est donc perdue à ce moment.

(selon le préfixe de l'ordre d'opération), et la sortie est raccordée au ruban des résultats C, les rubans remplaçant donc le tambour des nombres. Comme le mot de ruban contient 22 chiffres sur la piste principale et 18 sur la piste auxiliaire (l'adresse n'y est jamais pertinente), tandis que les circuits de calcul ne traitent que des mots de 18 chiffres, des registres supplémentaires sont prévus pour contenir l'information en excès. Ceux-ci sont :

V (4 chiffres) pour l'adresse de ruban de la piste principale

E (18 chiffres) pour le mot de la piste auxiliaire.

Le registre E est le même que celui dont il est question dans I,§3.2. Lors d'un calcul sur ruban où il intervient, il n'est donc plus disponible pour d'autres usages. Son information antérieure est détruite lors d'un transfert avec piste auxiliaire.

Lors d'une lecture sur ruban, l'adresse de ruban accompagnant le dernier mot lu reste dans le registre V, qui ne peut d'ailleurs jamais être rempli d'une autre façon. L'information auxiliaire entre éventuellement dans E, mais peut être bloquée de façon que E garde son information ancienne. En cas de transfert vers ruban ou imprimeur, c'est cette dernière qui est alors utilisée (cfr.§5.2).

Le registre V a un rôle semblable à celui des registres d'indices, bien que beaucoup plus restreint.

Il est possible de calculer sur le contenu de V d'une façon analogue à ce qui a été dit en I, § 8.5, cette adresse devenant un nombre V de la forme I, équ.(12). Pour sélectionner ce nombre comme donnée fraîche d'une opération, on remplace les préfixes (I,§3.1) des ordres d'opération par 025##, l'adresse étant vide. La lecture de V vide ce registre, mais celui-ci se remplit lors de la lecture du mot suivant de ruban.

Il est prévu de pouvoir conditionner un renvoi normal (d'un emplacement du tambour des ordres vers un autre) par le contenu de V :

préfixe	adresse	notation	renvoi inhibé si
43100	À	Rv À' inh V = S	V = S
48100	À	Rv À' inh ft	V contient une adresse de fin de tranche

7. Calculs sur les rubans

7.1. Ordres d'opération

En modifiant les trois premiers chiffres du tableau des préfixes de I, § 3.1 et en prenant une adresse vide, on sélectionne la donnée fraîche à partir du ruban A ou B, et on exécute l'opération définie par les deux

derniers chiffres du préfixe; en même temps l'adresse de ruban du mot sélectionné va dans V. Le contenu de la piste auxiliaire va dans E, ou n'est pas utilisé. L'adresse est non pertinente puisque la lecture se fait sur la position actuelle du ruban.

Lors du calcul sur les ordres (II § 2), il est impossible d'exploiter successivement les deux moitiés d'un mot, car le ruban saute au mot suivant après exploitation du premier demi-mot. Une suite d'ordres telle que celle de II § 2. équ.(4) est donc interdite lors de calcul sur ruban.

préfixe	modalités de la sélection
023**	ruban A, sans piste auxiliaire
028**	" ", avec piste auxiliaire
024**	ruban B, sans piste auxiliaire
029**	" ", avec piste auxiliaire

7.2. Ordres d'inscription

Le résultat ω se trouvent dans le groupe calculateur est envoyé sur la piste principale du ruban C. L'adresse γ mentionnée dans l'ordre devient une adresse de ruban γ' après modification en fonction de $\underline{i}$ selon le § 2, et est notée devant ω sur la piste principale. Le contenu de E est toujours inscrit sur la piste auxiliaire.

préfixe	adresse	remarques
08i96	γ	ω conservé dans gr.calc.
08i46	γ	ω effacé dans gr.calc.

8. Impression

8.1. Impression des rubans de programmes ou de données

L'imprimeur est muni d'une clé à trois positions marquée PROGR.NOMBRES - PROGR.ORDRES - RESULTATS. L'information amenée à l'imprimeur est toujours

constituée de deux séquences de 22 chiffres, séquences situées sur la piste principale et la piste auxiliaire si l'on imprime à partir d'un ruban (par un ordre de transfert), séquences définies par le code de l'ordre d'impression si l'on imprime directement à partir des circuits du calcul, conformément au §8.3.

Lorsque la clé est en position NOMBRES ou ORDRES, la mise en page est réalisée automatiquement, à raison d'un mot par ligne, sous la forme (1) ou (2).

Position NOMBRES :

$$\| \underbrace{x\ y\ z\ u}_{\text{adresse de ruban}} \quad \underbrace{s.\ a\ b\ c\ d \ldots\ \sigma\ p\ q}_{\text{nombre à virgule flottante}} \| \qquad (1)$$

Position ORDRES :

$$\| \underbrace{x\ y\ z\ u}_{\text{adresse de ruban}} \quad \underbrace{\text{préfixe, adresse}}_{\text{ordre n°1}} \quad \underbrace{\text{préfixe, adresse}}_{\text{ordre n°2}} \| \qquad (2)$$

L'information typographique de la piste auxiliaire (pas l'adresse principale) peut être utilisée, suivant le §8.2 ci-dessous, et se superpose à la mise en page automatique.

Si l'ordre de transfert est donné sans piste auxiliaire, c'est le contenu de E qui est utilisé comme information typographique de piste auxiliaire; si l'on désire imprimer simplement les mots sous la forme (1) ou (2), il faut que le contenu de E soit $+ 0.10^{+0}$. Les positions NOMBRES et ORDRES sont normalement utilisées pour imprimer les données numériques ou les ordres d'un programme dans un but de contrôle.

8.2. Impression des résultats

En position RESULTATS, l'adresse de ruban x y z u n'est pas imprimée (sans pour cela être remplacée par des espaces), et les chiffres x et y déterminent les instructions typographiques du mot selon le code ci-dessous. Les chiffres z et u ne sont pas pertinents.

symbole	signification
x = 2	dix retours de chariot } après le mot et les traits éventuels déterminés par y
x = 4	deux retours de chariot } après le mot et les traits éventuels déterminés par y
x = 5	un retour de chariot } après le mot et les traits éventuels déterminés par y
y = 2	‖ après le mot
y = 4	| après le mot
y = 5	‖ avant le mot
y = 7	‖ avant et après le mot
y = 9	‖ avant et | après le mot

Le code x = 0 signifie qu'on continue sur la même ligne (pas de retour de chariot), et le code y = 0 signifie qu'on n'ajoute aucun trait, ni avant ni après le mot.

L'information des 17 chiffres de la piste auxiliaire correspondant à la mantisse et à l'exposant détermine des instructions typographiques supplémentaires pour chaque chiffre correspondant de la piste principale (instructions typographiques de symbole), selon le tableau ci-dessous. Les quatre chiffres correspondant à l'adresse de ruban sur la piste auxiliaire ne sont pas pertinents.

symbole	signification
0	impression normale du chiffre
1	impression avec un espace avant le chiffre
2	impression avec point décimal avant le chiffre
4	suppression du chiffre (non remplacement par espace)

Cette information doit elle-même être introduite au départ comme nombres sous la forme (1) et, elle fait partie des données numériques, occupant donc la piste principale d'un ruban de données; elle sera transférée sur le tambour en même temps que ces données à l'adresse de ruban contenue dans le not. Au moment de la constitution du ruban de résultats, cette information doit être transférée du tambour vers le registre B, qui la déverse automatiquement vers la piste auxiliaire du ruban de résultats ou de l'imprimeur au moment d'une impression.

Au sujet des signes, il y a lieu de faire les remarques suivantes :

a) un signe + (tant s que ←) n'est jamais imprimé mais remplacé par un espace.

b) le signe de l'exposant et l'exposant lui-même ne sont pas à imprimer lorsque les résultats sont à virgule fixe. La suppression de l'exposant ne fait pas de difficulté (44 sur la piste auxiliaire en regard de l'exposant d'après le tableau ci-dessus). La suppression du signe de l'exposant s'obtient en notant (-) pour le signe s de nombre sur la piste auxiliaire. Le signe ← est non pertinent et sera noté +.

c) le signe du nombre ne peut être supprimé par information typographique.

8.3. Impression sans passer par le ruban de résultats

Les résultats des calculs peuvent être imprimés directement à partir du groupe calculateur et des registres électroniques. Dans ce cas, le nombre à imprimer est le contenu du groupe calculateur. L'adresse qui accompagne ce nombre, et qui sert d'information typographique de not, est l'adresse ξ contenue dans l'ordre d'impression, éventuellement modifiée en ξ' par l'intervention d'un registre d'indices. Le contenu de B passe comme information typographique de symboles (piste auxiliaire). Le code de l'information typographique et les positions de la clé de l'impression sont ceux des § 8.1 et 8.2.

préfixe	adresse	notation	remarques
05i96	ξ	→ P	conservé dans le gr.calc.
05i46	ξ	= P	effacé dans le gr.calc.

Les informations typographiques de symboles auront dû être introduites au préalable dans E et sont à considérer comme des données numériques à prévoir sur le ruban de données.

9. Pupitres

9.1. Généralités

La machine est desservie par un pupitre de commande représenté à la fig.1 avec des subdivisions numérotées de I à XIV pour faciliter les références. Le pupitre d'impression portant l'imprimeur et un clavier sera décrit au § 9.6.

Seules les parties du clavier du pupitre de commande utilisées en exploitation normale ont été représentées en détail à la fig.1. Les parties grises ou simplement esquissées servent à des indications de service et permettent diverses manoeuvres pour la recherche des fautes.

Les significations des couleurs des lampes du tableau sont les suivantes : une lampe rouge allumée indique une alarme; une lampe verte peut être allumée en fonctionnement normal; les lampes bleues et jaunes s'allument pour indiquer à l'opérateur qu'il a effectué certaines manoeuvres, les lampes jaunes correspondant à des conditions plus usuelles que les lampes bleues.

Les abréviations utilisées sur le pupitre diffèrent quelque peu de la terminologie adoptée dans ce manuel; CYL désigne le tambour (cylindre); le terme BANDES désigne les rubans magnétiques; l'abréviation IMPR (impression) dénote une inscription non seulement sur l'imprimeur, mais aussi sur tambour ou rubans.

Le pupitre de commande comprend :

(I) La commande de l'alimentation de la machine. En fonctionnement normal, la lampe verte SECTEUR MIS est allumée et les lampes jaunes EINSCHALTEN FIL et HT doivent être allumées (seul le personnel de maintenance est autorisé à manoeuvrer les boutons correspondants).

(II) Les commandes de synchronisation permettent de ramener la machine dans un état standard avant de commencer un calcul : voir § 9.2.

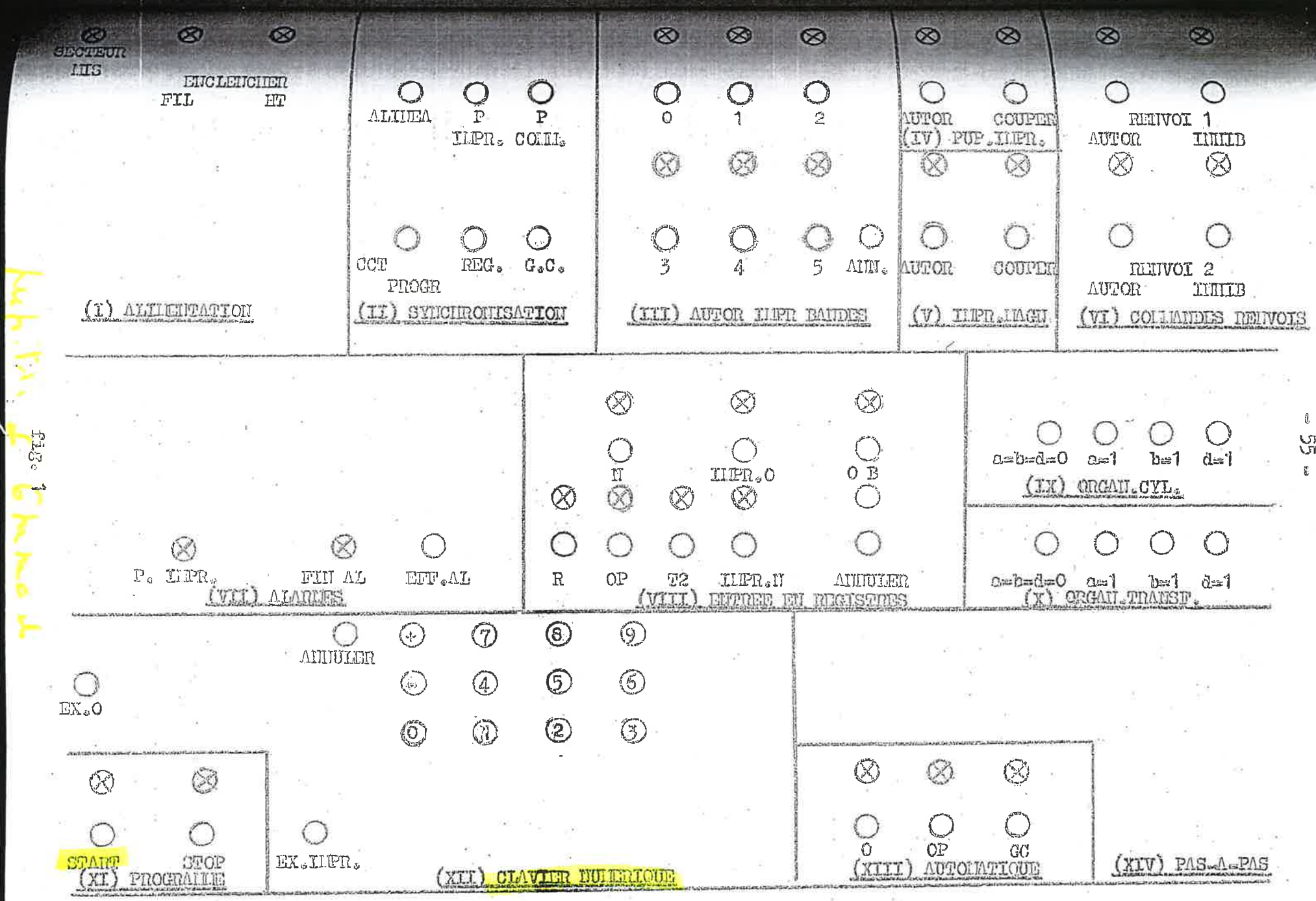
SECTEUR MIS
ENCLENCHER
FIL
ET
(I) ALIMENTATION
ALINEA
P IMPR.
P COMM.
CCT PROGR
REG.
G.C.
(II) SYNCHRONISATION
0
1
2
3
4
5
ANN.
(III) AUTOR IMPR BANDES
AUTOR
COUPER
(IV) PUP.IMPR.
AUTOR
COUPER
(V) IMPR.MAGN
RENVOI 1
AUTOR
INHIB
RENVOI 2
AUTOR
INHIB
(VI) COMMANDES RENVOIS
P. IMPR.
FIN AL
EFF.AL
(VII) ALARMES
N
IMPR.O
O B
R
OP
T2
IMPR.N
ANNULER
(VIII) ENTREE EN REGISTRES
a=b=d=0
a=1
b=1
d=1
(IX) ORGAN.CYL.
a=b=d=0
a=1
b=1
d=1
(X) ORGAN.TRANSF.
ANNULER
EX.O
START
STOP
(XI) PROGRAMME
EX.IMPR.
(XII) CLAVIER NUMERIQUE
0
OP
GC
(XIII) AUTOMATIQUE
(XIV) PAS-A-PAS

Fig. 1

(III, IV et V) Les autorisations d'inscription en mémoire décrites au § 9.3.

(VI) Le conditionnement manuel de certains ordres de renvoi : cfr. § 11.1 in fine.

(VII) Les lampes d'alarmes et le bouton d'effacement d'alarme. Aucune lampe rouge ne peut être allumée en fonctionnement normal. L'ordre de "fin avec alarme" (I § 11.2) allume la lampe FIN AL.

(VIII, IX, X et XII) Les dispositifs de commande manuelle de la machine comprenant notamment un clavier numérique pour la préparation des rubans de programme et de données (§ 9.4 et § 10).

(XI) Les commandes permettant de démarrer ou d'arrêter un programme. A l'arrêt la lampe jaune est allumée. En appuyant sur PROGR.START, on démarre le programme, ce qui éteint la lampe jaune et allume la lampe verte. Toute intervention doit alors être évitée, sauf en cas d'alarme. L'ordre de fin de programme arrête le programme, éteint la lampe verte et allume la lampe jaune.

(XIII et XIV) Les dispositifs utilisés pour la recherche des erreurs qui ne seront pas décrits ici. En fonctionnement normal, les trois lampes jaunes AUTOMATIQUE de (XIII) doivent être allumées.

9.2. Synchronisation

Avant toute intervention de l'opérateur modifiant l'exploitation, c.à.d. avant de commencer une préparation de ruban, ensuite avant de faire calculer la machine, etc, il importe de synchroniser celle-ci c.à.d. de la ramener dans un état standard où tous les registres électroniques sont remplis de zéros, le groupe calculateur étant organisé pour le calcul à virgule flottante. La synchronisation se fait en appuyant sur les six boutons de (II) dans leur ordre de succession ALINEA; P.IMP; P.COMM; CCT PROGR; REG; G.C. Pour obtenir la synchronisation des circuits de programme, le pupitre d'impression doit être enclenché et synchronisé.

Après toute synchronisation, on appuie sur le bouton 0 de la subdivision (III), ce qui allume la lampe jaune associée (voir explication au § 9.3).

9.5. Autorisation d'inscription

Aucune inscription magnétique (ni ruban, ni tambour) n'est effective tant qu'on n'a pas appuyé sur le bouton ILP.MAGN.AUTOR. de (V) ce qui allume une lampe jaune. Il en est de même pour l'imprimeur (bouton AUTOR de IV). Pour éviter l'inscription de signaux parasites, il est prudent d'appuyer sur les boutons COUPER de (IV) et (V), (allumage lampes vertes), pendant les périodes où les inscriptions correspondantes ne sont pas nécessaires. La synchronisation du pupitre de commande ramène automatiquement ILP.MAGN. en position COUPER. Afin d'éviter une distraction de l'opérateur, il est impossible de donner PROGR.START si l'on n'a pas autorisé les inscriptions magnétiques.

Indépendamment du choix par code de ceux des rubans 0,1...5 qui sont connectés aux positions A, B, C (cfr.§3), l'inscription sur un ruban n'est autorisée que si, en outre, le bouton correspondant de (III) a été enclenché (allumage lampe jaune). Plusieurs rubans peuvent être simultanément autorisés. On supprime toutes les autorisations en poussant sur ANNULER de (III), ce qui éteint toutes les lampes. Ceci se produit automatiquement lors de la SYNCHR. ou COM. Aucune autorisation n'est nécessaire pour la lecture.

Le choix de la connexion des rubans aux positions A, B, C est fixé par l'opérateur en envoyant l'adresse xyzw du premier ordre du § 3, vers le registre OB (cfr.§9.4.2). Après une synchronisation, OB est vidé et contient 0000, ce qui commande entre autres la connexion du ruban 0 en position C. Si 0 n'est pas autorisé dans (III), cette connexion est interdite et le sélecteur devant la réaliser tourne sans arrêt. Pour l'arrêter, on appuie sur 0 de (III). C'est pour cette raison que cette manoeuvre a été prescrite au § 9.2.

9.6. Commandes manuelles

Les dispositifs de commande manuelle permettent d'inscrire des mots ou parties de mots dans différentes mémoires, de donner et d'exécuter des ordres et de modifier l'organisation. Ces dispositifs ne peuvent être utilisés lorsqu'on se trouve sur PROGR.STOP.

Le clavier numérique sert à taper les mots comme il sera expliqué ci-dessous. Le bouton ANNULER de (XII) permet de détruire la partie déjà "tapée" d'un mot, en cas d'erreur de frappe de l'opérateur.

Le registre où doit se faire l'inscription est choisi à l'aide des boutons ENTREE en REGISTRE. Le bouton ANNULER de (VIII) supprime toutes les connexions.

Les manoeuvres que l'on a à exécuter normalement seront décrites séparément ci-dessous.

9.4.1. Organisation des tambours et des transferts

Pour introduire à la main une organisation des tambours ou des transferts, il n'est pas nécessaire de se préoccuper des ordres du § 3 ou du § 5.2. en entier, car la seule partie pertinente de ces ordres est le dernier symbole u de leur adresse. On manoeuvre donc les quatre boutons de (IX) ou (X), selon les modalités ci-après :

(a) appuyer sur le bouton a=b=d=0 (ceci détruit l'information ancienne)

(b) composer dans un ordre quelconque, à l'aide des trois autres boutons, le code correspondant au chiffre désiré pour u (dans le tableau du § 3 pour une organisation des tambours, ou dans le tableau du § 5.2 pour une organisation de transfert), selon le tableau ci-dessous :

u	a = 1	b = 1	d = 1
0			
1			*
4		*	
5	*		
6	*		*
8	*	*	*
9	*	*	

9.4.2. Organisation des rubans

Pour changer à la main l'organisation des rubans, il n'est pas nécessaire de se préoccuper du préfixe de l'ordre (§ 3) : il suffit d'entrer l'adresse xyz* dans OB de (VIII). Pour cela, on connecte OB au clavier numérique à l'aide du bouton de sélection ENTREE en REGISTRE et on tape l'adresse de quatre chiffres sur le clavier. On appuie ensuite sur ANNULER de (VIII) pour déconnecter OB du clavier. L'adresse envoyée dans OB y reste jusqu'à nouvel ordre.

9.4.3. Exécution d'un ordre

On introduit séparément l'adresse en N et le préfixe en OP par les manoeuvres suivantes : appuyer sur le bouton ENTREE EN REGISTRE N, taper l'adresse sur le clavier, appuyer sur le bouton ANNULER ENTREE EN REGISTRE puis sur le bouton OP, taper le préfixe sur le clavier, de nouveau ANNULER ENTREE EN REGISTRE. En appuyant finalement sur EX O de (XII), on fait exécuter l'ordre par la machine.

Si l'adresse de l'ordre est non pertinente, il n'est pas nécessaire de l'introduire dans N.

9.4.4. Inscription d'un mot-nombre sur ruban ou tambour

Il faut évidemment que le ruban soit en place dans un mécanisme et que celui-ci soit raccordé à la position d'inscription C (§ 3). Il faut, en outre, que le registre E soit vide. On doit ensuite introduire en permanence (pour tous les mots à inscrire) en OP le préfixe 53000 (transfert sur ruban) ou 54000 (transfert sur tambour, cfr. § 5.3). Ceci se fait par la succession des manoeuvres : ENTREE EN REGISTRE OP, taper le préfixe sur le clavier, ANNULER ENTREE EN REGISTRE.

Après ces préparatifs, on appuie sur ENTREE EN REGISTRE IMPR.N et on entre les mots un à un sur le clavier, en appuyant sur EX IMPR de (XII) après chaque mot. La manoeuvre de EX IMPR n'est rendue effective que si l'on a donné au préalable AUTOR.IMPR.MAGN.

On frappe sur le clavier le nombre précédé de son adresse, le mot ayant donc la forme (I) du § 8.1. Le groupement convenable est réalisé automatiquement,

mais il est indispensable de frapper s et σ aux bonnes places, même si les signes sont +. Il n'est pas nécessaire de taper les zéros de queue de la mantisse : la seconde inscription d'un signe l'interprète automatiquement comme signe de l'exposant et complète la mantisse à 15 chiffres par des zéros. On ne tape pas de point décimal puisqu'il est toujours convenu que celui-ci est en tête de la mantisse. Il faut inscrire les deux chiffres de l'exposant, même si ce sont des zéros.

Rappelons que dans une inscription sur ruban, l'adresse de mot est également inscrite, tandis que, dans une inscription sur tambour, elle disparait après avoir servi à déterminer la sélection.

9.4.5. Inscription d'un mot-ordre sur tambour ou ruban

Les manoeuvres sont les mêmes qu'au § 9.4.4 mais le bouton à choisir dans ENTREE EN REGISTRE est I.PR.O. On tape chaque mot sous la forme (2) du § 8.1 et on appuie ensuite chaque fois sur EX.I.PR.

9.4.6. Calcul des adresses pour l'inscription des mots-nombres sur ruban ou tambour

Si les adresses intervenant aux paragraphes précédents en tête des mots-nombres sont consécutives, il n'est pas nécessaire de les taper toutes : la machine peut les calculer, par les manoeuvres suivantes :

a) inscrire le premier mot-nombre (y compris l'adresse) par le procédé habituel (§ 9.4.4)

b) introduire le pas 0005 dans le registre d'indice G en appuyant le bouton ENTREE EN REGISTRE R, en tapant sur le clavier numérique le nombre 0005 0000 0000 0000 et en appuyant EFFACER ENTREE EN REGISTRE (le bouton R permet d'introduire des valeurs numériques dans les quatre registres d'indice simultanément en tapant les quatre valeurs l'une à la suite de l'autre, dans l'ordre G, H, I, K).

c) introduire en OP le code 53100 ou 54100 au lieu de 53000 ou 54000 au § 9.4.4, suivant qu'on inscrit sur ruban ou sur le tambour, par les manoeuvres indiquées dans ce paragraphe.

d) appuyer le bouton ENTRER EN REGISTRE T_2 et ensuite, pour chaque mot, taper uniquement le signe du nombre, la mantisse (sans les zéros de queue), le signe de l'exposant, les deux chiffres de l'exposant et appuyer le bouton EX. IMPR.

Ce procédé n'est pas valable pour l'inscription des mots-ordres, car il n'est pas prévu que l'on puisse introduire directement une paire d'ordres dans le registre T_2.

9.5. Pupitre d'impression

Le clavier du pupitre se présente comme indiqué à la fig.2

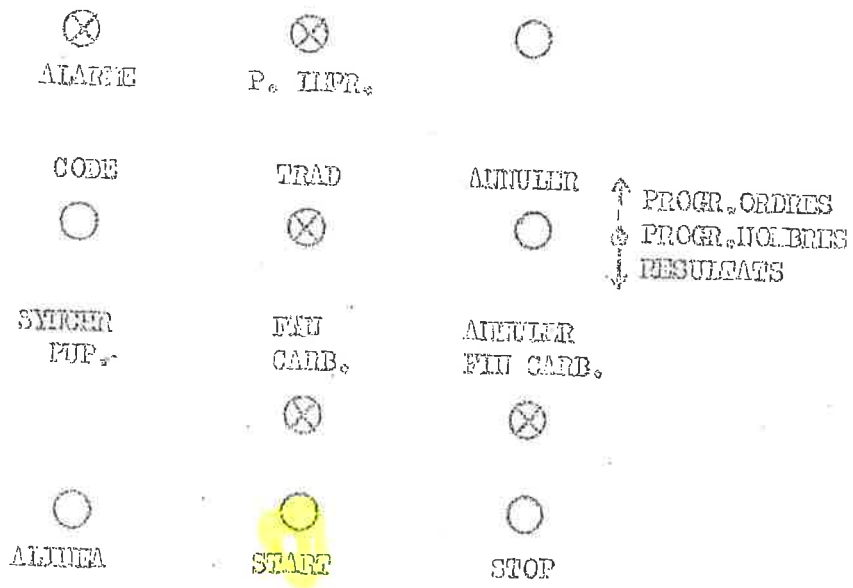

fig.2

Les boutons START et STOP ne font que doubler, pour la commodité d'accès, ceux du pupitre de commande marqués PUP.IMPR. respectivement AUTOR et COUPER. Les boutons ALINEA et SYNCHR doublent ceux de même nom du pupitre de commande (II).

Une des lampes rouges ALARME P.IMPR s'allume lorsque l'imprimeur a imprimé un mot faux, l'allumage de CODE signifiant manque de contrôle du check digit (voir description technique), et l'allumage de TRAD signifiant une erreur de traduction dans le sélecteur de l'imprimeur. Après avoir noté l'erreur, on redémarre l'imprimeur et on éteint les lampes en poussant sur le bouton ANNULER.

la lampe rouge FIN CARB s'allume lorsque le ruban de carbone est à bout de course. On remplace la bobine et on appuie sur le bouton ANNULER FIN CARB pour redémarrer l'imprimeur et éteindre la lampe.

Les trois alarmes qui viennent d'être mentionnées provoquent également l'allumage de la lampe rouge P.IMP dans (VII) du pupitre de commande.

La clé à trois positions est celle décrite au § 8.

L'imprimeur est muni d'un dispositif de contrôle des retours du chariot. Les instructions de retours de chariot sont données par les informations typographiques de mise en page (§ 8.2), mais il faut en plus donner l'autorisation d'effectuer un retour de chariot aux différents emplacements où il peut s'en produire dans la largeur de la page (lignes de longueurs différentes). Ceci se fait en plaçant des taquets, derrière le rouleau de la machine à écrire, à l'aplomb des emplacements possibles de fin de lignes.

10. Préparation des rubans

10.1. Généralités

Nous supposons que le ruban à préparer sera constitué de la façon suivante : il commencera par un mot vide d'adresse de ruban 0002 (fin ruban), suivi de la suite des mots-ordres constituant un groupe d'une ou de plusieurs tranches (au sens du § 4), elle-même suivie de la liste des mots-nombres constituant aussi un groupe d'une ou de plusieurs tranches. On n'inscrit aucune indication typographique sur la piste auxiliaire. Nous supposons que le ruban a été effacé et mis en bonne place (un peu avant début ruban) sur son mécanisme. Pour fixer les idées, nous choisirons le mécanisme n°5. Après inscription des mots sur le ruban, on en imprime normalement le contenu pour vérification (l'inscription simultanée sur ruban et imprimeur est impossible). Pendant la vérification, qui peut être assez longue, la machine sera normalement occupée à un autre problème. Si des erreurs ont été constatées, on attendra pour les corriger que la machine redevienne libre, et la correction (§ 10.5 ci-dessous) constitue donc une opération indépendante (avant laquelle il faudra synchroniser la machine). La synchronisation est aussi supposée faite avant la préparation proprement dite, dont la description occupe les §§ 10.2 à 10.4.

10.2. Choix du ruban et préparation de l'inscription

(a) Appuyer sur OB; frapper 0150 sur le clavier (ceci connectera le ruban 5 en position C et deux autres rubans arbitraires aux positions A et B); appuyer sur ANNULER de (VIII) (cfr. §9.4.2).

(b) Appuyer sur ANNULER de (III) pour détruire les connexions anciennes et ensuite sur 5 de (III) et sur IMPR.MAGN.AUTOR de (V).

(c) Appuyer sur OP; frapper 53000 sur le clavier (c'est le préfixe de l'ordre de transfert sur ruban du §5.3); appuyer sur ANNULER de (VIII) (cfr. §9.4.3, l'adresse n'est pas pertinente).

10.3. Inscription des mots-ordres

Appuyer sur IMPR.O de (VIII). La lampe associée doit rester allumée pendant toute la durée de l'inscription des mots-ordres. Chaque mot est composé comme dans §9.4.5 et est inscrit sur le ruban en appuyant sur EX.IMPR de (XI). A la fin de la liste d'ordres, on appuie sur ANNULER de (VIII) pour éteindre la lampe IMPR.O.

10.4. Inscription des mots-nombres

Appuyer sur IMPR.N de (VIII), ce qui allume la lampe associée pendant toute la durée de l'inscription. Chaque mot est composé comme dans §9.4.4 et est inscrit sur le ruban par EX.IMPR. de (XI). Finalement ANNULER (VIII).

10.5. Impression d'un ruban pour contrôle

La suite des manoeuvres ci-dessous a pour but de connecter le ruban n°5 en position de lecture A et d'organiser et commander le transfert vers l'imprimeur tranche par tranche.

(a) COUPER IMPR. MAGN. dans (VI) et ANNULER (III), puis appuyer 0 de (III)(cfr.c)

(b) synchroniser

(c) envoyer l'adresse 5100 dans OB, puis annuler VIII.

Ceci connecte le ruban n°5 en A et deux autres rubans arbitraires en B et C.

(d) envoyer le préfixe 92000 dans OP, sans annuler ensuite et appuyer sur EX.O de XI, (recherche fin ruban, cfr. §5.1). On voit s'arrêter le ruban lorsqu'il est en place.

(e) mettre la clé de l'imprimeur sur PROGR.ORDRES (cfr. § 8.1).

(f) envoyer le préfixe 50000 (transfert de A vers imprimeur, cfr. § 5.3) dans OP.

(g) appuyer sur EX.O. Le transfert de la première tranche s'effectue et on constate son achèvement par l'arrêt de l'imprimeur.

(h) s'il y a plusieurs tranches de mots-ordres, on répète (h) autant de fois qu'il est nécessaire. Après la dernière tranche, on met la clé de l'imprimeur sur PROGR.NOMBRES et on appuie de nouveau sur EX.O autant de fois qu'il y a de tranches de mots-nombres.

10.6. Correction des erreurs

Nous supposerons d'abord qu'il s'agit de corriger un seul mot à la fois. Comme ce n'est qu'en position de lecture qu'on peut mettre un ruban en place en y recherchant une adresse donnée (§ 5), il faudra passer de cette position à la position d'inscription alternativement pour chaque correction. Après synchronisation, les manoeuvres sont les suivantes :

(a) connecter le ruban n°5 en position A selon § 10.5.c.

(b) déroulement jusque fin ruban selon § 10.5.d.

(c) déroulement éventuel jusque fin groupe (si la correction est dans le second groupe) de la même façon, mais avec le préfixe 91000.

(d) déroulement jusqu'à l'adresse de ruban ξ du mot précédent le mot à corriger en envoyant l'ordre 93000 ξ (cfr. § 3) dans OP et N selon § 9.4.3. Ensuite EX.O.

(e) connecter le ruban n°5 en C et organiser le transfert vers le ruban selon § 10.2.a, b et c.

(f) surinscrire le nouveau mot complet selon le § 10.3 ou § 10.4.

Pour les corrections suivantes, reprendre à partir de (a) en omettant (b) et éventuellement (c).

Si un ou plusieurs mots ont été complètement omis, ou si une longue tranche est fausse, la correction par surinscription est impossible, et il faut composer un nouveau ruban par transferts partiels à partir du ruban erroné.

11. Transfert et exécution d'un programme

11.1. Généralités

A partir de l'adresse 0005 sur le tambour d'ordres, seront inscrites en permanence quelques paires d'ordres permettant le démarrage de la machine avec un minimum de manoeuvres; elles constituent le programme de démarrage. Après synchronisation, il suffit d'introduire l'adresse 0005 dans le registre M et d'appuyer le bouton PROGR.START de (XI), pour commencer l'exécution du programme.

Le programme de démarrage est conçu pour organiser un transfert du ruban en position 5 vers le tambour des ordres, depuis début ruban jusqu'à fin de tranche. C'est donc la tête de la première tranche du ruban qui doit contenir les ordres d'organisation ultérieurs nécessaires pour les étapes suivantes, qui sont normalement :

(a) transfert du groupe entier des mots-ordres vers le tambour d'ordres,

(b) transfert du groupe entier des mots-nombres vers le tambour de nombres,

(c) contrôle des transferts,

(d) exécution du programme.

L'ensemble (a), (b), (c) constitue le programme de transfert et contrôle.

Le contrôle des transferts peut se faire en principe de deux façons :

(a) En imprimant les mots transférés et comparant par transparence les listes de mots avec celles obtenues lors du contrôle du ruban (§10.5). Ceci demande du temps et immobilise partiellement la machine pendant cette comparaison, puisque les tambours sont occupés. Il est cependant permis pendant ce temps de préparer un autre ruban, puisque les tambours n'interviennent pas dans les opérations du §10.

(b) Par répétition de l'opération

$$\text{mot ruban} - \text{mot tambour} = 0$$

C'est ce contrôle qui sera expliqué en détail au §11.4.

Lors de la première utilisation d'un nouveau programme, il peut être utile d'imprimer des résultats intermédiaires pour déceler les fautes éventuelles, ou d'arrêter la machine après un calcul partiel pour chronométrer la durée de ce calcul. A cet effet, on intercale dans le programme des renvois vers des

séquences contenant ces ordres exceptionnels. En exploitation normale, ces renvois sont inhibés par la manoeuvre des boutons (VI). Il y a deux commandes indépendantes, et on rend un renvoi sensible à la position de l'une ou de l'autre de ces commandes en lui attribuant un code spécial :

préfixe	adresse	symbole	renvoi exécuté si
44i00	λ	Rv λ' selon 1	RENVOI 1 AUTOR
49i00	λ	Rv λ' selon 2	RENVOI 2 AUTOR

11.2. Démarrage

(a) Synchroniser la machine et le pupitre de commande.

(b) Mettre le ruban programme-données du problème à traiter dans le mécanisme 5 un peu avant début-ruban. Mettre un ruban effacé pour servir de ruban résultats dans un autre mécanisme p.ex. 0.

(c) Autoriser IMPR.MACH. dans (V), et IMPR.PUP. dans (IV) s'il y a lieu. Il n'est plus nécessaire d'autoriser l'inscription sur le ruban 0 puisque cette opération fait partie de la synchronisation (par convention).

(d) Mettre l'adresse 0005 dans M.

(e) Appuyer PROGR.START dans (XI).

11.3. Programme de démarrage

Les paires d'ordres inscrites en permanence sur le tambour d'ordres sont

adresse	ordre 1		ordre 2	
0005	99000	5400	92000	0000
0010	54000	0000	99000	54000

Le premier ordre organise la connexion des rubans 5 en A et 0 en C (en outre 4 en B, arbitraire). Le second commande le déroulement jusque fin ruban, en position A. Le troisième commande le transfert du ruban en A vers le tambour d'ordres. Du fait de la synchronisation, on a en effet a=b=d=0 pour l'organisation des tambours, ce qui établit toutes les connexions vers le tambour d'ordres,

et l'on a également a=b=d=0 pour l'organisation des transferts, ce qui organise les transferts jusqu'à fin de tranche et avec piste auxiliaire, mais celle-ci est non pertinente dans le cas d'un transfert sur tambour et ne remplira pas le registre E puisqu'elle est identiquement nulle sur un ruban de données. Le quatrième ordre répète le premier, mais est indispensable pour des raisons techniques.

Pour introduire initialement le programme de démarrage à la main dans le tambour d'ordres, procéder comme suit à partir de l'état synchronisé de la machine.

(a) Envoyer le préfixe 54000 (transfert vers tambour) dans OP selon §9.4.3 ANNULER (VIII).

(b) IMPR.MAGN.AUTOR.

(c) Envoyer le premier mot du programme de démarrage selon §9.4.5. Appuyer EX.IMPR. de (XI).

(d) Répéter (c) pour le second mot

(e) ANNULER (VIII) (pour vider OP et revenir à a=b=d=0).

11.4. Programme de transfert et de contrôle

Nous considérons le cas le plus simple et le plus fréquent où les mots-ordres constituent un groupe composé d'une tranche unique et où il en est de même des mots-nombres. Nous supposons aussi que les mots-ordres (y compris ceux du programme de transfert et de contrôle lui-même qui en forment le début) ont des adresses de ruban consécutives à partir de 0015 de façon à ce qu'après transfert, ils suivent le programme de démarrage. Nous supposons de même que les mots-nombres ont des adresses de ruban consécutives. Les modifications à imposer au cas où une des hypothèses précédentes n'est pas vérifiée seront discutées tout à la fin.

Du fait de l'exécution du dernier ordre du programme de démarrage et de l'hypothèse de la tranche unique, le transfert des mots-ordres de ruban vers tambour est terminé. Le ruban est alors en bonne place pour le transfert des nombres et celui-ci est commandé après avoir connecté les tambours en connexion normale, de façon que l'inscription se fasse sur le tambour de nombres.

On revient ensuite au début du ruban (on y est presque) et l'on compare mot par mot, les informations du ruban et celles transférées sur le tambour.

Pour la comparaison des mots-ordres, on transfère un mot entier du ruban dans le registre F et on le compare ensuite au contenu du tambour demi-mot par demi-mot en utilisant les codes prévus pour le calcul sur les ordres (II§2). De cette façon, on n'est pas obligé de parcourir deux fois le ruban pour comparer les ordres 1 et les ordres 2. Les calculs peuvent s'exécuter sans inconvénient à virgule flottante. Le passage d'un mot au mot suivant est automatique sur le ruban. Sur le tambour, la sélection est commandée en faisant appel au registre Z_1 et ajoutant 0005 à son contenu au début de chaque pas. En cas d'erreur, on provoque une fin avec alarme. Le contrôle est arrêté à la fin de tranche.

A l'achèvement du contrôle des ordres, le ruban est automatiquement en bonne place pour le contrôle des mots-nombres qui se fait de la même façon mais, cette fois, il n'est pas nécessaire de passer par le registre F. Comme l'organisation est toujours à virgule flottante, la différence rub-tamb dans E est maintenant 0.10^p avec p quelconque (exposant du nombre en cours).

Avant le contrôle des nombres, on revient à la connexion normale des tambours. Après le contrôle, le programme s'exécute immédiatement.

Le programme de transfert et contrôle se compose des ordres suivants :

adresses	ordres	
0002	vide vide	indication fin ruban
0015	connex.tambours 0006 tft rub.A sur tambour	transfert mots-nombres
0020	déroul.rub.A 0050 connex.tambours 0005	contrôle mots-ordres
0025	org.transferts 0055 - Z_1	sans piste aux. ; 1 mot

0030	Tft rub.A en F	
	+ OR1 (0000 + Z_1)	
0035	- OR1 F	
	+ OR2 (0000 + Z_1)	
0040	= OR2 F	
	= E	
0045	- mod	
	+* E	
0050	= ch	
	Rv- 0090	Alarme
0055	Z_1 + 0005 = Z_1	
	Rv inh ft 0030	
0060	connex.tambours 0006	contrôle mots-nombres
	$\beta = Z_1$	
0065	+* (0000 + Z_1)	
	-* rub A	
0070	= E	
	- mod	
0075	+* E	
	= ch	
0080	Rv- 0090	Alarme
	Z_1 + 0005 = Z_1	
0085	Rv inh ft 0065	
	Rv 0095	
0090	fin alarme	
	vide	
0095	...	suite du programme

Remarques

1. Le contrôle se fait à partir de l'ordre 0055 du présent programme, car cela n'aurait pas de sens de commencer avant.
2. L'adresse β est l'adresse du premier mot-nombre sur le tambour.
3. Le registre Z_1 contient à tout moment l'adresse du mot que l'on contrôle, ce qui permet de repérer facilement une faute éventuelle.

4. Après exécution de ce programme, les tambours sont organisés suivant la connexion normale (adresse 0006); les rubans sont organisés selon le programme de démarrage (5400 dans le cas présent); les transferts se font sans piste auxiliaire et mot par mot; les registres E, F, Z_1 ne sont plus vides.

Le codage complet de ce programme est donné ci-dessous :

adres. de ruban

0002	00000	0000	00000	0000
0015	90000	0006	54000	0000
0020	93000	0050	90000	0005
0025	95000	0009	06600	0055
0030	52000	0000	04168	0000
0035	02078	0000	04169	0000
0040	02079	0000	07146	0000
0045	00082	0000	02166	0000
0050	07446	0000	41000	0090
0055	01100	0005	48000	0030
0060	90000	0006	06600	β
0065	04166	0000	02376	0000
0070	07146	0000	00082	0000
0075	02166	0000	07446	0000
0080	41000	0090	01100	0005
0085	48000	0065	40000	0095
0090	47000	0000	00000	0000

Dans le cas où le programme à transférer se compose de plusieurs tranches et qu'au passage d'une tranche à l'autre les adresses font un saut, il y a lieu de modifier le programme ci-dessus en donnant un ordre de transfert par tranche et en organisant chaque fois convenablement le contenu de Z_1. Il faut aussi veiller à mettre le ruban correctement en place au début du contrôle.

12. Programme d'impression

12.1. Généralités

Les résultats obtenus au cours d'un calcul sont normalement notés dans l'ordre naturel, à virgule flottante, et sans information typographique. Nous supposerons que ces résultats se trouvent sur ruban (ruban préliminaire de résultats), l'adresse pouvant être non pertinente, mais en tous cas divisés en tranches munies de signaux de fin de tranche. Les modifications à introduire dans ce qui suit sont minimes, si les résultats se trouvent dans le tambour de nombres.

Comme les résultats imprimés doivent normalement se présenter en virgule fixe, il y a lieu de faire un calcul de transformation. Celui-ci se fait en exploitant dans l'ordre, tranche par tranche, les résultats du ruban préliminaire et les reportant dans le même ordre sur le ruban définitif. En même temps, on inscrit l'information typographique de not sur l'adresse de la piste principale et celle de symbole sur la piste auxiliaire du ruban définitif. On reporte également d'un ruban à l'autre les indicatifs de fin de tranche. Au § 12.2, nous donnons un exemple de programme de préparation du ruban définitif.

Supposant corrects les résultats du ruban préliminaire (ils ont dû être contrôlés au cours du calcul même), il importe encore de comparer le ruban préliminaire au ruban définitif : c'est le programme de contrôle du ruban définitif du §12.3. En dernier lieu, il n'y a plus qu'à exécuter un transfert du ruban définitif vers l'imprimeur. Celui-ci peut être fait deux fois, de façon à contrôler par transparence les résultats imprimés.

12.2. Préparation du ruban définitif

A titre d'exemple, supposons que les résultats soient à imprimer sous la forme ab,cdef avec 4 chiffres seulement après la virgule. Ceci n'est possible que s'ils sont inférieurs à 10^2, ce qui est supposé admis. Par addition sans normalisation de 0.10^2 (cfr.I §3.6) qu'on tiendra en permanence dans F, les résultats prennent la forme

$$, abcdefghi... \quad 10^2$$

et c'est sous cette forme qu'ils seront inscrits sur le ruban définitif, mais les chiffres de tête peuvent être des zéros (si cela était indésirable, un calcul préalable plus compliqué est repris). Selon le §8.2, les résultats seront imprimés sous la forme désirée, si le nombre inscrit sur la piste auxiliaire est

$$- \,,\, 00200044\ldots + 44$$

que nous supposons mis dans E au début du programme de préparation de façon qu'il s'inscrive à chaque mot (cfr. §7.2). Il y a lieu, en outre, d'inscrire sur la piste principale du ruban définitif une adresse de la forme xyOu donnant les instructions typographiques de mot selon le §8.2 et reportant dans u les indicatifs de fin de tranche du ruban préliminaire. A chaque pas, on a donc la suite d'ordres

+ rub.préliminaire

+ * F

= rub.définitif; xyOu

L'adresse xyOu du dernier ordre est à choisir d'après une série de conditionnements selon la position du mot dans la page, et cela à partir d'un comptage des mots. Une grande variété de mise en page est possible, et nous nous contenterons d'illustrer la programmation par un exemple. Nous supposerons que l'on inscrit l mots identiques par ligne, seuls le premier et le dernier mot de chaque ligne étant munis (respectivement précédé ou suivi) de traits particuliers. Nous supposerons en outre qu'il y a a mots par alinéa et p mots par page. Chaque alinéa et chaque page peuvent commencer et finir par des instructions spéciales.

Une ligne comporte au maximum 60 caractères (y compris espaces, traits, etc). La séparation entre alinéas est normalement donnée par un double retour de chariot (ce qui laisse une ligne en blanc entre alinéas), et une page comporte normalement 50 lignes (y compris les blancs, les titres des colonnes, etc). Un comptage est organisé sur les quatre registres d'indices que nous désignerons ici par Z_m(mot), Z_l(ligne), Z_a(alinéa) et Z_p(page). Les compteurs sont vides

au départ, et on y ajoute respectivement 1, l, a et p avant chaque mot, ligne, alinéa ou page. Au début du programme, on part donc de $Z_m = 1$, et on aura $Z_m = m$ avant le $m^{ième}$ mot. Comme $Z_l = l$ au début, la différence $-Z_m + Z_l$ est positive pour $m < l$ et change de signe (donnant -0 selon I §3.3) pour $m = l$, donc avant le $l^{ième}$ mot, c.à.d. avant le dernier mot de la première ligne nécessitant un aiguillage vers un programme spécial. Ce programme renvoie à l'endroit de l'entrée où on ajoute 1 au contenu de Z_l qui devient donc 2l et au second tour l'aiguillage se fera vers le programme spécial de fin de ligne avant le $2l^{ième}$ mot. Le même processus s'applique pour a et p en suivant l'ordre hiérarchique de p, a, l. La structure du programme est donc celle de la fig.3

Voici quelques remarques supplémentaires sur ce programme :

(a) Les séquences D, K, H, I, J contiennent la séquence + rub | ++F | = rub mentionnée ci-dessus commandant l'inscription d'un mot, la seule différence étant l'adresse xy00. Ainsi, si l'on désire disposer les résultats en colonnes verticales séparées par des traits simples, et indiquer par des traits doubles les bords de gauche et de droite de la page, les adresses seront celles du tableau ci-dessous, on y a aussi montré la forme sous laquelle le mot apparaissait

séquence		xy00	mot	
D	début ligne	0900	||ab.cdef|	
H	mot normal	0400	ab.cdef|	
I	fin ligne	5200	ab.cdef|| suivi de	1 retour ch.
J	fin alinéa	4200	ab.cdef|| suivi de	2 retours ch.
K	fin page	2200	ab.cdef|| suivi de	10 retours ch.

(b) La séquence D ajoute l'unité à Z_m puisqu'elle contient l'inscription d'un mot (le premier de la ligne). La séquence E ajoute l'unité à Z_m en prévision de l'inscription d'un mot dans une des séquences H, I, J, K dont une et une seule est nécessairement exploitée.

(c) Dans le schéma suivant, il n'apparait aucune prévision pour terminer le programme p.ex. en fin de tranche. Ceci peut se faire en intercalant un renvoi

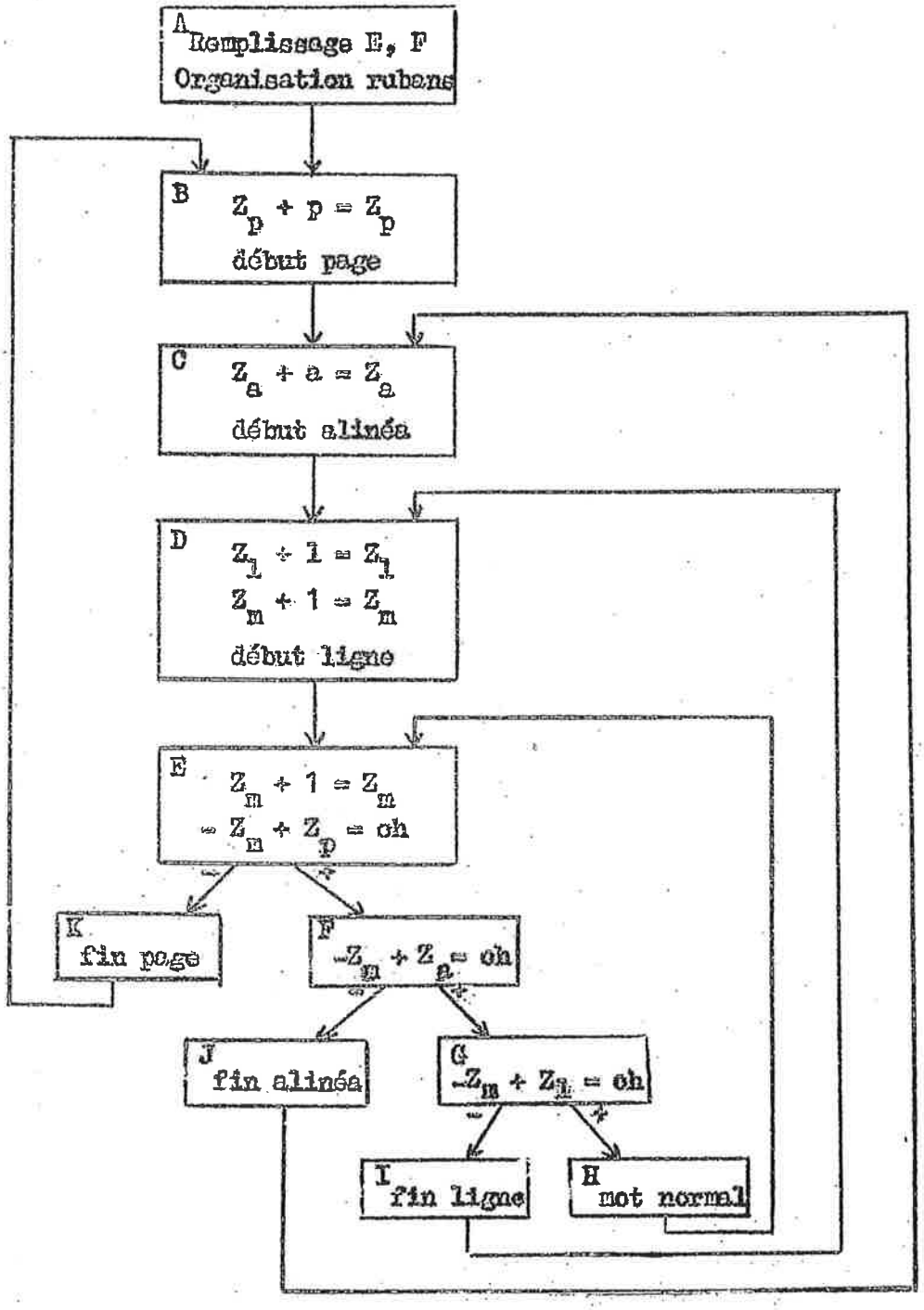
A
Remplissage E, F
Organisation rubans
B
$Z_p + p = Z_p$
début page
C
$Z_a + a = Z_a$
début alinéa
D
$Z_l + l = Z_l$
$Z_m + 1 = Z_m$
début ligne
E
$Z_m + 1 = Z_m$
$- Z_m + Z_p = oh$
K
fin page
F
$- Z_m + Z_a = oh$
J
fin alinéa
G
$- Z_m + Z_l = oh$
I
fin ligne
H
mot normal

fig. 3

dans une séquence d'inscription selon :

a	+ rub
b	Rv d inh ft
c	Rv x
d	+ *F
e	= rub; xy00

x	+ *F
y	= rub xy01
z	suite du programme

Si le mot lu en a n'est pas ft, on saute de a à d et le programme est identique à celui décrit jusqu'ici. Si le mot lu en a est ft, on saute à la séquence spéciale z; par l'ordre y, le dernier mot est inscrit avec le symbole ft sur le ruban définitif et le programme est achevé, ou répété pour une autre tranche. L'inscription de la paire d'ordres b, c doit être prévue dans une ou plusieurs des séquences (normalement J ou K), selon l'endroit ou les endroits de la page où peuvent tomber les fins de tranche.

(d) Les séquences "début page" et "début alinéa" dans B et C sont prévues pour pouvoir noter un numéro de page (obtenu par $Z_p \times 1/p$) et des titres de colonnes, ou un numéro d'alinéa.

12.3. Contrôle du ruban définitif

En calculant (rub prélim - rub définitif), mot par mot en virgule flottante, on obtient 0.10^2 dans l'exemple traité au § 12.1. En effet, le mot du ruban préliminaire a un exposant non supérieur à 2, et c'est lui qui s'aligne éventuellement sur le mot du ruban définitif, de sorte que, malgré l'amputation éventuelle du mot définitif, la différence est nulle. On peut donc accumuler les différences dans le groupe calculateur, et calculer d'un coup

$$\sum(\text{rub prélim} - \text{rub définitif}) = 0 ?$$

comme au § 11.4, pour les mots-nombres.

Si, ne fût-ce qu'un résultat préliminaire avait un exposant supérieur à 2, cette inadvertance se constaterait par l'apparition d'une différence 0.10^p avec $p > 2$, et l'exposant p dominerait dans la somme à partir de ce moment. Par altération exno et comparaison avec 2, cette inadvertance pourrait donc se déceler.

13. Le registre V

La machine contient un registre supplémentaire de 4 chiffres appelé V dont nous n'avons pas parlé jusqu'ici, car ses emplois sont assez limités. Il joue un rôle analogue aux registres d'indices (comptage, etc...) mais il est impossible de calculer sur le contenu de V autrement que par l'intermédiaire du groupe calculateur.

On peut remplir V à partir de l'adresse α de l'ordre de remplissage, ou à partir d'un nombre (dernier résultat ω) amputé en $\hat{\omega}$ selon I §8.2, selon un des trois premiers ordres du tableau ci-dessous. On peut calculer sur le contenu de V d'une façon analogue à ce qui a été dit en I §8.5, cette adresse devenant un nombre V de la forme I équ.(18). Pour sélectionner ce nombre comme donnée fraîche d'une opération, on remplace les préfixes (I §3.1) des ordres d'opération par le dernier préfixe du tableau ci-dessous.

préfixe	adresse	
0770	α	$\alpha = V$
07296	0	$\hat{\omega} \rightarrow V$
07246	0	$\hat{\omega} = V$
022**	0	opération sur donnée fraîche de V

Il est important de savoir que le contenu de V est détruit après chaque transfert ~~ou impression~~ avec piste auxiliaire, car ces transferts se font à travers V, exactement comme pour le registre E. ~~Lorsqu'on fait un transfert sans piste auxiliaire ou une inscription sur ruban ou sur imprimeur,~~ le ~~registre V doit être vide.~~

Chapitre IV

Programmes des fonctions élémentaires

voir p 80

1. Généralités

Les séquences contenant la programmation des fonctions élémentaires sont normalement notées à demeure à partir de l'adresse 8265 jusqu'au bout du tambour d'ordres. De même l'extrémité du tambour de nombres, à partir de l'adresse 9400 est réservée en permanence pour noter les constantes numériques intervenant dans ces séquences, et contient quelques cases de travail nécessaires au calcul des fonctions élémentaires.

Dans les problèmes spécialement longs où les fonctions élémentaires n'interviennent pas ou n'interviennent pas toutes, la place correspondante de mémoire peut être récupérée en tout ou en partie (en y surinscrivant d'autres informations), quitte à restaurer ensuite l'information relative aux fonctions élémentaires par un transfert à partir du ruban des fonctions élémentaires.

L'appendice I donne des renseignements complets sur l'information contenue dans le ruban des fonctions élémentaires et sur la façon de la transférer en tout ou en partie sur les tambours. Le § 3 ci-dessous explique la façon dont on renvoie aux séquences de ces fonctions au cours d'un programme.

L'appendice II justifie en détail la construction des séquences à partir des méthodes de calcul décrites dans l'article de V.Belevitch et F.Storrer "Le calcul numérique des fonctions élémentaires dans la machine IRSIA-FNRS" (Bull.Acad.Roy.Belg.Cl.des Sc.5ésérie, t.XLII.pp.543-578) auquel il sera renvoyé en abrégé par CNFE. Sauf indication contraire, les notations de cet article sont conservées, mais certaines valeurs numériques ont été affectées d'un exposant différent ainsi qu'il est expliqué dans l'appendice II.

2. Ruban des fonctions élémentaires

Le ruban des fonctions élémentaires comporte 471 mots, dont 106 nombres et 364 paires d'ordres. Les ordres forment le premier groupe après le début de

ruban, et ce groupe comporte deux tranches. La première (adresses 0015 à 0095) est le programme de transfert et de contrôle et devient inutile une fois exécutée; la seconde (adresses 8265 à 9995) est le programme proprement dit. L'adresse du dernier mot de la première tranche est modifiée en 0096 pour signifier fin de tranche, et celle 9995 de la fin du groupe des ordres est modifiée en 9998 pour indiquer fin de tranche et fin de groupe.
Les nombres forment un second groupe et leurs adresses de ruban sont numérotées de 9400 à 9925. La dernière adresse est de nouveau modifiée en 9926 pour indiquer fin de tranche.

Si le premier groupe est entièrement transféré dans le tambour des ordres et le second entièrement dans le tambour des nombres, aux adresses respectives mentionnées dans les rubans et si l'on oublie le programme de contrôle, l'information se répartit comme représenté aux tableaux I et II. Ceux-ci appellent les remarques suivantes :

(a) Pour les nombres, certaines constantes numériques intervenant dans plus d'une fonction sont réunies aux adresses 9400-9445. Les séparations en trait plein séparent ensuite les constantes intervenant respectivement dans x^{-1}, $x^{-\frac{1}{2}}$, sin x, Arc tg x, exponentielles et logarithmes. On trouvera les valeurs numériques dans CNFE. La tolérance pour x^{-1} et $x^{-\frac{1}{2}}$ a été forcée à la valeur commune 5.10^{-14} mise dans les constantes universelles. Les autres tolérances ont été légèrement forcées.

(b) La séparation en trait interrompu dans les constantes de sin x met en évidence le fait que les quatre premières n'interviennent que dans le contrôle. Comme, d'autre part, le contrôle de arc tg x fait appel au calcul du sinus, mais sans contrôle, les dernières constantes du sinus y interviennent seules, et forment une séquence continue avec les constantes de arc tg. Signalons enfin que l'Arc tg x et les exponentielles font appel à la séquence de l'inverse.

Tableau I

Emplacement des nombres sur le tambour*

canal / secteur	94	95	96	97	98	99
00	1	a_1	a_5	T	w_0	a_{11}
05	2	T	a_7	$10^{1,0}$	w_1	a_{12}
10	0.10^{15}	4	a_9	$10^{1,1}$	w_2	a_{13}
15	$1/2\pi$	3	a_{11}	$10^{1,2}$	w_3	a_{14}
20	10^{-8}	1/3	a_{13}	$10^{1,3}$	w_4	π
25	5	a_1	a_{15}	$10^{1,4}$	w_5	T
30	10	a_3	a_{17}	$10^{1,5}$	w_6	vide
35	$0,05.10^{01}$	a_5	0.10^{02}	$10^{1,6}$	w_7	
40	5.10^{-14}	a_7	50	$10^{1,7}$	w_8	
45	0.10^{04}	a_9	100	$10^{1,8}$	log 2	cases de travail
50	a_0	a_{11}	0.10^{-49}	$10^{1,9}$	a_1	
55	b_0	a_{13}	$10^{0,01}-1$	T'	a_2	
60	a_1	a_{15}	$10^{0,02}-1$	a_1	a_3	
65	a_2	10^{15}	$10^{0,03}-1$	a_2	a_4	
70	a_3	$\pi/2$	$10^{0,04}-1$	a_3	a_5	
75	$\sqrt{10}$	$\pi/6$	$10^{0,05}-1$	a_4	a_6	
80	3/2	T	$10^{0,06}-1$	a_5	a_7	
85	a	a	$10^{0,07}-1$	a_6	a_8	
90	b	b	$10^{0,08}-1$	49	a_9	
95	c	a_3	$10^{0,09}-1$	g	a_{10}	

* Les constantes sont présentées ici en notation traditionnelle (au lieu de la notation en virgule flottante réellement utilisée) sauf lorsque la forme flottante est essentielle (constantes explicitement non normalisées).

12

	99	98	97	96	95	94	93	92	91	90	89	88	87	86	85	84	83	82
00	log	log	log	log	exp	exp	exp	exp	exp	Arc tg	Arc tg	Arc tg	sin	sin	racine	racine	inverse	
05																		
10									10									
15																		
20																		
25																		
30																		
35																		
40																		
45																		
50																		
55																		
60																		
65					94					54			26	18	36			1
70					1					1					1			
75					log					exp			sin	sin	sin			inverse
80																		
85																		
90					10								43				26	
95													1				1	
99	94																	

Tableau III

Fonction	Information à transférer		Cases de travail
	Gr.1. (Ordres)	Gr.2. (Nombres)	
inverse	8265-8390	9400-9470	9945-9950
racine	8395-8570	9400-9445 et 9475-9500	9990-9995
sinus	8575-8785	9400-9445 et 9505-9560	9980-9995
Arc tg	8265-8390 et 8790-9055 et 8660-8700	9400-9470 et 9525-9630	9975-9995 9945-9950
expon.	9060-9525 et 8265-8390	9400-9470 et 9635-9790	9945-9995
logar.	9530-9995	9400-9445 et 9795-9925	9960-9995

(c) Pour les ordres, la succession des séquences est la même que pour les nombres et la numérotation des paires d'ordres dans les séquences individuelles est rappelée. Les séparations en trait interrompu mettent en évidence, dans la séquence du sinus, la partie intervenant pour le contrôle de Arc tg (mots 18-26), et dans l'exponentielle et le logarithme, les entrées subsidiaires, discriminant p.ex. entre log x et log (1+x).

(d) Si dans un programme donné, on ne désire faire appel qu'à certaines séquences, l'information n'est utilisée que partiellement, tant pour le groupe 1 (ordres) que pour le groupe 2 (nombres), ainsi qu'il ressort du tableau III.

Ce tableau indique également les cases de mémoire de nombres utilisées comme cases de travail dans chaque cas, cases ne pouvant donc servir à d'autres usages dans un programme utilisant la séquence en question, du moins aux moments où la séquence risque d'être appelée.

3. Instructions d'emploi

Le tableau IV ci-dessous résume les renvois à effectuer pour obtenir les diverses fonctions élémentaires, et donne la durée moyenne approximative du calcul, contrôle compris.

Fonction	Adresse λ du renvoi	Durée en sec
$1/x$	8265	0,3
$1/\sqrt{x}$	8395	0,5
sin x	8575	1,0
Arc tg x	8790	1,3
$10^x - 1$	9060	1,5
10^x	9105	1,5
$\log_{10}(1+x)$	9530	1,5
$\log_{10} x$	9575	1,5

Les séquences de calcul de toute fonction élémentaire f(x) supposent que l'état initial (avant de donner l'ordre de renvoi) du groupe calculateur est $\omega = x$ (c'est au programme principal à veiller qu'il en soit bien ainsi) et conduisent à l'état final $\omega = f(x)$. Le renvoi est à noter dans le programme principal selon les instructions de I, §8.6.c a.à.d. en donnant la paire d'ordres

préfixe	adresse	signification
01900	0000	0 + M = K
40000	λ	Rv λ

l'adresse λ étant choisie selon le tableau IV. Il faut veiller à ce que la paire d'ordres forme bien une paire dans le programme principal, quitte à insérer un ordre vide. Le retour au programme à l'ordre suivant immédiatement l'emplacement quitté est produit automatiquement par l'ordre Rv K, de code 40400 0000, figurant dans chacune des séquences des fonctions élémentaires.

Les programmes de toutes les fonctions élémentaires utilisent les registres E, F, ch et K, mais n'utilisent aucun autre registre électronique.

4. Remarques sur la structure des séquences

Les structures utilisées pour le contrôle sont celles de I §§9-10 lorsque le contrôle achevant le calcul d'une fonction échoue, le calcul est repris, le nombre maximum de reprises autorisées étant de 3.

Les cases de travail de la mémoire des nombres sont arbitrairement désignées r_1, r_2,... r_n et sont disposées consécutivement dans le canal 99 de façon que r_n soit le dernier secteur 9995. Pour toutes les fonctions, r_1 sert de mémoire à l'argument et r_2 au nombre de reprises. Pour l'inverse, on a exceptionnellement pris $r_1 = 9940$, $r_2 = 9945$ de façon à ne chevaucher avec aucune case de travail d'autres séquences; cela est nécessaire car la séquence de l'inverse intervient comme sous-séquence éventuelle dans plusieurs autres séquences (Arc tg, exp).

Pour aucune des fonctions élémentaires, la séquence ne vérifie si le calcul est légitime a priori s.à.d. si l'argument est aligné, s'il est positif pour une racine carrée, etc.... Un calcul illégitime est effectué, mais ne satisfera normalement pas au contrôle. La positivité de l'argument pour un logarithme est cependant explicitement vérifiée.

Pour ne pas préoccuper le programmateur qui utilisera les séquences des fonctions élémentaires, on a standardisé dans ces séquences l'usage du seul registre d'adresse X, les autres registres restant donc toujours libres pour le programme principal. Dans certains cas (notamment pour les séquences utilisant l'inverse), un renvoi avec mémorisation de l'adresse quittée (I, § 8.6.a) intervient dans une séquence de fonction élémentaire, ce qui nécessite un second registre d'adresses. Dans de tels cas, on emploie encore X, l'ayant libéré au préalable de l'information qu'il contenait vers une case de travail de la mémoire magnétique; après retour à la séquence principale, le contenu primitif de X est ensuite restauré.

Ch. V. - Compilation

1. Définition

La préparation d'un programme particulier où interviennent des sous-programmes usuels (comme la résolution des équations linéaires, l'analyse harmonique, etc, trop longs pour être mémorisés en permanence sur le tambour) demande que l'on incorpore dans le programme à préparer des sous-programmes disponibles une fois pour toutes, quitte à y modifier certaines constantes ou adresses pour que le sous-programme vienne s'insérer en bonne place et que les données puissent être disposées sur le tambour de nombres de la façon convenant au mieux à la disposition de l'ensemble des données adoptées pour le problème particulier. Au lieu de faire cette adaptation sur papier, on peut faire exécuter les modifications correspondantes par la machine. Il faut alors que les sous-programmes de routine soient disponibles une fois pour toutes sur un ruban (ruban de routine) et que le ruban de données du problème particulier soit constitué en partie de mots propres à ce problème, en partie de tranches transférées du ruban de routine avec modifications éventuelles. On appelle compilation l'opération constituant un ruban de données définitives par ce procédé. Les mots particuliers et les instructions de transfert et de modification constituent alors le ruban de données particulier. A partir de celui-ci, et du ruban de routine, la machine constitue un ruban de données compilé, et l'imprime pour contrôle visuel par l'opérateur. Ce dernier ruban est ensuite traité comme au chap. III §11.

2. Attribution des rubans

Les mécanismes de ruban sont attribués aux divers types de ruban comme suit :

1 ruban de routine
2 ruban de données compilé
4 ruban de résultats définitif
5 ruban de données particulier
0 ruban de résultats préliminaires

3 restant comme réserve. Dans le cas d'une exploitation sans compilation, 2 reste libre et 5 est directement rempli par l'opérateur.

Les connexions des mécanismes aux positions A B C se suivent donc dans l'ordre suivant au cours de l'exploitation; les chiffres entre parenthèses indiquent que le ruban connecté est arbitraire, de façon à avoir A ≠ B ≠ C.

	A	B	C
(a) <u>préparation avec compilation</u>			
préparation proprement dite	(0)	(1)	5
compilation	5	1	2
impression pour contrôle	2	(1)	(5)
permutation manuelle des bacs 2 et 5			
(b) <u>préparation sans compilation</u>			
préparation proprement dite	(0)	(1)	5
impression pour contrôle	5	(1)	(0)
(c) <u>exécution</u>			
transfert, exécution, calcul des résultats préliminaires	5'	(4)	0
transfert résultats prélim. vers résultats définitifs	0	(3)	4
contrôle et impression	0	4	(3)

<u>Remarque</u>. La notation 5' dans (c) signifie qu'on donne l'ordre d'organisation pour 5. Si le programme a été préparé par compilation, le ruban est alors en réalité le n°2 car on les a permutés à la main. Cette opération est nécessaire parce que le programme de démarrage appelle 5 dans tous les cas.

3. Structure du ruban de routine et modifications à prévoir

Chaque routine indépendante se compose d'une séquence de mots ordres et d'une séquence de mots nombres. Dans chaque séquence, les adresses sont consécutives à partir de 0000. Chaque séquence forme un <u>groupe</u> séparé. Le ruban de

routine comprend toutes les séquences de mots-ordres et ensuite toutes les séquences de mots-nombres, dans la même disposition relative, car c'est dans cette succession qu'il sera consulté. Il est essentiel qu'il n'y ait nulle part de signaux de fin de tranche, pour que le programme compilé constitue une tranche unique pour les ordres et une tranche unique pour les nombres, ce qui facilite son transfert ultérieur sur le tambour.

Lors de la programmation d'une routine, la description associée mentionnera clairement les cases de travail (de préférence les mêmes que pour les fonctions élémentaires) et les registres utilisés.

Aucune routine ne contiendra de programme de transfert initial ni d'impression, ceux-ci constituant des routines séparées, communes à tous les programmes.

Pour les mots-nombres, il n'y a normalement à prévoir lors de la compilation qu'un décalage constant des adresses de ruban, ce qui se fait facilement au moyen du registre d'indice en ordonnant un transfert avec changement d'adresse

Pour les mots-ordres, outre ce même décalage, il y a lieu de prendre certaines précautions pour les renvois et les constantes de comptage, et pour l'affectation des registres.

Les renvois internes dans la routine ne doivent pas être modifiés lors d'une compilation si, dans la programmation de la routine, on a pris la précaution de les noter en adresse relative c.à.d. sous la forme Rv(α + M), donc si l'on renvoie de α mots en avant ou en arrière (ch.I § 8.6.a) par rapport à celui (M) qu'on exploite.

Les renvois externes avec mémorisation de l'adresse quittée (p.ex. vers les séquences de fonctions élémentaires) doivent être notés comme expliqué au ch.I § 8.6.c, sans autre précaution.

Pour le comptage des constantes qui peuvent varier d'une application à l'autre (p.ex. nombre d'équations dans un système linéaire) et, plus généralement, pour toute adresse employée comme nombre pouvant varier, il importe de noter cette adresse comme 0000 dans la routine. Lors de la compilation, cette adresse est modifiée par addition en la valeur désirée. Ceci se fait par un calcul sur les ordres, directement de ruban vers ruban, pour les deux demi-mots de chaque mot à modifier. Le transfert normal doit donc être arrêté au mot précédent (transfert jusqu'à une adresse S cfr.ch.III § 5.2).

Pour les registres, il conviendra de préparer un diagramme d'occupation dans chaque cas, et d'introduire éventuellement des modifications par calcul sur les ordres.

4. Routines initiale et finale

Dans tout programme, il faut incorporer le programme initial de transfert et de contrôle (ch.III §11.4) et le programme final d'impression (ch.III § 12). Les routines correspondantes forment le premier et dernier groupe de la partie mots-ordres du ruban de routine, et ne comportent pas de mots-nombres.

La routine initiale est exactement le tableau du §11.4, où l'on fait β = 0000, si l'on admet une fois pour toutes que les mots-nombres de tout problème commencent à l'adresse 0000. En outre le dernier mot est muni du signal de fin de groupe.

La routine finale comprend les ordres du tableau ci-dessous. Elle est notée en adresse flottante et les séquences A à K rappellent le schéma du ch.III §12.2. Il a été admis que l'information typographique de mot et la constante 0.10^p de réalignement des résultats étaient respectivement placées aux adresses 0000 et 0005 pour tout problème, lors de la préparation du ruban des mots-nombres. Il a été admis, dans le programme ci-dessous, que l'achèvement de l'impression coïncidait nécessairement avec une fin d'alinéa, ce qui est le cas normal. La séquence L est celle notée x y z au ch.III §12.2.c. Les séquences M N Q correspondent au §12.3. La séquence P commande les impressions finales.

A	0	org.rub.	ABC = 034; cfr.§5.2.c 2ième ligne
		dér.fin rub.A	remise au début du rub.prélim.de résultats
	5	= rub 0002	inscr.signal fin rub.sur rub.définitif de résultats, supposé vierge et mis en place
		+*0000	
	10	= E	inform.typogr. dans E
		+*0005	
	15	= F	constante 10^p dans F
		vide	

B	20	vide $Z_1 + 0000 = Z_1$	0 sera remplacé par p lors de la compilation
C	25	vide $Z_2 + 0000 = Z_2$	0 sera remplacé par a
D	30	vide $Z_3 + 0000 = Z_3$	0 sera remplacé par l
	35	$Z_4 + 0001 = Z_4$ $+ {}^*$rub A	
	40	$+ {}^*$ F = rub ; 0900	0900 = typo déb.ligne
E	45	$Z_4 + 0001 = Z_4$ $= {}^*Z_4$	
	50	$+ {}^*Z_1$ = ch	
	55	Rv (H + 0060)	60 + 60 = 120 ⟶ K
F		$= {}^*Z_4$	
	60	$+ {}^*Z_2$ = ch	
	65	Rv (H + 0030)	70 + 30 = 100 ⟶ J
G		$= {}^*Z_4$	
	70	$+ {}^*Z_3$ = ch	
	75	Rv (H + 0010)	80 + 10 = 90 ⟶ I
H		+ *rub A	
	80	+ *F = rub ; 0400	0400 = typo not normal
	85	Rv (H + 9955) vide	90 - 45 = 45 ⟶ E

I	90	$+^*$ rub A	
		$+^*$ F	
	95	= rub; 5200	5200 = typo fin ligne
		Rv (II + 9930)	100 - 70 = 30 ⟶ D
J	100	$+^*$ rub A	
		Rv(II + 0005) inh ft	105 + 5 = 110 } cfr
	105	vide	} III § 12.2.c.
		Rv(II + 0020)	110 + 20 = 130 }
	110	$+^*$ F	
		= rub; 4200	4200 = typo fin alinéa
	115	Rv(II + 9905)	120 - 95 = 25 ⟶ C
		vide	
K	120	$+^*$ rub A	
		$+^*$ F	
	125	= rub; 2200	2200 = typ fin page
		Rv(II + 9890)	130 - 110 = 0020 ⟶ B
L	130	$+^*$ F	
		= rub; 2201	2201 = type fin page et ft
	135	org.rub.	A B C = 043 cfr. § 2.o dern.ligne
		dér.fin rub A	} remise à déb.rub. pour les deux
	140	dér.fin rub B	} rubansde résultats
		vide	
M	145	+ rub A	
		- rub B	
	150	Rv(II + 9990)inh ft	155 - 10 = 145 ⟶ M
N		= E	
	155	- mod	
		+ E	
	160	= ch	
		Rv(II + 0020)	165 + 20 = 185 ⟶ Q (Alarme)

p	165	org.transf.ft	avec piste auxiliaire
		dér.fin rub B	1ère impression
	170	transf.B ---> impr	pour repréparer l'imprimeur
		fin	
	175	dér.fin rub B	2ième impression
		transf.B ---> impr	
	180	fin	
		vide	
Q	185	fin alarme	erreur transfert ruban résultats
	ft	vide	

Remarque : Les ordres 20, 25, 30 comprennent un ordre vide suivi d'un ordre de remplissage des différents registres, pour les raisons suivantes :

1°) Ces ordres doivent être accessibles par des renvois; l'autre ordre de la paire doit donc être vide.

2°) Si l'ordre vide est le premier de la paire, la modification de l'ordre, lors de la compilation, peut se faire en travaillant uniquement sur l'ordre 2, parce que une sortie OR2 qui n'est pas précédée d'une sortie OR1 donne une paire dont le premier ordre est vide (cfr.II §2). L'inverse n'est pas possible.

On utilise cette remarque dans l'exemple donné au §5 du présent chapitre (ordres n + 10 et suivants).

Le codage détaillé est le suivant

0000	99000	0340	92000	0000
0005	03046	0002	04116	0000
0010	07146	0000	04116	0005
0015	07046	0000	00000	0000
0020	01100	0000	00000	0000

0025	01200	0000	00000	0000
0030	01300	0000	00000	0000
0035	01400	0001	02366	0000
0040	02066	0000	08046	0900
0045	01400	0001	03476	0000
0050	03166	0000	07446	0000
0055	41500	0060	03476	0000
0060	03266	0000	07446	0000
0065	41500	0030	03476	0000
0070	03366	0000	07446	0000
0075	41500	0010	02366	0000
0080	02066	0000	08046	0400
0085	41500	9955	00000	0000
0090	02366	0000	02066	0000
0095	08046	5200	49500	9930
0100	02366	0000	48500	0005
0105	00000	0000	40500	0020
0110	02066	0000	08046	4200
0115	40500	9905	00000	0000
0120	02366	0000	02066	0000
0125	08046	2200	40500	9890
0130	02066	0000	00046	2201
0135	99000	0430	92000	0000
0140	97000	0000	00000	0000
0145	02316	0000	02426	0000
0150	48500	9990	07146	0000
0155	00082	0000	02116	0000
0160	07446	0000	41500	0020
0165	95000	0000	97000	0000
0170	55000	0000	42000	0000
0175	97000	0000	55000	0000
0180	42000	0000	00000	0000
0188	47000	0000	00000	0000

5. Exemple de compilation

Nous considérons un problème dont le programme est entièrement particulier et comporte x mots-ordres et y mots-nombres. On demande ce qu'il faut noter en outre sur le ruban de données particulier au problème pour qu'il se transforme par compilation en ruban définitif de données de façon que les mots-ordres soient précédés par la routine initiale et suivis de la routine finale avec des valeurs données de p, l, a. Ce cas est assez académique, mais illustrera bien le procédé.

Pour les raisons expliquées au début du § 3, il est essentiel que le ruban définitif ne comprenne qu'une tranche pour les mots-ordres et une pour les mots-nombres. A cause de cela, on n'a pu munir le ruban de routine de signaux ft qui, sinon, se reporteraient dans le ruban définitif. La compilation doit donc se faire par transfert jusque V = S.

Le ruban particulier à préparer à la main devra donc se présenter comme suit (les explications suivent) :

02	vide	signal fin ruban
15	connex.rub.pour compilation	A B C = 512 cfr. § 2.a
	dér.fin rub B	(recherche routine initiale)
20	organis.transfert:	jusque S, sans piste auxiliaire
	0090 = S	
25	= rub 0002	inscription sur rub C, signal fin rub.
	transf.rub B → C	(transfert routine initiale)
30	0095 = Z_1	
	[5(x-1)] = S	
35	transf.rub A → C; i = 1	(transfert des x mots-ordres particuliers, aux adresses de 95 à 5x + 90)
	dér.fin cr. rub B	(à répéter autant de fois qu'il y a de routines sur le ruban de routines, jusqu'à ce qu'on soit au droit de la routine finale)
	⋮	
u	5x + 95 = Z_1	
	0015 = S	
u + 5	transf.rub B → C; i = 1	transfert des 4 premiers mots-ordres de la routine finale, les adresses sur le ruban définitif commençant à 5x + 95

	OR2 s + rub B	transfert du mot 20 de la routine finale, avec l'inscription de p; le premier demi-mot est vide et on ne s'en préoccupe pas.
u + 10	+* p	
	OR2 s = rub; i = 1	
u + 15	OR2 s + rub B	
	+* q	id. mot 25
u + 20	OR2 s = rub; i = 1	
	OR2 s + rub B	
u + 25	+* r	id. mot 30
	OR2 s = rub; i = 1	
u + 30	OT08 = 5	
	transf. rub B→C; i = 1	transfert suite routine finale
u + 35	5(y-1) = 5	
	transf. rub A→C; i = 0	transf. des y mots-nombres sans changement d'adresse
u + 40	fin	
ft	vide	
0 à 5(x-1)		mots-ordres particuliers au problème
0 à 5(y-1)		mots-nombres
ft		

La tranche d'adresses 0 à u + 40 constitue les instructions de compilation. En démarrant la machine, le programme de démarrage la vide dans le tambour d'ordres, puis l'exécute. La seconde moitié (nombres et ordres particuliers du programme ci-dessus, ne comporte pas de signal ft. à la séparation des ordres et des nombres, mais un signal à la fin. On vérifiera facilement qu'après compilation, le ruban en position C contient le programme définitif en deux tranches l'une formée des mots-ordres tels qu'ils devront être exécutés (le signal ft unique vient du transfert du dernier mot de la routine finale), la seconde des mots-nombres, le signal ft venant cette fois du dernier mot du programme ci-dessus.

L'impression du ruban compilé, pour contrôle, sera commandée à la main comme au ch.III §10.5 puisqu'il faut manoeuvrer la clé de l'imprimeur.

Appendice I

Information notée sur le ruban des fonctions élémentaires

fin rub.	0002	00000 0000	00000 0000
progr. de	8180	54000 0000	90000 0006
transf. et	8185	54000 0000	00000 0000
contrôle	8190	93000 8261	90000 0000
	8195	95000 0009	06600 8265
	8200	52000 0000	04168 0000
	8205	02078 0000	04169 0000
	8210	02079 0000	07146 0000
	8215	00082 0000	02166 0000
	8220	07446 0000	41000 8260
	8225	01100 0005	48000 8200
	8230	90000 0006	06600 9400
	8235	04116 0000	02376 0000
	8240	07146 0000	00082 0000
	8245	02166 0000	07446 0000
	8250	41000 8260	01100 0005
	8255	48000 8235	42000 0000
fin tranche 1	8261	47000 0000	00000 0000

progr. x^{-1}	8265	09096 9945	07046 0000
	8270	03076 0003	09046 9950
	8275	00012 0000	02016 0000
	8280	04036 9450	04016 9455
	8285	00092 0000	02036 0000
	8290	00022 0000	02036 0000
	8295	07196 0000	02035 0000
	8300	04016 9460	02136 0000
	8305	07196 0000	02035 0000
	8310	04016 9465	02136 0000
	8315	07196 0000	02035 0000
	8320	04016 9470	02136 0000
	8325	07196 0000	02035 0000
	8330	04016 9405	02136 0000
	8335	07196 0000	02035 0000
	8340	04016 9405	02136 0000
	8345	07196 0000	04036 9945
	8350	04076 9400	07046 0000
	8355	04076 9440	00032 0000
	8360	02066 0000	07446 0000
	8365	02116 0000	41400 0000
	8370	07146 0000	04016 9945
	8375	07046 0000	04066 9950
	8380	03066 0001	09096 9950
	8385	07446 0000	41000 8275
	8390	47000 0000	00000 0000
Progr. $x^{\frac{1}{4}}$	8395	09046 9990	03076 0003
	8400	09046 9995	00012 0000
	8405	04016 9990	07096 0000
	8410	04036 9485	04016 9490

Appendice II

Analyse des séquences des fonctions élémentaires

1. Programme de transfert et de contrôle

0002	vide vide	fin-ruban
8180	transf.rub. A ⟶ tamb.	transfert-ordres
8185	connex.norm.tamb. transf.rub. A ⟶ tamb. vide	transfert-nombres
8190 8195	déroulement jusque § = 8261 connex.initiale tamb. organisation transf. $Z_1 = \alpha$ (1er ordre, soit 8265)	contrôle tranche ordres
8200 8205 8210 8215 8220 8225	transfert ⟶ F + OR1 (Z_1) - OR1 F + OR2 (Z_1) - OR2 F = E - mod +* E = ch Rv- Alarme 8260 $Z_1 + 5 = Z_1$ Rv inh ft 8200	

8230	connex, norm, tamb.	contrôle tranche nombres
	$Z_1 = \beta$ (1er nombre, soit 9400)	
8235	$+ (Z_1)$	
	$=$ ruban	
8240	$=$ E	
	$=$ mod	
8245	$+$ E	
	$=$ ch	
8250	Iv Alarme 8260	
	$Z_1 + 5 = Z_1$	
8255	Iv inh 2% 8235	
	fin	
8260	Alarme	
-	vide	

Remarque : ce programme est analogue à celui du chap.III (§11.4), la seule différence étant que le groupe de mots-ordres comporte deux tranches, la première (le programme de contrôle lui-même) n'étant pas contrôlée. En outre, après achèvement du contrôle, on donne l'ordre de fin normale.

2. Séquence de l'inverse

2.1. Structure de la séquence (fig.1)

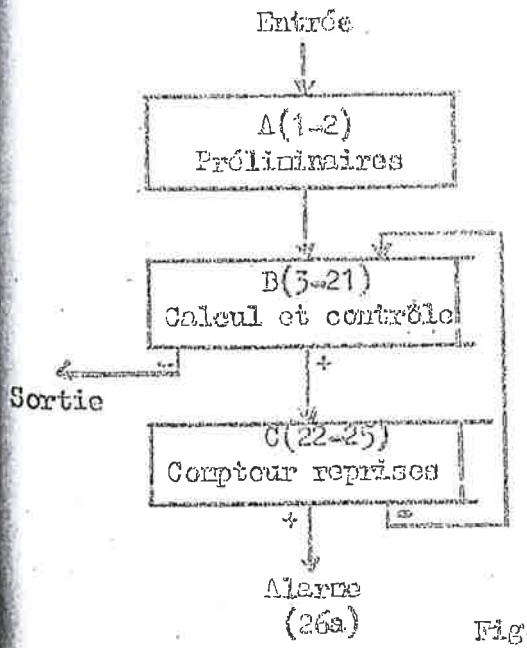

Fig.1

Les préliminaires consistent en l'envoi de l'argument dans une case de travail (pour usage différé) et dans le registre électronique F (pour usage immédiat). On envoie également dans une case de travail le nombre maximum de reprises autorisé.

Le calcul et contrôle se font selon CHET §6 avec les particularités suivantes pour la réduction à l'intervalle de base (0,1-1) qui ne sont pas décrites dans CHET.

Soit

$$x = (\text{sgn } x)\ X\ \ 10^{p}$$

où $X = \text{man } x$ (cfr. I, §5.2) est compris entre 0,1 et 1. D'autre part, l'altération -exp, donne

$$-\exp x = 0{,}1 . 10^{-p} = 10^{-p-1}$$

Donc

$$\frac{1}{x} = (\text{sgn } x)\ \frac{10}{X}\ (-\exp x) \qquad (1)$$

C'est cette expression qui est employée pour le calcul, déjà à la première approximation $10/X \cong a_0 X + b_0$, et les coefficients a_0 b_0 de CNFE sont donc multipliés par 10, mais rebaptisés a_0 b_0. C'est sur la première approximation ayant déjà l'exposant et le signe correct que se font les itérations, en utilisant E comme mémoire pour les résultats.

A la fin de la séquence, $1/x$ est aussi dans le registre E, ce qui permet d'obtenir $1/x^2$ au moyen du seul ordre supplémentaire $\times$ E.

Au cas où le contrôle ne se vérifie pas, le calcul peut être repris pas à pas en vérifiant les limites des erreurs relatives à chaque étape d'après CNFE § 6.

2.2. Détail des ordres

8265	1	$\rightarrow r_1$	x en case de travail
		$= F$	x en F
8270	2	$-^{*}\ \widehat{0003}$	
		$= r_2$	nombre de reprises autorisées
8275	3	+ man	première approximation
		+ F	
8280	4	$\times\ a_0$	
		$+\ b_0$	
8285	5	- exp	
		$\times$ F	
8290	6	+ sgn	
		$\times$ F	
8295	7	$\rightarrow$ E	$y_0 = (\text{sgn } x)\ (-\exp x)\ (a_0\ \ \text{man } x + b_0)$

		$\times$ (-F)	première itération
8300	8	$+ a_1$	
		$\times$ E	
8305	9	$\rightarrow$ E	$y_1 = y_0(a_1 - xy_0)$
		$\times$ (-F)	deuxième itération
8310	10	$+ a_2$	
		$\times$ E	
8315	11	$\rightarrow$ E	$y_2 = y_1(a_2 - xy_1)$
		$\times$ (-F)	troisième itération
8320	12	$+ a_3$	
		$\times$ E	
8325	13	$\rightarrow$ E	$y_3 = y_2(a_3 - xy_2)$
		$\times$ (-F)	quatrième itération
8330	14	$+ 2$	
		$\times$ E	
8335	15	$\rightarrow$ E	$y_4 = y_3(2 - xy_3)$
		$\times$ (-F)	dernière itération
8340	16	$+ 2$	
		$\times$ E	
8345	17	$\rightarrow$ E	$y = y_4(2 - xy_4)$
		$\times x_1$	contrôle
8350	18	$-^* 1$	
		$= F$	$F = (xy - 1)$
8355	19	$-^* T$	addition et soustraction
		$+$ mod	sans normalisation
8360	20	$+^* F$	$\omega = -T + \lvert xy - 1 \rvert$
		$=$ ch	si (-), on a $\lvert xy - 1 \rvert \leq T$ et contrôle correct

8365	21	+ E Rv- K	$\omega = y$ (mise du résultat dans le groupe calculateur) sortie
8370	22	= E	vide le groupe calculateur
8375	23	$\overset{\cdot}{\cdot}_1$ + F	remise de y en x qui a été utilisé à autre chose pendant le contrôle
8380	24	$+^* r_2$ $+^* \overset{\frown}{0001}$ $\rightarrow r_2$	contenu négatif de r_2 augmenté d'une unité (cfr. I § 9)
8385	25	= ch	si (-), reprise autorisée
-		Rv- 3	renvoi origine sous-séquence B
8390	26	{ Step { Alarme ordre vide	

3. Fonction $y = x^{-\frac{1}{2}}$

3.1. Réduction à l'intervalle (0,1 - 1)

Soit $x = X\,10^p$ avec X = man x, et p = exno x.

On a

$$y = \frac{1}{\sqrt{X}}\,10^{-p/2} = Y_0\,10^{-p/2\,-1}$$

où

$$Y_0 = 10/\sqrt{X}$$

est calculé par l'approximation $aX^2 + bX + c$ dont les coefficients sont ceux de CNFE multipliés par 10. On obtient y_0 dès la première approximation avec l'exposant correct par

$$y_0 = \begin{cases} Y_0 \cdot 10^{-p/2} - 1 & \text{pour p pair} \\ Y_0 \cdot 10^{-p/2} - 3/2 \sqrt{10} & \text{pour p impair} \end{cases}$$

Comme noex $z = 10^{z-1}$, l'argument à utiliser dans l'altération noex en vue du calcul de y_0 sera respectivement

$$z = -p/2 \qquad \text{pour p pair}$$

$$z = -p/2 - 1/2 \text{ pour p impair}$$

La soustraction éventuelle de 0,5 et la multiplication éventuelle par $\sqrt{10}$ se font dans une sous-séquence séparée (notée C fig.2) conditionnée par le signe de $g = -\text{ent}\,|f| + |f|$, où $f = -p/2$, puisque

$$g = \begin{cases} -0{,}0 \ldots & \text{p pair} \\ +0{,}5 \ldots & \text{p impair} \end{cases}$$

Dans la définition de g, la notation ent désigne la partie entière et se calcule comme expliqué au I,§3.6 équ(5). Certains calculs intermédiaires sur les exposants se font sans normalisation : il est en effet essentiel (1 § 5.4) que z soit obtenu avec l'exposant + 02 avant de lui appliquer l'altération noex (ordre 15a). C'est pour cette raison que la constante 1/2 est donnée sous la forme $0{,}05 \cdot 10^{01}$ (ordre 6b). A noter la nécessité de l'ordre q_a qui se manifeste dans le cas particulier p=0. Il faut en effet que g obtenu en 11b soit négatif tout comme pour tout autre exposant pair.

3.2. Structure de la séquence (fig.2)

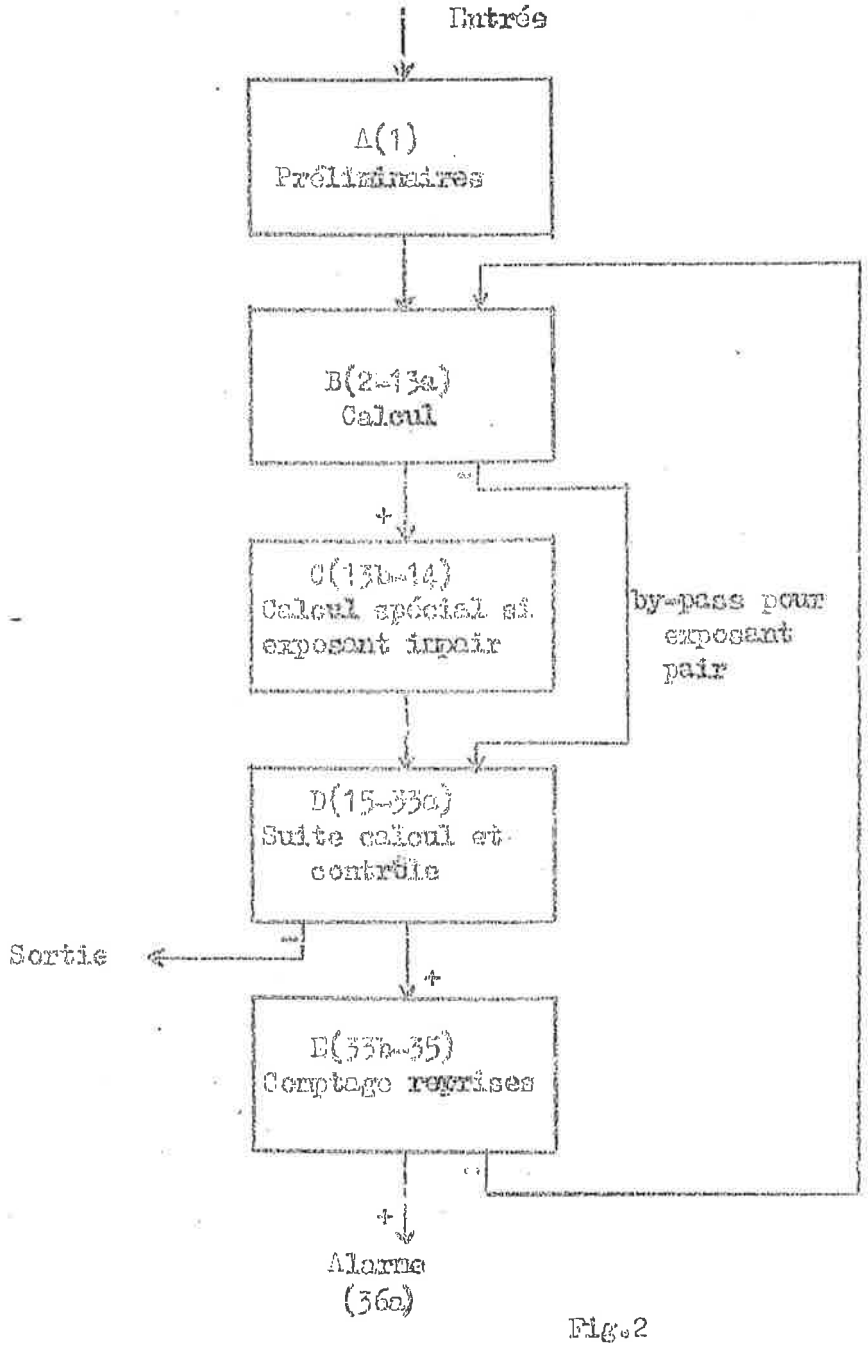

Fig.2

Le contrôle échoue à coup sûr si l'argument x était négatif.

Pour calculer $\sqrt{x}$ il suffit de faire suivre la séquence par l'ordre X x.

5.3. Calcul des prêts

8395	1	$= x_1$	[illegible]
		$\rightarrow^* \widehat{0003}$	nombre de reprises autorisées
8400	2	$= x_2$	[illegible]
		+ [illegible]	[illegible]
8405	3	$+ x_1$	
		$\rightarrow F$	
8410	4	$\times a$	
		$+ b$	
8415	5	$\times F$	
		$+ c$	
8420	6	$= E$	$Y_0 = aX^2 + bX + c$ avec $X = \text{[illegible]}$
		$\rightarrow^* ,03 \cdot 10^{01}$	calcul exponentiel
8425	7	exno	
		$\times x_1$	
8430	8	$= F$	$x = -\frac{b}{2}$ [illegible] + 02
		blanc	
8435	9	$\times \widehat{0000}$	[illegible]
		− mod	
8440	10	$+ F$	
		$+ ,00 \ldots 10^{15}$	
8445	11	+ mod	
		$+^* F$	$g = -\text{ent}\,\lvert x \rvert + {}^{*}\lvert x \rvert$
8450	12	$\rightarrow$ [illegible]	[illegible] est p pair
		$-,1 \cdot 10^{01}$	le gr. calc. contient $h = g \cdot 1$ [illegible] p pair
8455	13	R_v 15	[illegible]
		$+^* F$	

8460	14	= F	puisque p est impair, $z = h-p/2 = -1/2-p/2$ avec exposant + 02 — hypothèses pour p pair
		$-\sqrt{10}$	
8465	15	noex	le contenu du gr.calc.au début de la séquence est $\begin{cases} -\sqrt{10} \text{ pour p impair (13b)} \\ -1 \text{ pour p pair (11b)} \end{cases}$
		× F	F contient bien z obtenu en $\begin{cases} \text{(13a) pour p impair} \\ \text{(8a) pour p pair} \end{cases}$
8470	16	× (-E)	
		= E	$y_0 = -Y_0$ (noex z) $\begin{cases} (-1) \text{ pour p pair} \\ (-\sqrt{10}) \text{ pour p impair} \end{cases}$
8475	17	$-\widehat{5000}$	première itération
		× r_1	
8480	18	→F	
		× E	
8485	19	× E	
		÷ a_1	
8490	20	× E	
		→E	$y_1 = y_0 (a_1 - y_0^2 x/2)$
8495	21	× E	deuxième itération
		× F	
8500	22	÷ $\frac{3}{2}$	
		× E	
8505	23	→E	$y_2 = y_1 (3/2 - y_1^2 x/2)$
		× E	troisième itération
8510	24	× F	
		÷ $\frac{3}{2}$	
8515	25	× E	
		→E	$y_3 = y_2 (3/2 - y_2^2 x/2)$

8520	26	x E	dernière itération
		x F	
8525	27	$+ \frac{3}{2}$	
		x E	
8530	28	→ B	$y_4 = y_3 \, (3/2 - y_3^2 \, x/2)$
		x B	contrôle
8535	29	x x_1	
		$\rightarrow^{*}$ 0,1.10^{01}	
8540	30	= F	$y^2 x - 1$ en F
		$\rightarrow^{*}$ T	
8545	31	+ mod	
		$+^{*}$ F	
8550	32	= ch	si (-), correct car $\lvert y^2 x - 1 \rvert \leq \varepsilon$
		+ E	remise du résultat dans le gr.calc.
8555	33	Rv$^{\infty}$ K	sortie
		= E	vidange gr.calc.
8560	34	$+^{*}$ x_2	
		$+^{*}$ 0001	le gr.calc.contient x_2 négatif augmenté de 1 qui sera remis dans x_2 après renvoi à (2a).
8565	35	→ ch	(-) si reprise autorisée
		Rv$^{\infty}$ 2	
8570	36	stop alarme ordre vide	

Appendice III. Liste récapitulative des ordres

Les renvois indiquent le chapitre et paragraphe où la signification est expliquée en détail.

1. Préfixes des changements d'organisation du groupe calculateur

Agissent jusqu'à nouvel ordre; adresse vide

00011	virgule flottante	(I §2)
00021	virgule fixe	(II §1)
-00031	fin 1	(II §4)
-00041	fin 2	(II §4)
00014	bloc.al.	(I §11.1)
00024	débloc.al.	(I §11.1)
00034	eff.al.	(I §11.1)

2. Modification d'adresse ξ en ξ' par registre d'indice

$$\xi' = \begin{cases} \xi \\ \xi + Z_i \\ \xi + M \\ M + Z_i \end{cases} \quad \text{pour} \quad \begin{cases} i = 0 \\ i < 5 \\ i = 5 \\ i > 5 \end{cases}$$

(I §8.3)

(I §8.6.a) p. 22 cf registre M

3. Préfixes d'opération

04i16	+	(I §3)
04i66	+*	(I §3)
04i26	-	(I §3)
04i76	-*	(I §3)
04i36	×	(I §3)
04i86	⊗	(I §3)
04i35	× ..	(I §3)
04i85	⊗ ..	(I §3)

Remarque : pour l'addition de nombres positifs (pas de prise de complément) dans le cas où il n'y a pas de report en tête, on peut abréger le calcul en donnant 67 au lieu de 66 pour le A.C (gain = 1,6 msec !) c'est le cas pour l'opération d'addition de l'adresse comme nombre : 03 i 67 α β γ δ

Pour vider un registre ou une case de mémoire en étant certain que T_2 est bien vide, on peut supprimer l'ordre de vidange du A.C.
Ainsi 09 0 46 α β γ δ devient 09 0 00 α β γ δ.
Intéressant si plusieurs cases sont à vider, on ne donne d'ordre de vidange que pour le premier pas.
(Gain ≃ 800 μsec).

- 04i67	**	(II §4)
-04i77	**	
-04i37	**	(II §3)
-04i87	**	
- 04i68	OR1 +	(II §2)
- 04i78	OR1 ⇔	
-04i69	OR2 +	
- 04i79	OR2 ⇔	

Les préfixes commençant par 04i désignent une sélection de α à partir du tambour. Pour les autres cas, on a de même

021**	E	(I §3.2)
020**	F	
03i**	$\widehat{Z}_i$	(I §8.5)
025**	$\widehat{V}$	(III §6.1)
022**	$\widehat{V}$	(III §13)
023**	rub A sans piste aux.	(III §7.1)
028**	rub A avec piste aux.	
024**	rub B sans piste aux.	
029**	rub B avec piste aux.	

4. Préfixes d'inscription et d'impression

09i96	→	(I §4)
09i46	⇔	
- 09i47	⇔	(II §3)
- 00049	OR1° ⇔	(II §2)
- 00099	OR1° →	
- 09i49	OR2 ⇔	
- 09i99	OR2 →	

cf p. 9

Les préfixes commençant par 09i désignent une sélection sur le tambour. Pour les autres cas, on a de même

070**	F	(I §4)
071**	E	
074**	ch	(I §7)
081**	mab	(III §7.2)
091**	impr	(III §8.3)

ordre de vidange du groupe calculateur

07346		(I §12)

5. Affectations

L'adresse est toujours vide

00012	+ norm	(I §5.2)
00062	- norm	
00022	+ sgn	
00072	- sgn	
00032	+ mod	
00082	- mod	
00042	+ exp	
00092	- exp	
00013	noex	
00023	exno	
00033	dc	(II §1)
00043	tri	

6. Préfixes de renvoi

40100	Rv	(I §6)
42000	fin	

9. Repères sur les rubans (III § 4)

0002	fin de ruban (D.R)
0004	fin de groupe
0001	fin de tranche
0003	fin de tranche et fin de groupe

10. Déroulement de rubans jusque

92000	0000	fin ruban A	
91000	0000	fin groupe A	
95100	ξ	ξ'	
97000	0000	fin ruban B	(III § 5.1)
96000	0000	fin groupe B	
98100	ξ	ξ'	

11. Organisation transfert (III § 5.2)

95000	0000	jusque ft avec piste aux.
95000	0003	jusque ft sans piste aux.
95000	0001	jusque S avec piste aux.
95000	0006	jusque S sans piste aux.
95000	0004	un mot avec piste aux.
95000	0003	un mot sans piste aux.

12. Préfixes de transfert

50100	rub A → impr.	
55100	rub A → rub C	
51100	rub A → bandb.	
53100	rub B → impr.	(III § 5.3)
56100	rub B → rub C	
52100	rub B → bandb.	

13. Adresse de ruban xy** comme instruction typographique de mot (III § 8.2)

x = 2	dix retours de chariot
x = 4	deux retours de chariot
x = 5	un retour de chariot
y = 2	|| après le mot
y = 4	| après le mot
y = 5	|| avant le mot
y = 7	|| avant et après le mot
y = 9	|| avant et | après le mot

14. Instructions typographiques de symbole (III § 8.2)

0	impression normale
1	espace avant le symbole
2	point décimal avant le symbole
4	suppression
3	remplacement par espace

Dessin pag du [illegible]